

Планът - Student Management System

Ще изградим **Student Management System**, като постепенно добавяме:

1. 👤 Students — добавяне, преглед, редактиране и изтриване
2. 🔍 Search — търсене на студент
3. 📖 Courses/Subjects — предмети
4. 📝 Grades — оценки
5. 📊 Average Grade — автоматично изчисляване на среден успех
6. 📅 Attendance — присъствия
7. 👤 Student Details — подробна страница за студент
8. 🤖 По-добър интерфейс
9. 🗄 Database — ако текущият проект още не е настроен правилно за нея
10. 🚪 Login/roles — ако искаме системата да има администратор и потребители

Но няма да правим всичко наведнъж. Първо ще направим една малка част, ще я стартираш и ще проверим.

- 1.
2. <https://github.com/esp-nv/CSharp/tree/main/06.CSharp-EntityFramework/03.Entity-Relations/Author-Solutions/01-Student-System> може ли с теб да изградим проект asp.net core mvc

ChatGPT каза:

Да — **и според мен това е много добър проект за следващата ти стъпка.** Видях конкретния Student-System в repository-то ти. Той в момента е EF Core console-style проект с Data и StartUp.cs, така че можем да вземем **модела и релациите като основа**, но да изградим истинско **ASP.NET Core MVC приложение от нулата**, вместо просто да прехвърляме кода. **G**



GitHub

Microsoft също има актуална поредица за ASP.NET Core MVC + EF Core, включваща CRUD,



relationships, migrations и работа с related data. **M** Microsoft Learn+1

Какво предлагам да направим

Да направим проект примерно:

Student Management System

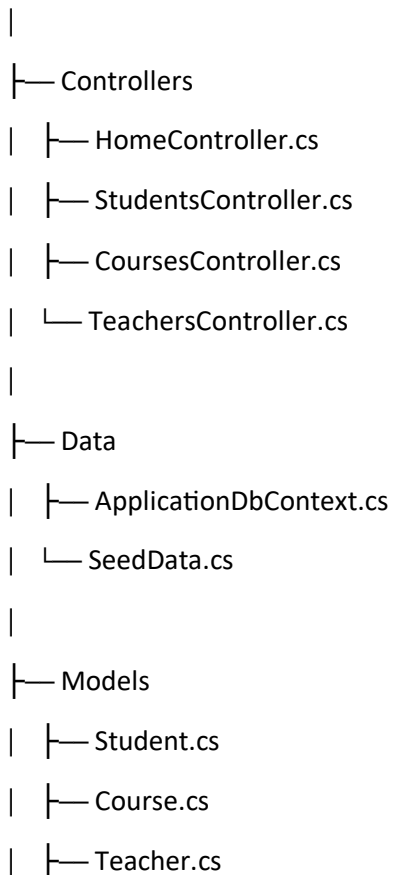
с:

- ASP.NET Core MVC
- C#
- Entity Framework Core
- SQL Server
- Razor Views
- Bootstrap
- Dependency Injection
- LINQ
- async/await
- CRUD
- Entity Relationships
- Validation
- Migrations
- ViewModels
- Search/Filtering
- Authentication/Authorization — на по-късен етап

Архитектура

Ще започнем сравнително просто:

StudentManagement



```

|   └─ Enrollment.cs
|
|
|   └─ ViewModels
|   |   └─ StudentViewModel.cs
|   |   └─ CourseViewModel.cs
|   |
|   |   └─ Views
|   |   |   └─ Home
|   |   |   └─ Students
|   |   |   └─ Courses
|   |   |   └─ Teachers
|   |   └─ Shared
|   |
|   |   └─ wwwroot
|   |   |   └─ css
|   |   |   └─ js
|   |   └─ lib
|   |
|   |   └─ appsettings.json
|   └─ Program.cs

```

Релациите

Тук вече ще използваме това, което си учил в Student-System.

Например:

Student

```

|
| 1
|
|
| *

```

Enrollment

```

|
| *
|
| 1

```

Course

Тоест:

Student → Enrollment → Course

Това е много полезно, защото няма да учиш MVC отделно от EF Core — ще видиш **как EF Core relationships се използват в реално web приложение.**

Искам да го направим заедно, не да ти дам готов проект

Това според мен е най-важната част.

Можем да работим така:

Аз:

Създай Student.cs с тези properties...

Ти го пишеш.

После ми показваш кода.

Аз:

Проверявам го → обяснявам всяка грешка → подобряваме го.

След това:

1. Create project



2. Models



3. DbContext



4. SQL Server



5. Migrations



6. Controllers



7. Views



8. CRUD



9. Relationships



10. ViewModels



11. Validation



12. Search



13. Authentication



14. Polish + GitHub

Така накрая няма просто да имаш проект, а ще можеш **да обясниш на интервю защо всяка част е написана по този начин.**

И още нещо важно

Не бих копирал директно стария Student-System.

Ще го използваме като **учебна основа**, но ще направим нов проект с актуален ASP.NET Core MVC. Това е по-смислено и за GitHub портфолиото ти.

Първата задача

Ако си готов, започваме **от нулата**.

Кажи ми само коя среда използваш:

- **Visual Studio 2022**
- **VS Code**
- **JetBrains Rider**

и коя версия на .NET имаш (dotnet --version).

После ще направим **Стъпка 1: създаване на ASP.NET Core MVC проекта**, като вървим бавно и обяснявам *защо* правим всяко нещо.

1. <https://github.com/esp-nv/StudentManagement> той е направен само трябва да се довърши но може стъпка и аз ще ти казвам ако съм готова, но може ли с коментари за да уча

ChatGPT каза:

Да, **точно така е по-добре**. Видях repository-то StudentManagement в GitHub — то вече е създадено като **ASP.NET Core MVC Student Management system**, така че няма смисъл да

създаваме нов проект.  GitHub

Ще го **довършим заедно стъпка по стъпка**, а не просто ще ти давам готов код.

И да — **ще слагаме коментари в кода**, но не прекалено много. Ще обясняваме важните неща, например:

```
// DbContext represents a session with the database.
```

```
// Through this class EF Core allows us to query and save entities.
```

```
public class ApplicationDbContext : DbContext
```

```
{  
}
```

После ще ти обяснявам **защо** е нужен DbContext, какво прави DbSet, защо използваме async, защо имаме Controller/View/Model и т.н.

Как ще работим

Ще имаме следния принцип:

Аз давам само една стъпка → ти я правиш → казваш „готово“ → проверяваме → продължаваме.

Ако получиш грешка, **не я заобикаляме**. Показваш ми грешката и я разбираме.

Целта е накрая да можеш сама да кажеш на интервю:

„Използвам EF Core за достъп до базата, Controller обработва HTTP request-a, View отговаря за представянето, а ViewModel ми позволява да не подавам директно Entity към View.“

Това е много по-ценно от това просто проектът да работи.

Стъпка 1 — Отваряме съществуващия проект

В **Visual Studio 2022**:

1. Отвори:

File → Open → Project/Solution

2. Намери папката:

StudentManagement

3. Отвори:

4. StudentManagement.sln

5. Изчакай Visual Studio да зареди проекта.

6. В **Solution Explorer** разгледай какво има вътре.

Не променяй нищо още.

Искам само да ми изпратиш **снимка на Solution Explorer-a** или да копираш тук структурата, която виждаш.

Например:

StudentManagement

|

|— Controllers

|— Models

|— Views

|— wwwroot

|— Data

|— Program.cs

└─ appsettings.json

Оттам ще направим **Стъпка 2: разбираме твоята текуща структура и проверяваме какво вече е направено**, преди да пишем какъвто и да е код.

2. да

ChatGPT каза:

Разбира се 😊 Кажете ми какво искате да направим.

3. Вие казахте:

тя е като репо

ChatGPT каза:

Ако имаш предвид „тя е като репо“ като израз — можеш да ми дадеш малко контекст какво искаш да кажеш с „репо“, за да ти отговоря точно.

4. Вие казахте:

<https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement>

ChatGPT каза:

Да, вече виждам кое имаш предвид. Това е репото esp-nv/StudentManagement.

В папката StudentManagement има стандартна **ASP.NET Core MVC** структура:

- Controllers
- Models
- Views
- wwwroot
- Program.cs
- appsettings.json

- StudentManagement.csproj  github.com

Ако това е проектът, за който каза „тя е като репо“, кажете ми какво точно искате да направим с него — например да го **разгледаме, обясним как работи, поправим грешка или да добавим функционалност**.



Източници

5. Вие казахте:

да добавим функционалност.

ChatGPT каза:

Да 👍 Можем да добавим функционалност директно върху този проект. Прегледах структурата на репото — то е ASP.NET Core MVC проект

с Controllers, Models, Views, wwwroot, Program.cs и .csproj.  GitHub

Кажете ми **каква функционалност трябва да добавим**. Например:

- 🔍 търсене на студенти
- 📊 оценки и изчисляване на среден успех
- 📅 присъствие/отсъствия
- 👤 профил на студент
- 📝 добавяне/редактиране/изтриване на студенти
- 🗝 login/register и роли
- 📖 предмети и записване на студенти към предмети
- 📈 dashboard със статистики
- 🖨 генериране на справка/документ
- или нещо конкретно от заданието ти

Ако имаш задание, просто го изпрати тук (текст или снимка) и ще го направим стъпка по стъпка върху точно това репо, включително кои файлове да промениш и готовия код.



Източници

6. Вие казахте:

но стъпка по стъпка

ChatGPT каза:

Разбира се. Ще го правим **много бавно и стъпка по стъпка**, без да прескачаме. Ще променяме по нещо, ще го пускаш и ще проверяваме дали работи.

Вече потвърдих структурата на проекта —

имаме Controllers, Models, Views, wwwroot, Program.cs и .csproj.  GitHub

Стъпка 1

Първо ми кажи **каква точно функционалност трябва да добавим**.

Ако имаш условие от преподавател/задание, **копирай го тук или изпрати снимка**.

След това ще ти кажа само:

1. кой файл да отвориш
2. какво точно да добавиш
3. как да го стартираш
4. какво трябва да видиш

И чак когато потвърдиш, че първата стъпка работи, продължаваме към **Стъпка 2**.



Източници

7. Вие казахте:

правим заедно с теб Student Management System

ChatGPT каза:

Да. Ще го направим **заедно от нулата, стъпка по стъпка**, като използваме точно твоето репо StudentManagement в GitHub.

В момента проектът има основната ASP.NET структура

— Controllers, Models, Views, wwwroot, Program.cs и т.н.  GitHub

Планът

Ще изградим **Student Management System**, като постепенно добавяме:

1.  Students — добавяне, преглед, редактиране и изтриване
2.  Search — търсене на студент
3.  Courses/Subjects — предмети
4.  Grades — оценки
5.  Average Grade — автоматично изчисляване на среден успех
6.  Attendance — присъствия
7.  Student Details — подробна страница за студент
8.  По-добър интерфейс
9.  Database — ако текущият проект още не е настроен правилно за нея
10.  Login/roles — ако искаме системата да има администратор и потребители

Но няма да правим всичко наведнъж. Първо ще направим една малка част, ще я стартираш и ще проверим.

Стъпка 1 — кажи ми само това

Отвори проекта в **Visual Studio** или **Visual Studio Code** и го стартирай с Ctrl + F5 / dotnet run.

След това ми кажи:

„**Стартира**“ — ако сайтът се отваря,

или

„**Дава грешка**“ — и ми копирай грешката.

Оттам започваме **Стъпка 2**.



Източници

8. Вие казахте:

готово

ChatGPT каза:

Супер!  Започваме.

Стъпка 2 — да видим какво вече имаме

Преди да пишем нов код, ще разгледаме съществуващия проект, за да не счупим нещо.

В **Solution Explorer** намери папката:

Models

и ми напиши **какви файлове има вътре**.

Например:

Models

└─ Student.cs

└─ ...

Само ми изпрати имената на файловете в Models. След това ще направим първата функционалност.

9. Вие казахте:

<https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement> ето линк

ChatGPT каза:

Да, точно този линк ми трябва. 👍 Проверих го директно и виждам структурата на проекта. В StudentManagement има Controllers, Models, Views, wwwroot, Program.cs и останалите

файлове.  GitHub

Нека **не гадаем какво има вътре**, а да започнем от реалния код.

Стъпка 2.1

Натисни в GitHub:

Models → Student.cs

ако има такъв файл.

Ако няма Student.cs, натисни Models и ми изпрати **какви файлове виждаш вътре**.

Може просто да ми копираш списъка, например:

Models

- Student.cs

- ErrorViewModel.cs

След това ще ти кажа **точно какво да направиш в първия файл**. Няма да прескачаме напред.



Източници

10. Вие казахте:

няма файл-а Student.cs

ChatGPT каза:

Точно така — тогава започваме от **модела** Student. Това е логичната първа стъпка за Student Management System. 👍

Стъпка 2 — създаваме Student.cs

Във Visual Studio:

1. В **Solution Explorer** намери папката **Models**.
2. Натисни **десен бутон** върху **Models**.
3. Избери **Add → Class...**
4. Име на файла:
5. **Student.cs**
6. Натисни **Add**.

В новия **Student.cs** ще има някакъв готов код. **Изтрий всичко** и постави:

```
using System.ComponentModel.DataAnnotations;
```

```
namespace StudentManagement.Models
```

```
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string FirstName { get; set; }  
  
        [Required]  
        public string LastName { get; set; }  
  
        [Required]  
        [EmailAddress]  
        public string Email { get; set; }  
  
        public string Course { get; set; }  
  
        public int Age { get; set; }  
    }  
}
```

Какво направихме?

Създадохме обект **Student**, който има:

- **Id** — уникален номер
- **FirstName** — име

- LastName — фамилия
- Email — имейл
- Course — курс/специалност
- Age — възраст

[Required] означава, че полето не трябва да бъде празно, а [EmailAddress] проверява дали имейлът е във валиден формат.

Сега не прави нищо друго. Запази файла с Ctrl + S и ми напиши „**готово**“. След това продължаваме със следващата стъпка.

11. Вие казахте:

готово и е качен в gitHub

ChatGPT каза:

Супер! 🎉 Значи първата част е готова и вече имаме Student.cs в GitHub.

Стъпка 3 — създаваме базата с данни

Сега ще направим така, че приложението да може да **съхранява студентите**.

Първо трябва да проверим дали проектът вече има DbContext.

Направи само това:

В **Solution Explorer** потърси папка или файл с име:

Data

или файл:

ApplicationDbContext.cs

или:

StudentManagementContext.cs

Ако намериш такъв, **не го променяй**.

Просто ми кажи какво намери, например:

Имам папка Data и вътре ApplicationDbContext.cs

или

Нямам нищо такова.

После ще направим **само следващата стъпка**. 🚀

12. Вие казахте:

Нямам нищо такова.

ChatGPT каза:

Перфектно. 👍 Тогава ще създадем базата сами.

Стъпка 3.1 — инсталираме Entity Framework Core

Ще използваме **Entity Framework Core**, за да може нашият Student модел да се записва в база данни.

Ако си във **Visual Studio**:

1. От горното меню избери **Tools → NuGet Package Manager → Manage NuGet Packages for Solution**.
2. Отвори **Browse**.
3. Потърси:
4. Microsoft.EntityFrameworkCore.SqlServer
5. Инсталирай пакета в проекта **StudentManagement**.

След това потърси и инсталирай:

Microsoft.EntityFrameworkCore.Tools

Важно

Не инсталирай още други пакети и **не променяй кода**.

Когато и двата пакета са инсталирани, ми напиши:

„Инсталирани са“

и продължаваме със следващата малка стъпка. 🚀

1. може ли десен бутон върху самия solution от там да избера Manage NuGet Packages for Solution.

ChatGPT каза:

Да, **точно така**. 👍 Даже това е удобен начин.

Направи следното:

1. В **Solution Explorer** натисни **десен бутон върху Solution StudentManagement**.
2. Избери:
Manage NuGet Packages for Solution...
3. В таба **Browse** потърси:
Microsoft.EntityFrameworkCore.SqlServer
4. Избери твоя проект StudentManagement и натисни **Install**.
5. После потърси:
Microsoft.EntityFrameworkCore.Tools
6. Отново избери проекта StudentManagement и натисни **Install**.

След инсталацията трябва да ги виждаш в:

Dependencies → Packages

като приблизително:

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Tools

Когато ги инсталираш, кажи ми **„готово“** и продължаваме. 🚀

1. Microsoft.EntityFrameworkCore.SqlServer дава версия 10.0.11

ChatGPT каза:

Да, това е нормално — **но не я инсталирай още.** 😊

Версия **10.0.11** е от новото поколение EF Core и е важно да съобразим версията с **.NET версията на твоя проект**. Ако проектът е например на .NET 8, по-добре ще използваме EF Core 8; ако е .NET 9 — EF Core 9 и т.н.

Направи само това:

В Solution Explorer натисни десен бутон върху проекта:

StudentManagement → **Properties**

После намери:

Target Framework

и ми кажи какво пише там.

Например:

.NET 8.0

или

.NET 9.0

или

.NET 10.0

Не инсталирай пакета още. Кажи ми Target Framework-а и ще ти кажа точно коя версия да избереш.

2. Вие казахте:

8.0

3. ChatGPT каза:

Перфектно 👍 Проектът е на **.NET 8.0**, затова ще използваме **EF Core 8**, а не 10.

Стъпка 3.1 — правилната версия

В **Manage NuGet Packages for Solution**:

1. Потърси:
Microsoft.EntityFrameworkCore.SqlServer
2. Намери падащото меню за **Version**.
3. Избери най-новата версия, започваща с **8.0** — например 8.0.x.
4. Отбележи проекта StudentManagement.
5. Натисни **Install**.

След това потърси:

Microsoft.EntityFrameworkCore.Tools

и пак избери **8.0.x** и инсталирай.

Важно

Не избирай:

10.0.11 ❌

Избираме:

8.0.x ✅

След като и двата пакета са инсталирани, **не прави следващата стъпка още**.

Напиши ми „**готово**“, когато приключиш.

4. Вие казахте:

8.0.30

5. ChatGPT каза:

Да! ✅ **8.0.30** е правилният избор за проект на **.NET 8.0**.

Избери **8.0.30** за:

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

и ги инсталирай в проекта **StudentManagement**.

След това провери в:

Dependencies → Packages

да виждаш нещо подобно:

Microsoft.EntityFrameworkCore.SqlServer 8.0.30

Microsoft.EntityFrameworkCore.Tools 8.0.30

Като ги видиш и двата, кажи „**готово**“. Тогава ще създадем ApplicationDbContext.cs. 🚀

6. Вие казахте:

готово

7. ChatGPT каза:

Супер! 🚀 Вече имаме модела Student и Entity Framework Core 8.0.30.

Стъпка 3.2 — създаваме ApplicationDbContext

Сега ще направим връзката между нашия Student клас и базата данни.

1. Създай папка Data

В **Solution Explorer**:

- десен бутон върху проекта **StudentManagement**
- **Add → New Folder**
- кръсти папката:

Data

2. Създай ApplicationDbContext.cs

Десен бутон върху новата папка **Data**:

Add → Class...

Име:

ApplicationDbContext.cs

3. Замени кода

Изтрий всичко във файла и постави:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagement.Data
```

```
{  
  
    public class ApplicationDbContext : DbContext  
    {  
  
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)  
            : base(options)  
        {  
  
        }  
  
        public DbSet<Student> Students { get; set; }  
    }  
}
```

Какво направихме?

Този ред:

```
public DbSet<Student> Students { get; set; }
```

казва на Entity Framework:

„В базата данни искам таблица за студентите.“

Следващата стъпка ще бъде да **свържем този DbContext с SQL Server**, но засега спираме тук.

Запази с **Ctrl + S** и ми кажи „**готово**“.

1. може ли стъпка по стъпка за начинаещ и с коментари

2. ChatGPT каза:

Разбира се. 🍌 Оттук нататък ще работим **като с човек, който тепърва започва с ASP.NET**, и няма да прескачаме стъпки.

Ще ти обяснявам:

- **къде да натиснеш;**
- **какво да създадеш;**

- какво да напишеш;
- какво прави всеки ред чрез коментари;
- и след всяка малка стъпка ще спираме, за да провериш.

Стъпка 3.2 — папката Data

В момента трябва да имаш нещо подобно:

StudentManagement

```
|
|— Controllers
|— Models
| |— ErrorViewModel.cs
| |— Student.cs
|— Views
|— wwwroot
|— Program.cs
|— ...
```

1. Създаваме папка

В **Solution Explorer**:

- Намери проекта **StudentManagement**.
- Натисни върху него с **десен бутон**.
- Избери **Add → New Folder**.
- Напиши:

Data

- Натисни **Enter**.

Вече трябва да виждаш:

StudentManagement

```
|
|— Controllers
|— Data    ← новата папка
|— Models
|— Views
|— wwwroot
```

Спираме тук 

Не създавай файла още.

Напиши ми само „**имам Data**“, когато папката е създадена. След това ще направим **само файла** ApplicationDbContext.cs и ще ти обясня ред по ред какво означава.

1.

```
<Project Sdk="Microsoft.NET.Sdk.Web"> <PropertyGroup>
  <TargetFramework>net8.0</TargetFramework> <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings> </PropertyGroup> <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.30" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.30">
      <PrivateAssets>all</PrivateAssets> <IncludeAssets>runtime; build; native; contentfiles;
      analyzers; buildtransitive</IncludeAssets> </PackageReference> </ItemGroup> <ItemGroup>
      <Folder Include="Data\" /> </ItemGroup> </Project>
```

2. ChatGPT каза:

Да — **точно така трябва да изглежда**. 👍 Виждам, че:

- проектът е на **.NET 8**;
- Microsoft.EntityFrameworkCore.SqlServer е **8.0.30**;
- Microsoft.EntityFrameworkCore.Tools е **8.0.30**;
- папката Data вече е създадена.

Тоест сме готови за следващата малка стъпка.

Стъпка 3.3 — създаваме ApplicationDbContext.cs

Сера **само това**:

5. В Solution Explorer намери папката **Data**.
6. Десен бутон върху Data.
7. Избери **Add → Class...**
8. Напиши име:

ApplicationDbContext.cs

5. Натисни **Add**.

Вътре ще се появи някакъв код.

Изтрий кода

Изтрий всичко, което Visual Studio е сложило автоматично.

След това постави този код:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagement.Data
```

```
{
```

```
    // Този клас управлява връзката между нашето приложение
```

```
    // и базата данни.
```

```
    public class ApplicationDbContext : DbContext
```

```

{
    // Конструкторът получава настройките за базата данни.
    public ApplicationDbContext(
        DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    // Това представлява таблицата Students в базата данни.
    // Всеки запис в нея ще бъде един Student.
    public DbSet<Student> Students { get; set; }
}

```

Какво означава най-важната част?

Това:

```
public DbSet<Student> Students { get; set; }
```

означава приблизително:

„Искам в базата данни да имам колекция/таблица, която съдържа Student обекти.“

Тоест по-късно ще получим нещо подобно:

Students

```

-----
Id | FirstName | LastName | Email | Course | Age
-----

```

```
1 | Ivan   | Petrov | ... | IT   | 20
```

```
2 | Maria  | Ivanova | ... | CS   | 21
```

Не се притеснявай, ако все още не виждаш тази таблица. На този етап само сме описали как трябва да изглежда тя.

Сега спираме. 

Запази с **Ctrl + S**.

След това ми кажи само:

„ApplicationDbContext.cs е готов“

И чак тогава ще направим следващата стъпка — ще свържем ApplicationDbContext с приложението в Program.cs.

1. кода които ще слагаме там да няма коментари а те да са отвън
2. ChatGPT каза:

Да, разбира се. 👍 Даже за начинаещ е по-удобно така.

Оттук нататък ще правим така:

Във файла — само чист код:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagement.Data
```

```
{
```

```
    public class ApplicationDbContext : DbContext
```

```
    {
```

```
        public ApplicationDbContext(
```

```
            DbContextOptions<ApplicationDbContext> options)
```

```
            : base(options)
```

```
        {
```

```
        }
```

```
        public DbSet<Student> Students { get; set; }
```

```
    }
```

```
}
```

А извън кода — аз ще обяснявам подробно, например:

- using Microsoft.EntityFrameworkCore; — позволява ни да използваме Entity Framework Core.
- using StudentManagement.Models; — позволява на този файл да вижда класа Student.
- ApplicationDbContext — класът, който ще управлява връзката с базата данни.
- DbSet<Student> Students — казва, че ще имаме таблица за студентите.

Така кодът ще остане **чист и професионален**, а обясненията ще са отделно.

Ще спазваме това правило за **всички следващи стъпки**. ✅

3. Вие казахте:

така става

4. ChatGPT каза:

Да, точно така. 👍

Продължаваме по този начин:

1. Давам ти **една малка стъпка**.

2. Давам ти **чист код без коментари**.
3. След кода обяснявам **всеки важен ред на разбираем език**.
4. Ти го правиш и ми казваш „**готово**“.
5. Едва тогава минаваме към следващата стъпка.

В момента сме на ApplicationDbContext.cs. Ако вече си поставил кода и си го запазил, следващото ще бъде **свързването му с Program.cs**.

Кажете „**готово**“ и продължаваме.

1. може ли и още едо уточнение само ако се налага чрез `/// <summary> /// /// </summary>`

2. ChatGPT каза:

Да, разбира се. 👍 Това даже е **добра практика** в C#.

Оттук нататък ще използваме `/// <summary>` **само когато наистина има смисъл** — например за класове, публични методи или свойства, които е полезно да имат документация.

Няма да слагаме излишни коментари на всеки ред.

Например:

```
/// <summary>
/// Представява връзката между приложението и базата данни.
/// </summary>
public class ApplicationDbContext : DbContext
{
    /// <summary>
    /// Създава нова инстанция на ApplicationDbContext.
    /// </summary>
    public ApplicationDbContext(
        DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

А когато не е необходимо описание, ще оставяме кода чист.

Нашето правило оттук нататък:

- ❌ Без `//` коментари вътре в кода, освен ако наистина са необходими.
- ✅ `/// <summary>` — само когато е полезно.

-  Основните обяснения — **извън кода**, стъпка по стъпка.
-  Няма да ти давам огромни парчета код наведнъж.

Готови сме да продължим с ApplicationDbContext.cs → Program.cs. 

1. да

2. **ChatGPT каза:**

Супер.  Продължаваме **бавно и само с една стъпка**.

Стъпка 3.4 — свързваме базата с Program.cs

Вече имаме:

Data

└─ ApplicationDbContext.cs

Сега трябва да кажем на ASP.NET Core:

„Използвай ApplicationDbContext и SQL Server.“

1. Отвори Program.cs

В **Solution Explorer** намери:

Program.cs

Отвори го.

Вероятно вътре вече имаш код, подобен на този:

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
// ...
```

Не изтривай целия файл.

2. Добавяме using

Най-отгоре на Program.cs добави:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

Това позволява на Program.cs да използва:

- `EntityFrameworkCore`
- нашия `ApplicationDbContext`

3. Добавяме регистрацията на базата

Намери този ред:

```
builder.Services.AddControllersWithViews();
```

Непосредствено преди него добави:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

Така началото на Program.cs трябва да изглежда приблизително така:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
builder.Services.AddControllersWithViews();
```

Какво направихме?

Тази част:

```
builder.Services.AddDbContext<ApplicationDbContext>
```

регистрира нашия ApplicationDbContext в ASP.NET Core.

А тази:

```
options.UseSqlServer(...)
```

казва:

„Ще използваме Microsoft SQL Server като база данни.“

А това:

```
builder.Configuration.GetConnectionString("DefaultConnection")
```

казва:

„Вземи настройките за връзка от appsettings.json, от connection string с име DefaultConnection.“

 **Все още нямаме DefaultConnection в appsettings.json.** Това е нарочно — ще го направим в следващата стъпка.

Сега спираме. 

Запази Program.cs с **Ctrl + S**.

Ако Visual Studio показва червени линии, **не се притеснявай** — възможно е да е заради липсващия connection string.

Кажете ми **„готово“** и ще направим следващата стъпка: appsettings.json.

3. Вие казахте:

```
namespace StudentManagement { public class Program { public static void Main(string[] args) { var
builder = WebApplication.CreateBuilder(args); // Add services to the container.
builder.Services.AddControllersWithViews(); var app = builder.Build(); // Configure the HTTP request
pipeline. if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error"); // The
default HSTS value is 30 days. You may want to change this for production scenarios, see
https://aka.ms/aspnetcore-hsts. app.UseHsts(); } app.UseHttpsRedirection(); app.UseStaticFiles();
app.UseRouting(); app.UseAuthorization(); app.MapControllerRoute( name: "default", pattern:
"{controller=Home}/{action=Index}/{id?}"); app.Run(); } } } имам това
```

4. ChatGPT каза:

Да 👍 Това е напълно нормален Program.cs за ASP.NET Core MVC. Няма проблем, че при теб е с namespace StudentManagement и class Program.

Ще работим **върху твоя код**, без да го пренаписваме излишно.

Стъпка 3.4 — първо добавяме using

В самото начало на файла, **преди**:

```
namespace StudentManagement
```

добави:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

Така началото трябва да стане:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

```
namespace StudentManagement
```

```
{
```

```
    public class Program
```

```
    {
```

Защо?

Microsoft.EntityFrameworkCore ни дава достъп до Entity Framework Core.

StudentManagement.Data ни позволява да използваме нашия:

```
ApplicationDbContext
```

Стъпка 3.5 — добавяме връзката с базата

Сега намери:

```
builder.Services.AddControllersWithViews();
```

Непосредствено над него постави:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
```

```
options.UseSqlServer(  
    builder.Configuration.GetConnectionString("DefaultConnection")));
```

Следователно тази част от твоя Program.cs трябва да изглежда така:

```
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlServer(  
        builder.Configuration.GetConnectionString("DefaultConnection")));  
  
builder.Services.AddControllersWithViews();
```

Какво означава това?

AddDbContext<ApplicationDbContext> казва на приложението:

Използвай нашия ApplicationDbContext.

UseSqlServer казва:

Базата данни ще бъде SQL Server.

GetConnectionString("DefaultConnection") казва:

Вземи информацията за връзката от appsettings.json, където по-късно ще създадем DefaultConnection.

Важно

При теб има готови коментари като:

```
// Add services to the container.
```

и:

```
// Configure the HTTP request pipeline.
```

Засега не ги пипай. Те са стандартни коментари, генерирани от Visual Studio. Когато стигнем до почистване на проекта, ще решим кои да оставим.

Също така **не променяй нищо друго** в Program.cs.

Направи само тези две промени и натисни **Ctrl + S**.

После ми покажи **целия** Program.cs, както го направи сега, за да го проверим заедно, преди да продължим.

1. Вие казахте:

```
using Microsoft.EntityFrameworkCore; using StudentManagement.Data; namespace  
StudentManagement { public class Program { public static void Main(string[] args) { var builder =  
WebApplication.CreateBuilder(args); builder.Services.AddDbContext<ApplicationDbContext>(options  
=> options.UseSqlServer( builder.Configuration.GetConnectionString("DefaultConnection"))); // Add  
services to the container. builder.Services.AddControllersWithViews(); var app = builder.Build(); //  
Configure the HTTP request pipeline. if (!app.Environment.IsDevelopment()) {
```

```
app.UseExceptionHandler("/Home/Error"); // The default HSTS value is 30 days. You may want to
change this for production scenarios, see https://aka.ms/aspnetcore-hsts. app.UseHsts(); }
app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting(); app.UseAuthorization();
app.MapControllerRoute( name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run(); } }
```

2. ChatGPT каза:

Да — **кодът ти е правилен**.  Няма грешка в добавената част.

Само бих оправил **подравняването**, за да бъде по-четим. Функционално е същото.

Твоят Program.cs с по-добро форматиране

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

namespace StudentManagement
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            builder.Services.AddDbContext<ApplicationDbContext>(options =>
                options.UseSqlServer(
                    builder.Configuration.GetConnectionString("DefaultConnection")));

            // Add services to the container.
            builder.Services.AddControllersWithViews();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (!app.Environment.IsDevelopment())
            {
                app.UseExceptionHandler("/Home/Error");

                // The default HSTS value is 30 days. You may want to change this for production scenarios,
                see https://aka.ms/aspnetcore-hsts.

                app.UseHsts();
            }
        }
    }
}
```

```

    }

    app.UseHttpsRedirection();
    app.UseStaticFiles();

    app.UseRouting();

    app.UseAuthorization();

    app.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");

    app.Run();
}
}
}

```

Какво проверихме?

Имаш правилно:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

и:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
```

```
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

Това е връзката между **ASP.NET Core → Entity Framework Core → SQL Server**.

 Има само едно нещо: DefaultConnection **все още не съществува**. Това ще го направим в appsettings.json.

Следваща стъпка

Не променяй нищо повече в Program.cs.

Запази файла с **Ctrl + S** и ми кажи „**готово**“.

После ще отворим appsettings.json и ще направим **само connection string-a**, като ще ти обясня какво представлява.

1. готово
2. ChatGPT каза:

Супер! 👍 Program.cs е готов.

Стъпка 3.6 — appsettings.json

Сега ще кажем на приложението **къде се намира SQL Server базата**.

1. Намери файла

В **Solution Explorer** намери:

appsettings.json

Обикновено е директно в проекта:

StudentManagement

|

|— Controllers

|— Data

|— Models

|— Views

|— wwwroot

|— appsettings.json ← този файл

|— Program.cs

|— StudentManagement.csproj

Отвори appsettings.json.

2. Покажи ми какво има вътре

Засега нищо не променяй.

Копирай съдържанието на appsettings.json и го изпрати тук.

Ще го погледнем заедно и **след това** ще добавим DefaultConnection. Така няма да рискуваме да изтрием нещо, което вече е настроено. 👍

1. Вие казахте:

```
{ "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } },  
  "AllowedHosts": "*" }
```

2. ChatGPT каза:

Перфектно. 👍 Файлът е чист и можем спокойно да добавим connection string.

Стъпка 3.7 — добавяме ConnectionStrings

В твоя appsettings.json в момента имаш:

```
{
```

```
  "Logging": {
```

```
    "LogLevel": {
```

```
      "Default": "Information",
```

```
    "Microsoft.AspNetCore": "Warning"
  }
},
"AllowedHosts": "*"
}
```

Ще добавим ConnectionStrings **между** Logging и AllowedHosts.

Замени съдържанието на файла с:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=StudentManagementDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "AllowedHosts": "*"
}
```

Какво направихме?

Тази част:

```
"ConnectionStrings": {
```

съдържа настройките за връзка с базата.

Тази част:

```
"DefaultConnection":
```

е името на връзката.

То е **същото име**, което използвахме в Program.cs:

```
builder.Configuration.GetConnectionString("DefaultConnection")
```

А това:

```
Server=(localdb)\\mssqllocaldb
```

казва, че ще използваме **SQL Server LocalDB**, който обикновено се използва за локална разработка с Visual Studio.

Това:

Database=StudentManagementDb

е името на нашата база данни.

И така:

ASP.NET Core



ApplicationDbContext



SQL Server LocalDB



StudentManagementDb



Важна проверка

След като поставиш кода:

1. Натисни **Ctrl + S**.
2. Виж дали appsettings.json има **червени вълнообразни линии**.
3. Засега **не стартирай миграция и не създавай база**.

Кажете ми само „**готово**“, ако няма грешки, и следващата ни стъпка ще бъде да проверим дали проектът се **компилира успешно** преди да създаваме базата.

1. готово
2. **ChatGPT каза:**

Супер! 🙌 Вече имаме трите основни части:

Student



ApplicationDbContext



Connection String

Сега **няма да правим база данни още**. Първо ще проверим дали всичко, което направихме досега, се компилира.

Стъпка 3.8 — проверяваме проекта

Във Visual Studio:

1. От менюто избери **Build**.
2. Натисни **Build Solution**.

Или използвай клавишната комбинация:

Ctrl + Shift + B

Какво трябва да видиш?

В долната част на Visual Studio ще се появи:

Build succeeded.

Ако видиш:

Build succeeded.

0 Error(s)

значи всичко дотук е наред. 

Ако има грешки

Не се опитвай да ги поправяш сам.

Копирай ми грешката **точно както я показва Visual Studio** и ще я разгледаме заедно.

Засега не прави нищо друго.

Направи Ctrl + Shift + B и ми кажи какво показва **Build**.

1. Build started at 9:28 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>D:\GitHub\StudentManagement\StudentManagement\Models\Student.cs(10,23,10,32):
warning CS8618: Non-nullable property 'FirstName' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Student.cs(13,23,13,31):
warning CS8618: Non-nullable property 'LastName' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Student.cs(17,23,17,28):
warning CS8618: Non-nullable property 'Email' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Student.cs(19,23,19,29):
warning CS8618: Non-nullable property 'Course' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll 1>Done building project "StudentManagement.csproj". ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:28 PM and took 05.416 seconds =====

2. ChatGPT каза:

Перфектно!  **Проектът се компилира успешно.**

Най-важният ред е:

Build: 1 succeeded, 0 failed

Тоест **нямаме грешки**. Имаме само 4 предупреждения (warning).

Какви са предупрежденията?

Те са в Student.cs и са за:

- FirstName
- LastName
- Email

- Course

Причината е, че в проекта сме включили:

```
<Nullable>enable</Nullable>
```

Това кара C# да проверява дали string свойствата могат да бъдат null.

Например:

```
public string FirstName { get; set; }
```

C# казва:

„Ти ми казваш, че FirstName никога няма да бъде null, но не си дал начална стойност.“

Ще го поправим ли?

Да, но нека го направим сега, преди да продължим. Това е добра практика и ще изчистим предупрежденията.

Отвори:

Models

└─ Student.cs

и ми изпрати **текущото съдържание на Student.cs**.

Ще го коригираме заедно, като запазим правилото ни: **чист код без обикновени коментари, а обясненията — извън кода**.

1.

```
using System.ComponentModel.DataAnnotations; namespace StudentManagement.Models {  
    public class Student { public int Id { get; set; } [Required] public string FirstName { get; set; }  
    [Required] public string LastName { get; set; } [Required] [EmailAddress] public string Email {  
        get; set; } public string Course { get; set; } public int Age { get; set; } }
```

2. ChatGPT каза:

Да, това е текущият ти Student.cs. 👍

Ще поправим предупрежденията по **най-лесния начин за начинаещ**.

Стъпка 3.9 — поправяме предупрежденията

Замени съдържанието на Student.cs с това:

```
using System.ComponentModel.DataAnnotations;  
  
namespace StudentManagement.Models  
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string FirstName { get; set; } = string.Empty;
```

[Required]

```
public string LastName { get; set; } = string.Empty;
```

[Required]

[EmailAddress]

```
public string Email { get; set; } = string.Empty;
```

```
public string Course { get; set; } = string.Empty;
```

```
public int Age { get; set; }
```

```
}
```

```
}
```

Какво променихме?

Например преди имаме:

```
public string FirstName { get; set; }
```

Сега имаме:

```
public string FirstName { get; set; } = string.Empty;
```

string.Empty означава **празен текст** ("").

Така казваме на C#:

„Това свойство започва с празна стойност, а не с null.“

Направихме същото с:

- FirstName
- LastName
- Email
- Course

Атрибутите:

[Required]

и:

[EmailAddress]

остават. Те са важни за валидацията на данните.

Сега направи само това

5. Замени кода.
6. Натисни **Ctrl + S**.

7. Натисни **Ctrl + Shift + B**.

8. Провери резултата.

Очакваме:

Build: 1 succeeded, 0 failed

и предупрежденията от Student.cs да са изчезнали.

Изпрати ми резултата от Build, както направи преди. После продължаваме. 🚀

1. Build started at 9:33 PM... ===== Build: 0 succeeded, 0 failed, 1 up-to-date, 0 skipped
===== Build completed at 9:33 PM and took 00.033 seconds =====

2. ChatGPT каза:

Перфектно! ✅ Това също е **успешен резултат**.

Този път Visual Studio казва:

Build: 0 succeeded, 0 failed, 1 up-to-date

Какво означава up-to-date?

Visual Studio е проверил проекта и е установил:

„Нищо не се е променило, което да изисква ново компилиране.“

Тоест:

- ❌ 0 failed → **нямаме грешки**
- ✅ 1 up-to-date → проектът вече е компилиран и няма нужда от повторно компилиране
- ✅ всичко е наред

Следваща стъпка — проверяваме с Rebuild

За да сме напълно сигурни след промяната в Student.cs, направи:

Build → Rebuild Solution

или натисни:

Ctrl + Alt + F7

Това ще компилира проекта **отначало**, вместо да използва вече компилираните файлове.

После ми изпрати резултата.

Очакваме нещо подобно:

Build: 1 succeeded, 0 failed

Ако няма warnings — още по-добре. 👍

3. Вие казахте:

```
using System.ComponentModel.DataAnnotations; namespace StudentManagement.Models { public class Student { public int Id { get; set; } [Required] public string FirstName { get; set; } = string.Empty; [Required] public string LastName { get; set; } = string.Empty; [Required] [EmailAddress] public string Email { get; set; } = string.Empty; public string Course { get; set; } = string.Empty; public int Age { get; set; } } }
```

4. ChatGPT каза:

Да — Student.cs **вече е напълно наред.** 

Точно този код искаме да имаме:

- Id — идентификатор на студента.
- FirstName — име.
- LastName — фамилия.
- Email — имейл.
- Course — курс/специалност.
- Age — възраст.
- [Required] — задължително поле.
- [EmailAddress] — проверява формата на имейла.
- = string.Empty — премахва предупрежденията за null.

Не променяй нищо повече в този файл.

Следващата ни цел

Вече имаме:

Student.cs



ApplicationDbContext.cs



Program.cs



appsettings.json

Следващата важна стъпка е да **създадем базата данни чрез Migration.**

Но преди това ще направим една кратка проверка, за да сме сигурни, че ApplicationDbContext.cs е точно както трябва.

👉 Изпрати ми **текущия код** от ApplicationDbContext.cs и ще го проверим заедно.

5. Вие казахте:

Build started at 9:35 PM... ===== Build: 0 succeeded, 0 failed, 1 up-to-date, 0 skipped
===== Build completed at 9:35 PM and took 00.029 seconds =====

6. ChatGPT каза:

Да, това е **ОК.** 

Build: 0 succeeded, 0 failed, 1 up-to-date

Най-важното е:

- **0 failed** → няма грешки.

- **1 up-to-date** → проектът вече е компилиран и Visual Studio няма какво ново да компилира.

Значи можем да продължим. 👍

Следваща стъпка — проверяваме ApplicationDbContext.cs

Преди да създадем базата, искам да проверя последната част от настройката.

Отвори:

Data

└─ ApplicationDbContext.cs

и ми копирай **целия код от файла** тук.

Ще го проверим ред по ред, и ако всичко е наред, ще преминем към **Migration**. 🚀

1.

```
using Microsoft.EntityFrameworkCore; using StudentManagement.Models; namespace StudentManagement.Data { public class ApplicationDbContext : DbContext { public ApplicationDbContext( DbContextOptions<ApplicationDbContext> options ) : base(options) { } public DbSet<Student> Students { get; set; } } }
```

2. ChatGPT каза:

Да, точно това искаме. ✅ ApplicationDbContext.cs е правилен.

Само ще направим **една малка подобрявка**, за да избегнем евентуално предупреждение за Students.

Стъпка 3.10 — малка поправка

Промени:

```
public DbSet<Student> Students { get; set; }
```

на:

```
public DbSet<Student> Students { get; set; } = null!;
```

Целият файл ще бъде:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagement.Data
```

```
{
```

```
    public class ApplicationDbContext : DbContext
```

```
    {
```

```
        public ApplicationDbContext(
```

```
            DbContextOptions<ApplicationDbContext> options)
```

```
            : base(options)
```

```
        {
```

```
        }
```

```
public DbSet<Student> Students { get; set; } = null!;  
}  
}
```

Защо = null!?

DbSet ще бъде създаден и зададен от **Entity Framework Core**.

C# обаче не знае това предварително и може да ни покаже предупреждение за null.

null! означава:

„Знам какво правя — не предупреждавай компилатора за тази стойност.“

Това е често използван подход при DbSet в Entity Framework Core.

Сега:

1. Направи промяната.
2. Натисни **Ctrl + S**.
3. Направи **Build → Rebuild Solution**.
4. Изпрати ми резултата.

След това, ако всичко е чисто, ще преминем към **първата Migration** и ще започнем реално да създаваме базата данни. 🚀

1. ctr+shift+b

2. ChatGPT каза:

Да 👍 Ctrl + Shift + B е **Build Solution**.

Направи сега:

1. Натисни **Ctrl + Shift + B**.
2. Изчакай да приключи.
3. Копирай резултата отдолу и ми го изпрати.

Ако получиш:

Build: 1 succeeded, 0 failed

или:

0 failed

значи сме добре. ✅

Не прави Migration още. Първо ще проверим резултата от Build.

3. Вие казахте:

Build started at 9:40 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug
Any CPU ----- 1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:40 PM and took 01.057 seconds =====

4. ChatGPT каза:

Перфектно! 🎉

Това е точно резултатът, който искаме:

Build: 1 succeeded, 0 failed

И най-хубавото — **нямаме никакви warnings**. ✅

Дотук сме направили:

Student.cs



ApplicationDbContext.cs



Program.cs



appsettings.json



Build успешно ✅

Стъпка 4 — създаваме базата данни

Сега ще използваме **Migration**.

Migration е начинът, по който Entity Framework Core превръща моделите ни (Student) в структура на база данни.

Първата Migration ще бъде нещо като:

Student модел



EF Core Migration



Students таблица



SQL Server база

Първо само отваряме конзолата

Във Visual Studio отиди на:

Tools → NuGet Package Manager → Package Manager Console

Ще се отвори прозорец в долната част на Visual Studio.

⚠️ **Не въвеждай команда още.**

Когато видиш конзолата с нещо подобно на:

PM>

ми напиши „отворих я“.

След това ще ти дам **само първата команда** и ще ти обясня какво прави.

1. отворих я

2. **ChatGPT каза:**

Супер. 👍 Сега ще направим **само първата Migration**.

Стъпка 4.1 — създаваме първата Migration

В **Package Manager Console** трябва да виждаш:

PM>

До него напиши точно:

Add-Migration InitialCreate

и натисни **Enter**.

Какво прави тази команда?

Add-Migration казва на Entity Framework Core:

„Виж какви модели имам в проекта и създай описание на промените, които трябва да направим в базата.“

InitialCreate е просто **името на Migration-a**.

Можехме да го наречем и CreateDatabase, но InitialCreate е стандартно и ясно име за първата Migration.

Какво трябва да се случи?

Ако всичко е наред, Visual Studio ще създаде нова папка:

Migrations

с няколко файла вътре.

Например:

Migrations

├─ 20260829xxxxxx_InitialCreate.cs

├─ 20260829xxxxxx_InitialCreate.Designer.cs

└─ ApplicationDbContextModelSnapshot.cs

Не променяй тези файлове ръчно.



След командата

Не изпълнявай друга команда.

Копирай ми **целия резултат**, който се появи в Package Manager Console след:

Add-Migration InitialCreate

Ако има грешка — няма проблем. Ще я разгледаме заедно.

3. **Вие казахте:**

може ли да настроим и Microsoft SQL Server Manager - аз използвам 22 и connection string-a ,
server name->DESKTOP-O6FIRML\SQLEXPRESS022

4. ChatGPT каза:

Да, може — и според мен е по-добре да го настроим с твоя SQL Server, вместо с LocalDB. 👍

Твоят SQL Server instance е:

DESKTOP-O6FIRML\SQLEXPRESS022

И понеже използваш SQL Server Management Studio 22, ще настроим приложението да използва този SQL Server.

Важно преди да продължим

Не изпълнявай Add-Migration **още**. Първо ще сменим connection string-a.

Стъпка 4.1 — променяме appsettings.json

Отвори:

appsettings.json

В момента имаш нещо подобно:

```
"ConnectionStrings": {  
  "DefaultConnection":  
    "Server=(localdb)\\mssqllocaldb;Database=StudentManagementDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"  
}
```

Замени **само стойността на DefaultConnection** с:

```
"ConnectionStrings": {  
  "DefaultConnection": "Server=DESKTOP-O6FIRML\SQLEXPRESS022;Database=StudentManagementDb;Trusted_Connection=True;TrustServerCertificate=True"  
}
```

Какво означава?

Server=DESKTOP-O6FIRML\SQLEXPRESS022

казва на приложението:

Свържи се с SQL Server instance SQLEXPRESS022 на моя компютър.

Database=StudentManagementDb

казва:

Базата, която ще използваме, се казва StudentManagementDb.

Trusted_Connection=True

означава:

Използвай Windows удостоверяването, с което си влязъл в Windows.

TrustServerCertificate=True

позволява на локалната връзка да приеме сертификата на SQL Server.

Защо има \\?

В JSON единичната обратна наклонена черта \ е специален символ.

Затова:

DESKTOP-O6FIRML\SQLEXPRESS022

се записва в JSON като:

DESKTOP-O6FIRML\\SQLEXPRESS022

Но реалното име на SQL Server остава:

DESKTOP-O6FIRML\SQLEXPRESS022



Засега спираме тук

Не изпълнявай Add-Migration още.

Направи промяната в appsettings.json, запази с **Ctrl + S** и ми кажи:

„готово“

След това ще направим **проверка на връзката към** DESKTOP-O6FIRML\SQLEXPRESS022, преди да създаваме Migration-a.

1. DESKTOP-O6FIRML\SQLEXPRESS02 ми е сървара
2. **ChatGPT каза:**

Да, разбрах. 👍 Тогава имаме **SQL Server Express**, чийто Server Name е:

DESKTOP-O6FIRML\SQLEXPRESS02

Но щом каза, че **засега искаш да работим локално**, има два различни варианта:

- **LocalDB** — (localdb)\mssqllocaldb
- **Твоят SQL Server Express** — DESKTOP-O6FIRML\SQLEXPRESS02

И двата са локални на твоя компютър. Ако искаш базата да се вижда директно в **SQL Server Management Studio 22**, тогава е по-добре да използваме **твоя SQL Server Express**.

Ако изберем SQL Server Express

В appsettings.json ще бъде:

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "ConnectionStrings": {
```

```
"DefaultConnection": "Server=DESKTOP-
O6FIRML\\SQLEXPRESS02;Database=StudentManagementDb;Trusted_Connection=True;TrustServer
Certificate=True"

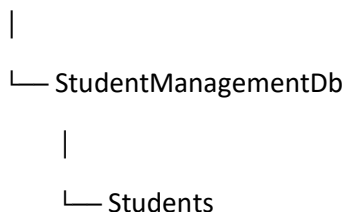
},

"AllowedHosts": "*"

}
```

Това ще създаде/използва база:

DESKTOP-O6FIRML\\SQLEXPRESS02



Моят съвет: щом вече имаш SQL Server Express и искаш да използваш SSMS 22, нека използваме него. Така после ще можеш да отваряш StudentManagementDb директно в SQL Server Management Studio и да виждаш таблицата Students.

Ако си съгласен, **следващата ни стъпка е само да сменим connection string-а** — няма да пускаме Migration още.

3. Вие казахте:

смених го { "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }, "ConnectionStrings": { "DefaultConnection": "Server=DESKTOP-O6FIRML\\SQLEXPRESS02;Database=StudentManagementDb;Trusted_Connection=True;TrustServerCertificate=True" }, "AllowedHosts": "*" }

4. ChatGPT каза:

Да.  Точно така трябва да изглежда appsettings.json.

Връзката вече е:

ASP.NET Core



Entity Framework Core



DESKTOP-O6FIRML\\SQLEXPRESS02



StudentManagementDb

Нека проверим нещо важно

Преди да създаваме Migration, искам да се уверим, че **SQL Server instance-ът наистина се казва точно:**

DESKTOP-O6FIRML\\SQLEXPRESS02

В **SQL Server Management Studio 22** при свързване трябва да имаш:

- **Server type:** Database Engine
- **Server name:** DESKTOP-O6FIRML\SQLEXPRESS02
- **Authentication:** това, с което обичайно се свързваш

Ако вече си се свързал успешно с този Server Name в SSMS — супер. 👍

Следващата малка стъпка

Върни се във Visual Studio → **Package Manager Console**.

Провери най-долу полето:

Default project

То трябва да е:

StudentManagement

⚠ **Все още не изпълнявай команда.**

Кажете ми какво пише при **Default project**.

1. Вие казахте:

Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any dependencies. Package Manager Console Host Version 6.14.3.1 Type 'get-help NuGet' to see all available NuGet commands. PM>

2. ChatGPT каза:

Да, това е **нормално**. 👍 Това е стандартният текст, който Visual Studio показва при отваряне на **Package Manager Console**.

Най-важното е, че долу виждаш:

PM>

Това означава, че конзолата е готова за команда.

Стъпка 4.1 — проверяваме Default project

В горната част на **Package Manager Console** трябва да има падащо меню:

Default project:

Провери дали е избрано:

StudentManagement

Ако е избрано — **не въвеждай нищо в PM> още**.

Кажете ми само:

„StudentManagement е избран“

и тогава ще изпълним първата Migration команда. 🚀

1. избран

2. ChatGPT каза:

Супер! 👍 Всичко е настроено правилно.

Стъпка 4.2 — създаваме първата Migration

В **Package Manager Console**, където пише:

PM>

въведи:

Add-Migration InitialCreate

и натисни **Enter**.

Какво ще направи командата?

Entity Framework Core ще вземе нашия модел:

Student.cs

и ще създаде инструкции за базата данни.

Ще се появи папка:

Migrations

с файлове вътре.

Не променяй тези файлове ръчно.



След командата

Не изпълнявай друга команда.

Копирай ми **целия резултат от Package Manager Console**, който се появи след:

Add-Migration InitialCreate

Ако има грешка — спокойно. Ще я разгледаме стъпка по стъпка.

1. PM> Add-Migration InitialCreate Build started... Build succeeded. To undo this action, use Remove-Migration. PM>

2. ChatGPT каза:

Перфектно! 🎉 **Migration-ът е създаден успешно.**

Това:

Build succeeded.

To undo this action, use Remove-Migration.

означава, че Entity Framework Core е успял да анализира Student модела и да създаде Migration-a.

Какво имаме вече?

В Solution Explorer би трябвало да се е появила папка:

StudentManagement

|

|— Controllers

```
├─ Data
|   └─ ApplicationDbContext.cs
├─ Migrations    ← нова папка
|   ├── ...InitialCreate.cs
|   ├── ...InitialCreate.Designer.cs
|   └─ ApplicationDbContextModelSnapshot.cs
├─ Models
|   └─ Student.cs
├─ Views
├─ wwwroot
├─ appsettings.json
└─ Program.cs
```

Не променяй нищо в Migrations.

Какво следва?

В момента Migration-ът е само **описание на промените**. Базата данни все още трябва да бъде създадена.

Следващата команда ще бъде:

Update-Database

Тя ще каже на Entity Framework:

„Изпълни Migration-а върху SQL Server и създай StudentManagementDb.“

Но още не я изпълнявай. 

Първо отвори **SQL Server Management Studio 22** и провери дали можеш да се свържеш към:

DESKTOP-O6FIRML\SQLEXPRESS02

Ако вече си свързан — кажи ми „свързан съм“ и ще направим Update-Database заедно.

1. Вие казахте:

ок

2. ChatGPT каза:

Супер. 👍 Нека продължим **само с една стъпка**.

Стъпка 4.3 — Update-Database

Върни се във Visual Studio → **Package Manager Console**.

Трябва да виждаш:

PM>

Въведи:

Update-Database

и натисни **Enter**.

Какво ще направи?

Тази команда ще вземе Migration-а:

InitialCreate

и ще го приложи към твоя SQL Server:

DESKTOP-O6FIRML\SQLEXPRESS02

Ще се създаде:

StudentManagementDb

а вътре ще има таблица за студентите.

Важно

След като натиснеш Enter, **не изпълнявай друга команда**.

Копирай ми целия резултат от:

PM> Update-Database

Ако е успешно — ще проверим базата в **SQL Server Management Studio 22**. Ако има грешка — ще я оправим заедно.

1. PM> Update-Database Build started... Build succeeded.
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (242ms) [Parameters=[], CommandType='Text', CommandTimeout='60'] CREATE DATABASE [StudentManagementDb]; Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (54ms) [Parameters=[], CommandType='Text', CommandTimeout='60'] IF SERVERPROPERTY('EngineEdition') <> 5 BEGIN ALTER DATABASE [StudentManagementDb] SET READ_COMMITTED_SNAPSHOT ON; END; Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (5ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] SELECT 1 Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (9ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] CREATE TABLE [__EFMigrationsHistory] ([MigrationId] nvarchar(150) NOT NULL, [ProductVersion] nvarchar(32) NOT NULL, CONSTRAINT [PK__EFMigrationsHistory] PRIMARY KEY ([MigrationId])); Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] SELECT 1 Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (9ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] SELECT OBJECT_ID(N'[__EFMigrationsHistory]'); Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (8ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] SELECT [MigrationId], [ProductVersion] FROM [__EFMigrationsHistory] ORDER BY [MigrationId]; Microsoft.EntityFrameworkCore.Migrations[20402] Applying migration '20260829190300_InitialCreate'. Applying migration '20260829190300_InitialCreate'. Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (6ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] CREATE TABLE [Students] ([Id] int NOT NULL IDENTITY, [FirstName] nvarchar(max) NOT NULL, [LastName] nvarchar(max) NOT NULL, [Email] nvarchar(max) NOT NULL, [Course] nvarchar(max) NOT NULL, [Age] int NOT NULL, CONSTRAINT [PK_Students] PRIMARY KEY ([Id])); Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] INSERT INTO

```
[__EFMigrationsHistory] ([MigrationId], [ProductVersion]) VALUES  
(N'20260829190300_InitialCreate', N'8.0.30'); Done. PM>
```

2. ChatGPT каза:

🎉 **Перфектно! Базата е създадена успешно.**

Точно това искахме да получим.

Какво се случи?

Най-важните редове са:

```
CREATE DATABASE [StudentManagementDb];
```

Това означава, че SQL Server е създаде базата:

StudentManagementDb

След това Entity Framework е създаде таблицата:

```
CREATE TABLE [Students]
```

И таблицата съдържа:

Id

FirstName

LastName

Email

Course

Age

Тоест в момента имаме:

SQL Server

└─ StudentManagementDb

|

└─ Students

|

└─ Id

|

└─ FirstName

|

└─ LastName

|

└─ Email

|

└─ Course

|

└─ Age

|

└─ __EFMigrationsHistory

__EFMigrationsHistory е вътрешна таблица на Entity Framework Core, която следи кои Migration-и вече са приложени.

А това:

Done.

означава, че всичко е приключило успешно. 

Следващата стъпка

Сега **няма да пишем нов код**.

Първо ще отворим **SQL Server Management Studio 22** и ще проверим с очите си дали:

StudentManagementDb

└─ Tables

└─ dbo.Students

наистина съществува.

Това е важно, защото така ще видиш връзката:

C# модел → EF Core → Migration → SQL Server → таблица

Направи само това

В **SQL Server Management Studio 22**:

1. Намери твоя сървър:
DESKTOP-O6FIRML\SQLEXPRESS02
2. Отвори **Databases**.
3. Потърси:
StudentManagementDb
4. Разгъни я.
5. Разгъни **Tables**.

Каж ми какво виждаш вътре в Tables.

Очаквам да има:

dbo.Students

dbo.__EFMigrationsHistory

И после ще направим **първата истинска функционалност на системата — добавяне на студент**.



1. ок
2. **ChatGPT каза:**

Супер. 

Тогава направи само проверката в **SQL Server Management Studio 22**:

Databases → StudentManagementDb → Tables

Трябва да виждаш:

- dbo.Students
- dbo.__EFMigrationsHistory

Ако ги виждаш, напиши **„виждам ги“**.

После започваме с първата функционалност: **страница за добавяне и управление на студенти.**



1. виждам ги

2. **ChatGPT каза:**

Супер! 🎉 Значи базата работи и връзката е успешна.

Сега започваме **първата реална функционалност** на нашия Student Management System:

Стъпка 5 — Students

Искаме потребителят да може да:

- вижда всички студенти;
- добавя студент;
- редактира студент;
- изтрива студент;
- разглежда подробности за студент.

За това ще използваме **MVC Controller + Views**.

Стъпка 5.1 — създаваме StudentsController

В **Solution Explorer**:

6. Намери папката:

Controllers

2. Натисни **десен бутон** върху нея.
3. Избери:

Add → Controller...

4. Ще се появи прозорец **Add New Scaffolded Item**.
5. Избери:

MVC Controller - Empty

6. Натисни **Add**.
7. За име въведи:

StudentsController

8. Натисни **Add**.

Трябва да получиш:

Controllers

└─ HomeController.cs

└─ StudentsController.cs

Какво представлява Controller?

В MVC имаме:

Model



Controller



View

При нас:

Student.cs



StudentsController.cs



HTML страниците

StudentsController ще отговаря за действията със студентите.

Например по-късно ще имаме:

Index → списък със студенти

Create → добавяне

Edit → редактиране

Details → подробности

Delete → изтриване



Засега спираме

Не пишем код още.

Създай StudentsController и ми изпрати **какво има вътре в него**.

Така ще работим върху твоя реален код и ще добавяме нещата едно по едно.

1. `using Microsoft.AspNetCore.Mvc; namespace StudentManagement.Controllers; public class StudentsController : Controller { public IActionResult Index() { return View(); } }`

2. ChatGPT каза:

Да, перфектно. 👍 Това е стандартният празен MVC controller. И понеже искаме да го разбереш, няма да добавяме всичко наведнъж.

Стъпка 5.2 — свързваме Controller-а с базата

В момента StudentsController знае само как да показва Index:

```
public IActionResult Index()
{
    return View();
}
```

Но той **още не знае за ApplicationDbContext**.

Ще го свържем с него.

1. Добавяме using

Най-отгоре добави:

```
using StudentManagement.Data;
```

Получаваш:

```
using Microsoft.AspNetCore.Mvc;
```

```
using StudentManagement.Data;
```

```
namespace StudentManagement.Controllers;
```

```
public class StudentsController : Controller
```

```
{
```

```
    public IActionResult Index()
```

```
    {
```

```
        return View();
```

```
    }
```

```
}
```

2. Добавяме ApplicationDbContext

Сега вътре в класа, **преди** Index(), добави:

```
private readonly ApplicationDbContext _context;
```

След това добавяме конструктор:

```
public StudentsController(ApplicationDbContext context)
```

```
{
```

```
    _context = context;
```

```
}
```

Целият файл вече трябва да изглежда така:

```
using Microsoft.AspNetCore.Mvc;
```

```
using StudentManagement.Data;
```

```
namespace StudentManagement.Controllers;
```

```
public class StudentsController : Controller
```

```
{
```

```
    private readonly ApplicationDbContext _context;
```

```

public StudentsController(ApplicationDbContext context)
{
    _context = context;
}

public IActionResult Index()
{
    return View();
}
}

```

Какво направихме?

Това:

```
private readonly ApplicationDbContext _context;
```

създава променлива `_context`, чрез която Controller-ът ще разговаря с базата.

А това:

```

public StudentsController(ApplicationDbContext context)
{
    _context = context;
}

```

казва:

„Когато ASP.NET създаде `StudentsController`, дай ми `ApplicationDbContext`.“

ASP.NET Core знае как да го направи, защото по-рано в `Program.cs` написахме:

```
builder.Services.AddDbContext<ApplicationDbContext>(...)
```

Това се нарича **Dependency Injection**.



Спираме тук

Не добавяй още `Create`, `Edit`, `Delete` и т.н.

Постави кода, запази с **Ctrl + S** и направи:

Ctrl + Shift + B

После ми изпрати резултата от Build.

1. Build started at 10:24 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 10:24 PM and took 04.105 seconds =====

2. ChatGPT каза:

Перфектно! 

Build: 1 succeeded, 0 failed

Това означава, че StudentsController вече е успешно свързан с ApplicationDbContext и проектът се компилира без грешки.

Стъпка 5.3 — вземаме студентите от базата

Сега ще направим **само едно нещо**: Index() да прочете студентите от базата.

В момента имаш:

```
public IActionResult Index()
{
    return View();
}
```

Замени **само този метод** с:

```
public async Task<IActionResult> Index()
{
    var students = await _context.Students.ToListAsync();

    return View(students);
}
```

Но ще ни трябва още един using.

Най-отгоре на файла добави:

```
using Microsoft.EntityFrameworkCore;
```

Така целият StudentsController.cs ще бъде:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

namespace StudentManagement.Controllers;

public class StudentsController : Controller
{
    private readonly ApplicationDbContext _context;

    public StudentsController(ApplicationDbContext context)
    {

```

```

        _context = context;
    }

    public async Task<ActionResult> Index()
    {
        var students = await _context.Students.ToListAsync();

        return View(students);
    }
}

```

Какво прави новият код?

```
public async Task<ActionResult> Index()
```

async означава, че методът може да изчаква операция, без да блокира приложението.

```
var students = await _context.Students.ToListAsync();
```

Това е най-важният ред.

Казваме:

„Отиди в таблицата Students, вземи всички записи и ги превърни в списък.“

Получаваме:

Database



Students



ToListAsync()



List<Student>

А:

```
return View(students);
```

подава получения списък към Index.cshtml.

Но има един проблем

Все още **нямаме View**, което да може да покаже този списък.

Това е следващата ни стъпка.

 Засега:

1. Постави кода.
2. Ctrl + S

3. Ctrl + Shift + B
4. Изпрати ми резултата от Build.

Не създавай View още. Ще го направим заедно на следващата стъпка.

1. Build started at 10:27 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 10:27 PM and took 01.164 seconds =====

2. ChatGPT каза:

Перфектно! 🎉 Отново:

Build: 1 succeeded, 0 failed

Значи кодът е правилен и вече можем да продължим.

Стъпка 5.4 — създаваме View за студентите

В StudentsController имаме:

```
return View(students);
```

Това означава, че ASP.NET ще търси View на следния адрес:

Views/Students/Index.cshtml

В момента тази папка вероятно **не съществува**.

1. Създаваме папка Students

В **Solution Explorer**:

1. Намери папката:

Views

2. Десен бутон върху Views.
3. Избери **Add → New Folder**.
4. Кръсти я:

Students

Трябва да получиш:

Views

└─ Home

└─ Shared

└─ Students

2. Създаваме Index.cshtml

Сера:

5. Десен бутон върху новата папка Students.
6. **Add → New Item...**

7. Потърси **Razor View**.

8. Име:

Index.cshtml

5. Натисни **Add**.

Ако Visual Studio ти предложи шаблон, можеш да избереш **Empty**.

 **Спираме тук**

Не пишем HTML още.

Искам първо да сме сигурни, че имаш:

Views

└─ Students

└─ Index.cshtml

Когато го създадеш, кажи ми „**готово**“.

След това ще направим първата ни страница, която ще показва студентите от SQL Server. 🧑

1. имам Razor Component

2. **ChatGPT каза:**

Да — **спри там**, не избирай Razor Component. 👍

За нашия проект използваме **MVC с Razor Views**, защото StudentsController връща:

```
return View(students);
```

Razor Component е различно нещо.

Стъпка 5.4 — правилният начин

Понеже Visual Studio при теб показва Razor Component, ще направим файла ръчно. Това е напълно нормално.

1. В папката Views/Students

Направи:

десен бутон върху Students → Add → New Item...

2. Потърси

В полето за търсене напиши:

Razor View

Ако **няма** Razor View, не избирай Razor Component.

3. Ако пак не го намира

Тогава ще създадем файла ръчно:

десен бутон върху Students → Add → New Item...

Потърси:

Text File

и го кръсти точно:

Index.cshtml

Трябва да получим:

Views

└─ Students

└─ Index.cshtml

Защо .cshtml?

.cshtml е Razor View файлът, който ASP.NET Core MVC използва за генериране на HTML страница.

Нашият:

StudentsController

има:

Index()

и когато посетим:

/Students

MVC ще търси:

Views/Students/Index.cshtml

 **Засега не добавяй код в Index.cshtml.**

Първо само създай файла Index.cshtml.

Ако **не намираш нито** Razor View, **нито** Text File, кажи ми какви опции ти показва **Add → New Item...**, и ще те насоча точно според твоя Visual Studio.

3. Вие казахте:

има Text Editor >Razor(ASP.NET Core)>general; scroll bars и tabs

4. ChatGPT каза:

Да — това, което виждаш:

Text Editor → Razor (ASP.NET Core) → General / Scroll Bars / Tabs

е **настройка на редактора**, а не шаблон за създаване на файл. Няма проблем. 👍

Нека го направим по най-сигурния начин.

Стъпка 5.4 — създаваме Index.cshtml ръчно

1. Намери папката Students

В **Solution Explorer** трябва да имаш:

Views

└─ Home

└─ Shared

└─ Students

2. Десен бутон върху Students

Избери:

Add → New Item...

3. В прозореца Add New Item

Горе вдясно има поле за търсене.

Напиши:

Text File

Ако намериш **Text File**, избери го.

Името на файла трябва да бъде:

Index.cshtml

След това **Add**.

4. Провери

В Solution Explorer трябва да получиш:

Views

└─ Students

└─ Index.cshtml

⚠ Ако Visual Studio създаде:

Index.cshtml.txt

това **не е правилно**. Кажете ми, ако стане така, и ще го поправим.

🛑 **Засега не пиши код**

Само създай:

Index.cshtml

Когато го виждаш в Views → Students, напиши „**имам Index.cshtml**“.

После ще сложим **първия Razor код** и ще ти обясня как работи.

5. Вие казахте:

още няколко неща

6. ChatGPT каза:

Разбира се. 👍 Кажете ми всички уточнения наведнъж.

Ще ги спазвам оттук нататък, за да не се налага да променяме начина, по който работим.

7. Вие казахте:

asp.net core web app(razor pages); razor class library; use the new razor formatting engine in the ide (require restart); formatting(Text Editor → Razor (ASP.NET Core) →Advanced)

8. ChatGPT каза:

Да, разбрах. 👍 Това са **настройки/шаблони на Visual Studio**, които виждаш, и е важно да ги разграничим от самото създаване на MVC View.

Нашият проект е **ASP.NET Core MVC**, а не Razor Pages. Затова:

- ❌ **ASP.NET Core Web App (Razor Pages)** — не го избираме.
- ❌ **Razor Class Library** — не ни трябва.
- ❌ **Razor Component** — не го избираме.
- ⚙️ **Use the new Razor formatting engine in the IDE** — това е настройка на Visual Studio и няма отношение към създаването на Index.cshtml.
- ⚙️ **Text Editor → Razor (ASP.NET Core) → Advanced** — това са настройки за форматиране/редактиране на Razor файловете, не типът на View.

Защо е важно?

Нашата структура е MVC:

Controller



View



HTML

Имаме:

Controllers

└─ StudentsController.cs

и той използва:

```
return View(students);
```

Затова View-то трябва да бъде:

Views

└─ Students

└─ Index.cshtml

а **не**:

Pages

└─ Students

което би било характерно за Razor Pages.

Нека не правим нищо на сяпо

Понеже при твоя Visual Studio не виждаме стандартния шаблон **Razor View**, най-сигурно е първо да видим **точно какво ти предлага менюто**.

Направи:

десен бутон върху Views → Students → Add → New Item...

и ми изпрати **снимка на прозореца** Add New Item.

Така ще ти кажа **точно кое да натиснеш при твоята версия на Visual Studio**, вместо да те карам да избираш нещо на случаен принцип. 👍

1. стандартния шаблон Razor View как мога да си го добавя

2. **ChatGPT каза:**

Да. 👍 И всъщност при **ASP.NET Core MVC** е най-добре да имаш шаблона **Razor View**, за да можем удобно да създаваме Index.cshtml, Create.cshtml, Edit.cshtml и т.н.

При теб вероятно липсва компонентът за **ASP.NET/Web development scaffolding** във Visual Studio.

Стъпка 1 — отваряме Visual Studio Installer

Затвори **Visual Studio** или го остави отворен.

След това:

1. Отвори **Visual Studio Installer**.
2. Намери твоята инсталация на Visual Studio.
3. Натисни **Modify**.

Стъпка 2 — проверяваме Workloads

В таба **Workloads** потърси:

ASP.NET and web development

Трябва да е отменено:

☒ ASP.NET and web development

Това е основният workload, който ни трябва за ASP.NET Core MVC.

Ако не е отменено:

4. Отметни го.
5. Натисни **Modify / Install**.
6. Изчакай инсталацията да приключи.

Стъпка 3 — проверяваме Individual components

Ако ASP.NET and web development вече е инсталиран, **не инсталирай нищо на сяло**.

Отиди на:

Individual components

и потърси:

scaffolding

или:

ASP.NET

Ако видиш компоненти, свързани с **ASP.NET Core scaffolding**, ще ги проверим.

Важно

Не ти препоръчвам да променяш настройки като:

Use the new Razor formatting engine in the IDE

Това **няма да добави Razor View шаблона**. То е само за начина, по който Visual Studio форматира Razor кода.

Но има и по-лесен вариант

Дори ако шаблонът **Razor View** не се появи, можем спокойно да създадем:

Views/Students/Index.cshtml

като обикновен файл. **Не е задължително да имаш шаблона, за да работи MVC View.**

Аз лично бих предпочел първо да проверим дали **ASP.NET and web development** е инсталиран, преди да променяме каквото и да е.

👉 Отвори **Visual Studio Installer** → **Modify** → **Workloads** и ми кажи дали **ASP.NET and web development** е отменено.

1. има отметка
2. ChatGPT каза:

Да, тогава **не пипаме Workload-а**. 👍 Той вече е инсталиран.

Има още една важна подробност: за MVC scaffolding в Visual Studio понякога липсват отделни компоненти, въпреки че **ASP.NET and web development** е отменен.

Стъпка 2 — проверяваме Individual components

В **Visual Studio Installer** → **Modify**:

1. Натисни **Individual components**.
2. В полето за търсене напиши:

scaffolding

3. Кажи ми **какви резултати ти показва**.

🛑 Засега **не слагай нови отметки**.

Ще видим точно какво имаш и ще ти кажа кое ни трябва.

3. **Вие казахте:**

имам катинари навсякъде

4. ChatGPT каза:

Да, това е нормално. 👍 Катинарите означават, че тези компоненти са **вече инсталирани** или са управлявани от избрания workload/инсталацията.

След като **ASP.NET and web development** е отменен и компонентите са заключени, **не е нужно да инсталираме нищо допълнително**.

И тук има една важна корекция от мен: **не е необходимо да имаме шаблона Razor View, за да продължим**. Можем да създадем .cshtml файла ръчно и той ще работи абсолютно нормално като MVC View.

Да се върнем към проекта

Затвори Visual Studio Installer и се върни в проекта.

В:

Views

└─ Students

направи:

Right click → Add → New Item...

Ако не намираш Razor View, избери **Text File** и го кръсти:

Index.cshtml

Това е напълно достатъчно.

Важно

Не избирай:

- ASP.NET Core Web App (Razor Pages)
- Razor Class Library
- Razor Component

Те са за други сценарии.

Нашият проект е:

ASP.NET Core MVC

и ще използваме:

StudentsController



Views/Students/Index.cshtml

👉 Ако вече си създад Index.cshtml, кажи „**има го**“. Ако още не си, направи го като Text File и ми кажи какво се появи в Solution Explorer.

5. Вие казахте:

имам razor page- empty; razor view - empty; razor layout; razor view start; razor view imports

6. ChatGPT каза:

А, **това е точно което ни трябва!** 👍

Имаш:

Razor View - Empty

Това е правилният шаблон за нашия MVC проект.

Стъпка 5.4 — създаваме Index.cshtml

В Views → Students:

1. Десен бутон върху папката **Students**.
2. **Add → New Item...**

3. Избери:
Razor View - Empty
4. Натисни **Add**.
5. За име въведи:

Index.cshtml

6. Натисни **Add**.

Трябва да получиш:

Views

└─ Students

└─ Index.cshtml

Важно

Избери точно:

Razor View - Empty

а не:

- Razor Page - Empty
- Razor Layout
- Razor View Start
- Razor View Imports

За нашия StudentsController правилният избор е **Razor View - Empty**. 

 Засега **не слагай код в Index.cshtml**.

Когато файлът е създаден и го виждаш под:

Views → Students → Index.cshtml

напиши „**готово**“.

После ще направим първата View, която ще прочете и покаже студентите от базата. 

1. готово и ето кода там @* For more information on enabling MVC for empty projects, visit <https://go.microsoft.com/fwlink/?LinkID=397860> *@ @{ }

2. ChatGPT каза:

Да, това е напълно нормално.  Visual Studio е създаде празен **Razor View** и е добавил стандартния си шаблон.

Тъй като се разбрахме **да няма обикновени коментари вътре в кода**, ще изчистим този шаблон.

Стъпка 5.5 — подготвяме Index.cshtml

Изтрий всичко в Index.cshtml и постави само:

```
@model IEnumerable<StudentManagement.Models.Student>
```

<h1>Students</h1>

Какво направихме?

Първият ред:

```
@model IEnumerable<StudentManagement.Models.Student>
```

казва на View-то:

„Тази страница ще получи списък от обекти Student.“

Това съответства на нашия Controller:

```
var students = await _context.Students.ToListAsync();
```

```
return View(students);
```

Тоест:

Database

↓

_context.Students

↓

ToListAsync()

↓

List<Student>

↓

Index.cshtml

A:

<h1>Students</h1>

засега просто ще покаже заглавието **Students**.

Важно

Няма да правим таблицата още. Искам първо да проверим, че **Controller → View** връзката работи правилно.

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Изпрати ми резултата.

След това ще стартираме приложението и ще отворим:

/Students

Торгава ще видим първата ни страница от Student Management System. 🚀

1. само главно меню без линк към страницата, но <https://localhost:7295/Students> съществува
2. **ChatGPT казва:**

Точно така. 👍 Това всъщност е **добър знак**.

<https://localhost:7295/Students> съществува, което означава, че **маршрутът и страницата работят**. Просто в главното меню още няма линк към Students.

Стъпка 5.8 — добавяме Students в главното меню

Главното меню, което виждаш, идва от:

Views

└─ Shared

└─ _Layout.cshtml

Този файл съдържа навигацията:

Home | Privacy

Ще добавим:

Home | Students | Privacy

1. Отвори _Layout.cshtml

В Solution Explorer:

Views

└─ Shared

└─ _Layout.cshtml

Отвори файла.

2. Намери тази част

Трябва да има нещо подобно:

```
<li class="nav-item">
```

```
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
```

```
</li>
```

3. Между Home и Privacy добавяме

```
<li class="nav-item">
```

```
    <a class="nav-link text-dark" asp-area="" asp-controller="Students" asp-  
action="Index">Students</a>
```

```
</li>
```

Получава се:

```
<li class="nav-item">
```

```
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
  <a class="nav-link text-dark" asp-area="" asp-controller="Students" asp-  
  action="Index">Students</a>
```

```
</li>
```

```
<li class="nav-item">
```

```
  <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
```

```
</li>
```

Какво означава това?

Тук:

asp-controller="Students"

казваме:

Линкът води към StudentsController.

А:

asp-action="Index"

казва:

Използвай метода Index().

Така:

Students → StudentsController → Index()

и браузърът отива към:

/Students

След това

1. Запази с **Ctrl + S**.
2. Не променяй нищо друго.
3. Върни се в браузъра и презареди страницата с **Ctrl + F5**.
4. Трябва да се появи:

Home Students Privacy

Натисни **Students**.

Ако те отведе към <https://localhost:7295/Students> и покаже **Students**, значи навигацията вече работи. 

1. ок
2. ChatGPT каза:

Супер! 

Значи вече имаме работеща основа:

-  Student модел

- ☒ ApplicationDbContext
- ☒ SQL Server връзка
- ☒ InitialCreate Migration
- ☒ StudentManagementDb
- ☒ Students таблица
- ☒ StudentsController
- ☒ Students/Index.cshtml
- ☒ линк **Students** в главното меню

Следваща стъпка — показваме студентите като таблица

В момента /Students вероятно показва само:

Students

Следващото, което ще направим, е да превърнем страницата в нещо подобно:

Id	First Name	Last Name	Email	Course	Age
—	—	—	—	—	—

След това ще добавим бутон **Create Student**, с който ще можем да запишем първия студент в SQL Server.

Но ще вървим **една малка стъпка наведнъж**.

👉 Кажете ми, когато си на страницата:

<https://localhost:7295/Students>

и виждаш **Students**. Тогава ще направим таблицата.

1. готово

2. ChatGPT каза:

Чудесно. 👍 Сега ще направим **само таблицата**, без добавяне, редактиране или изтриване още.

Стъпка 5.9 — показваме студентите в таблица

Отвори:

Views

└─ Students

└─ Index.cshtml

Изтрий текущото съдържание:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

и постави това:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

```
<table class="table">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>First Name</th>
```

```
      <th>Last Name</th>
```

```
      <th>Email</th>
```

```
      <th>Course</th>
```

```
      <th>Age</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    @foreach (var student in Model)
```

```
    {
```

```
      <tr>
```

```
        <td>@student.FirstName</td>
```

```
        <td>@student.LastName</td>
```

```
        <td>@student.Email</td>
```

```
        <td>@student.Course</td>
```

```
        <td>@student.Age</td>
```

```
      </tr>
```

```
    }
```

```
  </tbody>
```

```
</table>
```

Какво прави кодът?

Тази част:

```
@model IEnumerable<StudentManagement.Models.Student>
```

казва, че View-то получава **списък от студенти**.

Тази част:

```
@foreach (var student in Model)
```

обикаля през всички студенти, които Controller-ът е получил от базата.

За всеки студент създаваме един <tr> — един ред в таблицата.

Например:

```
<td>@student.FirstName</td>
```

показва името на студента.

Важното в момента

Нашата база данни все още е празна.

Затова след зареждането на /Students ще видиш заглавията:

First Name | Last Name | Email | Course | Age

но **няма да има редове със студенти.**

Това е правилно. Следващата ни задача ще бъде да направим **Create Student** и да добавим първия запис.

Сера

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Ако Build е успешен, отвори /Students или презареди страницата.

Кажете ми какво виждаш.

1. Вие казахте:

празна таблица

2. ChatGPT каза:

Точно така!  **Празната таблица е правилният резултат.**

Това означава, че всичко по веригата работи:

SQL Server



StudentManagementDb



Students



ApplicationDbContext



StudentsController



Index.cshtml



Празна таблица

Просто още **нямаме записан студент**.

Стъпка 5.10 — добавяме бутон „Add Student“

Сега ще направим само бутон. След това ще направим формата.

Отвори:

Views

└─ Students

└─ Index.cshtml

Най-горе имаш:

```
<h1>Students</h1>
```

Веднага **под него** добави:

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

Така началото на файла ще изглежда:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

```
<table class="table">
```

Какво прави този ред?

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

Това създава Bootstrap бутон **Add Student**.

asp-action="Create" казва:

Когато натисна бутон, отиди към Create action в текущия controller — StudentsController.

Тоест очакваме:

Add Student

↓

StudentsController

↓

Create()

 **Но Create() все още не съществува.**

Затова, ако натиснеш бутон сега, ще получиш грешка/404. Това е нормално и точно това ще направим в следващата стъпка.

Сега само:

1. Добави реда.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Отвори /Students.
5. Провери дали виждаш син бутон **Add Student**.

Не натискай бутона още. 😊

Кажете ми „**бутонът е там**“, когато го видиш.

1. бутонът е там
2. **ChatGPT каза:**

Супер! 👍 Бутонът е там и вече имаме връзката към бъдещата форма.

Стъпка 5.11 — създаваме Create action

В момента бутонът сочи към:

/Students/Create

но StudentsController все още няма Create() метод.

Сега ще добавим **само GET версията** на Create. Тя ще показва празната форма.

Отвори:

Controllers

└─ StudentsController.cs

Под метода Index() добави:

```
public IActionResult Create()
{
    return View();
}
```

Така StudentsController трябва да изглежда:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

namespace StudentManagement.Controllers;

public class StudentsController : Controller
{
    private readonly ApplicationDbContext _context;
```

```

public StudentsController(ApplicationDbContext context)
{
    _context = context;
}

public async Task<ActionResult> Index()
{
    var students = await _context.Students.ToListAsync();

    return View(students);
}

public IActionResult Create()
{
    return View();
}
}

```

Какво прави Create()?

Когато натиснем:

Add Student

ASP.NET ще намери:

StudentsController



Create()



View()

и ще търси View:

Views

└─ Students

└─ Create.cshtml

Този файл още няма да съществува.

Сера само проверяваме Controller-a

1. Добави метода.
2. **Ctrl + S**

3. Ctrl + Shift + B

4. Изпрати ми резултата от Build.

Не създавай Create.cshtml **още**. След успешния Build ще го направим като Razor View - Empty, както направихме с Index.cshtml.

1. Build started at 11:58 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 11:59 PM and took 00.865 seconds =====

2. ChatGPT каза:

Перфектно.  Create() е добавен и проектът се компилира без грешки.

Стъпка 5.12 — създаваме Create.cshtml

Сега ще създадем View-то за формата.

В **Solution Explorer**:

1. Десен бутон върху:
Views → Students
2. Избери **Add → New Item...**
3. Избери:
Razor View - Empty
4. Име:

Create.cshtml

5. Натисни **Add**.

Трябва да имаш:

Views

└─ Students

└─ Create.cshtml

└─ Index.cshtml



Засега не пишем код

Само създай Create.cshtml.

Когато го видиш в Solution Explorer, кажи „**готово**“.

След това ще направим формата **поле по поле**, за да е ясно как всяко нещо работи.

3. Вие казахте:

готово

4. ChatGPT каза:

Супер!  Сега ще направим **само основата на формата**. После ще добавим запазването в базата.

Стъпка 5.13 — моделът на формата

Отвори:

Views

└─ Students

└─ Create.cshtml

Изтрий всичко вътре и постави:

@model StudentManagement.Models.Student

```
<h1>Add Student</h1>
```

```
<form asp-action="Create" method="post">
```

```
    <div class="mb-3">
```

```
        <label asp-for="FirstName" class="form-label"></label>
```

```
        <input asp-for="FirstName" class="form-control" />
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <label asp-for="LastName" class="form-label"></label>
```

```
        <input asp-for="LastName" class="form-control" />
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <label asp-for="Email" class="form-label"></label>
```

```
        <input asp-for="Email" class="form-control" />
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <label asp-for="Course" class="form-label"></label>
```

```
        <input asp-for="Course" class="form-control" />
```

```
    </div>
```

```
    <div class="mb-3">
```

```
        <label asp-for="Age" class="form-label"></label>
```

```
        <input asp-for="Age" class="form-control" />
```

</div>

<button type="submit" class="btn btn-primary">Save</button>

<a asp-action="Index" class="btn btn-secondary">Cancel

</form>

Какво направихме?

Първият ред:

@model StudentManagement.Models.Student

казва:

Тази форма работи с един обект Student.

След това:

<form asp-action="Create" method="post">

казва:

При натискане на **Save** изпрати данните към Create action чрез POST.

Например:

<input asp-for="FirstName" class="form-control" />

автоматично се свързва с:

Student.FirstName

Същото правим за:

FirstName

LastName

Email

Course

Age

Save изпраща формата, а Cancel ни връща към списъка.

Важно

В момента формата **ще се покаже**, но натискането на Save няма да запише студента, защото все още нямаме POST Create action.

Това ще бъде **следващата малка стъпка**.

Засега:

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Ако Build е успешен, стартирай приложението.

4. Отиди на /Students.
5. Натисни **Add Student**.

Трябва да видиш формата с:

Add Student

First Name

Last Name

Email

Course

Age

Save Cancel

Не натискай Save още.

Кажете ми как изглежда формата.

1. както при теб
2. **ChatGPT каза:**

Чудесно! 👍 Значи **View-то работи правилно** и формата се показва.

Сега идва важната част: ще направим така, че натискането на **Save** действително да запише студента в SQL Server.

Стъпка 5.14 — POST Create

В момента имаме:

Create.cshtml

↓

Save

↓

/Students/Create

Но Controller-ът няма метод, който да приеме изпратените данни.

Отвори:

Controllers

└─ StudentsController.cs

Под сегашния Create() метод добави:

[HttpPost]

[ValidateAntiForgeryToken]

```
public async Task<ActionResult> Create(Student student)
```

```

{
    if (!ModelState.IsValid)
    {
        return View(student);
    }

    _context.Students.Add(student);
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}

```

Ще имаме една малка промяна

За да разпознае Student, Controller-ът трябва да има:

```
using StudentManagement.Models;
```

Затова най-отгоре трябва да имаш:

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

```
using StudentManagement.Models;
```

Целият StudentsController.cs ще изглежда така:

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagement.Data;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagement.Controllers;
```

```
public class StudentsController : Controller
```

```
{
```

```
    private readonly ApplicationDbContext _context;
```

```
    public StudentsController(ApplicationDbContext context)
```

```
{
```

```
    _context = context;
```

```

    }

    public async Task<ActionResult> Index()
    {
        var students = await _context.Students.ToListAsync();

        return View(students);
    }

    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> Create(Student student)
    {
        if (!ModelState.IsValid)
        {
            return View(student);
        }

        _context.Students.Add(student);
        await _context.SaveChangesAsync();

        return RedirectToAction(nameof(Index));
    }
}

```

Какво прави POST методът?

Когато натиснеш **Save**, формата изпраща:

FirstName

LastName

Email

Course

Age

към този метод:

```
Create(Student student)
```

ASP.NET автоматично създава Student обект от въведените данни.

После:

```
_context.Students.Add(student);
```

казва на Entity Framework:

„Искам този студент да бъде добавен към таблицата Students.“

След това:

```
await _context.SaveChangesAsync();
```

е моментът, в който промяната реално се записва в SQL Server.

Накрая:

```
return RedirectToAction(nameof(Index));
```

връща потребителя обратно към списъка със студенти.

А какво прави [ValidateAntiForgeryToken]?

Това е стандартна защита при POST форми в ASP.NET Core MVC. Тя предпазва формата от определен тип атака, наречена **Cross-Site Request Forgery (CSRF)**.

Razor формата ни автоматично ще добави необходимия anti-forgery token.

Сега само това

1. Добави using StudentManagement.Models;
2. Добави POST Create.
3. **Ctrl + S**
4. **Ctrl + Shift + B**
5. Изпрати ми резултата от Build.

Още не натискай Save във формата. Първо проверяваме, че Controller-ът се компилира.

1. Build started at 12:10 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 12:10 AM and took 01.032 seconds =====

2. ChatGPT каза:

Перфектно! 

Build: 1 succeeded, 0 failed

Значи POST Create е добавен правилно.

Сега идва **първият ни истински тест** — ще запишем реален студент в SQL Server.

Стъпка 5.15 — добавяме първия студент

Стартирай приложението и отвори:

<https://localhost:7295/Students>

Натисни:

Add Student

Във формата въведи например:

First Name: Иван

Last Name: Петров

Email: ivan.petrov@example.com

Course: Computer Science

Age: 20

След това натисни:

Save

Какво трябва да се случи?

След натискането на Save:

Create.cshtml



POST Create(Student student)



_context.Students.Add(student)



SaveChangesAsync()



SQL Server



Students таблица



/Students

И трябва да се върнеш автоматично на страницата **Students**.

Този път таблицата вече трябва да съдържа:

First Name	Last Name	Email	Course	Age
Иван	Петров	ivan.petrov@example.com	Computer Science	20



Направи само този тест

Добави **един** студент и ми кажи какво се случи след натискането на **Save**.

Ако се появи грешка, **не променяй нищо** — изпрати ми текста на грешката и ще я оправим заедно.

1. ок

2. ChatGPT каза:

Супер. 👍

Направи теста:

1. **Students**
2. **Add Student**
3. Попълни петте полета.
4. Натисни **Save**.
5. Провери дали студентът се появява в таблицата.

После ми кажи само едно от двете:

- „добави се“ ✅
- или „има грешка“ ❌ — и ми изпрати грешката.

Засега не правим други промени.

3. **Вие казахте:**

добави се

4. ChatGPT каза:

Страхотно! 🎉 Това е важен момент — **първият студент вече се записва реално в SQL Server**.

Вече сме доказали, че целият поток работи:

Форма



StudentsController



Entity Framework Core



SQL Server



Students таблица

Стъпка 5.16 — проверяваме базата

Преди да добавяме нова функционалност, нека видим дали студентът наистина е в SQL Server.

Отвори **SQL Server Management Studio (SSMS)**.

В Object Explorer намери:

DESKTOP-O6FIRML\SQLEXPRESS02

└─ Databases

└─ StudentManagementDb

└─ Tables

└─ dbo.Students

Ако dbo.Students е там:

1. Десен бутон върху dbo.Students.
2. Избери **Select Top 1000 Rows**.

Трябва да видиш записания студент.

Защо правим тази проверка?

Защото вече знаем, че страницата го показва, но сега ще потвърдим директно в SQL Server:

StudentManagementDb

↓

dbo.Students

↓

Иван Петров

Това е полезно и за теб като начинаещ — ще виждаш връзката между **C# кода и реалната база данни**.

 Направи само тази проверка.

Кажете ми дали виждаш студента в dbo.Students.

1. записа го има
2. **ChatGPT каза:**

Перфектно! 🎉 Значи всичко работи както трябва.

Вече имаме **реално работещ CRUD проект**, но засега сме направили само **Create** и **Read**.

Какво имаме досега

-  Student модел
-  Entity Framework Core 8.0.30
-  SQL Server Express
-  ApplicationDbContext
-  Connection String
-  Migration InitialCreate
-  База StudentManagementDb
-  Таблица Students
-  StudentsController

-  Списък със студенти
-  Добавяне на студент
-  Записване в SQL Server
-  Проверка директно в SSMS

Следваща стъпка — Edit

Сега ще направим възможност да **редактираме съществуващ студент**.

Например:

Иван Петров

ivan.petrov@example.com

Computer Science

20

да можем да променим на:

Иван Петров

ivan.petrov@example.com

Software Engineering

21

Ще направим това отново **стъпка по стъпка**, без да прескачаме.

Стъпка 5.17

Първо ще добавим в таблицата един бутон:

Edit

който ще води към:

/Students/Edit/1

Но **още няма да правим Edit Controller-а и формата**.

Ако си готов, следващата ни малка стъпка е да добавим само **Edit линка към всеки студент**.

1. да

2. **ChatGPT каза:**

Чудесно.  Правим **само Edit бутона** към всеки студент.

Стъпка 5.17 — добавяме Edit

Отвори:

Views

└─ Students

└─ Index.cshtml

В момента в таблицата имаш нещо подобно:

@foreach (var student in Model)

```

{
  <tr>
    <td>@student.FirstName</td>
    <td>@student.LastName</td>
    <td>@student.Email</td>
    <td>@student.Course</td>
    <td>@student.Age</td>
  </tr>
}

```

1. Добавяме нова колона

В <thead> след Age добави:

```
<th>Actions</th>
```

Получаваш:

```

<thead>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Email</th>
    <th>Course</th>
    <th>Age</th>
    <th>Actions</th>
  </tr>
</thead>

```

2. Добавяме Edit към всеки ред

В <tr> след:

```
<td>@student.Age</td>
```

добави:

```

<td>
  <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
</td>

```

Така целият foreach ще бъде:

```
@foreach (var student in Model)
```

```

{
  <tr>

```

```

        <td>@student.FirstName</td>
        <td>@student.LastName</td>
        <td>@student.Email</td>
        <td>@student.Course</td>
        <td>@student.Age</td>
        <td>
            <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
        </td>
    </tr>
}

```

Какво означава това?

Тази част:

`asp-action="Edit"`

казва:

Отиди към Edit action.

А:

`asp-route-id="@student.Id"`

подава **Id** на конкретния студент.

Например ако Иван има:

`Id = 1`

бутона ще води към:

`/Students/Edit/1`

Ако друг студент има `Id = 2`:

`/Students/Edit/2`

Това е важно, защото по този начин знаем **кой точно студент искаме да редактираме**.



Сега само това

1. Добави колоната Actions.
2. Добави Edit бутона.
3. Ctrl + S
4. Ctrl + Shift + B
5. Отвори `/Students`.

Не натискай Edit още, защото Edit() в Controller-а все още не съществува.

Изпрати ми Build резултата и ще продължим.

1. Build started at 12:26 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 12:26 AM and took 01.009 seconds =====

2. ChatGPT каза:

Перфектно!  Отново:

Build: 1 succeeded, 0 failed

GET Edit вече е готов.

Стъпка 5.19 — създаваме Edit.cshtml

Сега ще създадем View-то, което ще покаже данните на избрания студент.

В **Solution Explorer**:

1. Десен бутон върху:
Views → Students
2. **Add → New Item...**
3. Избери:
Razor View - Empty
4. Име:

Edit.cshtml

5. Натисни **Add**.

Трябва да имаш:

Views

└─ Students

└─ Create.cshtml

└─ Edit.cshtml

└─ Index.cshtml

Засега не добавяй код

Само създай Edit.cshtml.

Когато го видиш в Views → Students, кажи „**готово**“ и ще направим формата за редактиране, отново стъпка по стъпка.

1. готово

2. ChatGPT каза:

Чудесно.  Сега ще направим формата за редактиране.

Стъпка 5.20 — Edit.cshtml

Отвори:

Views → Students → Edit.cshtml

Изтрий всичко вътре и постави:

@model StudentManagement.Models.Student

<h1>Edit Student</h1>

<form asp-action="Edit" method="post">

 <input type="hidden" asp-for="Id" />

 <div class="mb-3">

 <label asp-for="FirstName" class="form-label"></label>

 <input asp-for="FirstName" class="form-control" />

 </div>

 <div class="mb-3">

 <label asp-for="LastName" class="form-label"></label>

 <input asp-for="LastName" class="form-control" />

 </div>

 <div class="mb-3">

 <label asp-for="Email" class="form-label"></label>

 <input asp-for="Email" class="form-control" />

 </div>

 <div class="mb-3">

 <label asp-for="Course" class="form-label"></label>

 <input asp-for="Course" class="form-control" />

 </div>

 <div class="mb-3">

 <label asp-for="Age" class="form-label"></label>

 <input asp-for="Age" class="form-control" />

 </div>

```
<button type="submit" class="btn btn-primary">Save</button>
```

```
<a asp-action="Index" class="btn btn-secondary">Cancel</a>
```

```
</form>
```

Какво е новото тук?

Най-важният ред, който не сме имали в Create, е:

```
<input type="hidden" asp-for="Id" />
```

Той запазва Id на студента във формата.

Например:

Id = 1

Когато редактираме Иван, формата ще знае:

Редактираме студент №1.

Останалите полета се попълват автоматично от:

```
return View(student);
```

който направихме в Edit(int? id).

Така потокът вече е:

Students

↓

Edit

↓

Edit/1

↓

FindAsync(1)

↓

Edit.cshtml

↓

Показва данните на Иван



Засега спираме тук

Не добавяме POST Edit още.

Направи:

1. **Ctrl + S**
2. **Ctrl + Shift + B**
3. Изпрати ми Build резултата.

След това ще проверим дали при натискане на **Edit** се отваря формата с вече попълнените данни.

1. Вие казахте:

може ли да скрием `https://localhost:7295/Students/Edit/1` -> да скрием от лентата и да пише например името и фамилията за по-голяма сигурност

2. ChatGPT каза:

Да, можем — и е добра идея да го направим, но има едно важно уточнение:

скриването на Id в адресната лента не е реална защита само по себе

си. Например `/Students/Edit/1` не разкрива чувствителна информация, но можем да направим URL-а по-удобен и да не показваме числовия Id.

За начинаещ бих ти предложил първо да направим URL от типа:

`/Students/Edit/Ivan-Petrov`

или:

`/Students/Edit/Ivan/Petrov`

Така вместо:

`https://localhost:7295/Students/Edit/1`

ще виждаш например:

`https://localhost:7295/Students/Edit/Ivan/Petrov`

Но има още по-добър вариант

Ако целта е **сигурност**, можем да оставим Id вътрешно в приложението, но да не го показваме в URL-а. Например:

`/Students/Edit/Ivan-Petrov`

Controller-ът ще намери студента по името, а Id ще си остане само вътрешно.

Само че има един проблем: двама студенти могат да се казват „Иван Петров“. Затова име + фамилия не е уникален идентификатор.

По-правилният професионален вариант е да използваме **GUID/случаен идентификатор** в URL-а или да използваме route, който не показва вътрешния database Id.

Но понеже правим проекта **стъпка по стъпка**, предлагам да не променяме това точно сега, докато не завършим базовия CRUD.

След като направим:

- ☒ Create
- ☒ Read
- ☒ Edit
- ☐ Delete
- ☐ Details

ще направим отделна малка стъпка за **по-сигурни и чисти URL адреси** и ще обсъдим защо избраният вариант е безопасен.

Така няма да счупим работещия Edit в момента.

3. Вие казахте:

за сега няма да го променяме

4. ChatGPT каза:

Разбрах. 👍 Засега **не променяме URL адресите**.

Продължаваме по плана и първо завършваме основния CRUD.

В момента сме на **Edit**. Следва:

1. Проверяваме POST Edit с реална промяна.
2. След това правим **Delete**.
3. Накрая ще имаме основния CRUD напълно работещ.

Сера

Имаш вече добавения:

[HttpPost]

[ValidateAntiForgeryToken]

```
public async Task<ActionResult> Edit(int id, Student student)
```

Направи **Ctrl + Shift + B** и ми изпрати резултата, ако още не си го направил след добавянето му.

После ще променим например курса или възрастта на Иван и ще проверим дали промяната влиза и в SQL Server.

5. Вие казахте:

```
Build started at 12:45 AM... 1>----- Build started: Project: StudentManagement, Configuration:
Debug Any CPU ----- 1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build
completed at 12:45 AM and took 07.334 seconds =====
```

6. ChatGPT каза:

Перфектно! ✅ POST Edit се компилира без грешки.

Стъпка 5.23 — тестваме редактирането

Сега ще проверим целия процес с реална промяна.

1. Стартирай приложението.
2. Отвори **Students**.
3. Натисни **Edit** на Иван Петров.
4. Промени само **Course**, например от:

Computer Science

на:

Software Engineering

5. Натисни **Save**.

Какво трябва да се случи?

След Save трябва автоматично да се върнеш на:

/Students

и таблицата трябва да показва:

First Name	Last Name	Course
------------	-----------	--------

Иван	Петров	Software Engineering
------	--------	----------------------

След това можем да направим още една важна проверка в **SQL Server Management Studio** — дали `dbo.Students` също съдържа променения курс.

 Направи само тази промяна и ми кажи:

„промени се“ или „има грешка“.

7. Вие казахте:

промени се и в `dbo.Students` също

8. ChatGPT каза:

Отлично! 🎉 Това е точно резултатът, който искахме.

Вече имаме напълно работещи **Create + Read + Update** операции:

Create → добавя студент

Read → показва студентите

Update → променя студент

И най-важното — проверихме, че промяната стига **реално до SQL Server**.

Стъпка 5.24 — Delete

Следва последната основна CRUD операция:

Delete → изтрива студент

Ще го направим внимателно, защото не искам случайно да изтрием студента, който използваме за тестове.

Първо ще добавим само **Delete бутон** към таблицата.

Отвори:

Views → Students → Index.cshtml

В колоната Actions, където в момента имаш:

```
<td>
```

```
    <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
```

```
</td>
```

замени я с:

```
<td>
```

```
    <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
```

```
<a asp-action="Delete" asp-route-id="@student.Id" class="btn btn-danger">Delete</a>
</td>
```

Така всеки студент ще има два бутона:

Edit Delete

Какво прави Delete линкът?

Например за студент с Id = 1:

/Students/Delete/1

Точно както направихме при Edit.

 **Все още няма да работи**, защото още нямаме Delete() в Controller-а. Това е нарочно — ще го изградим на отделни стъпки.

Сега

1. Промени само Index.cshtml.
2. Запази.
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

После ще направим Delete метода.

1. Build started at 12:52 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== ===== Build completed at 12:52 AM and took 01.010 seconds =====

2. ChatGPT каза:

Перфектно!  Build-ът е успешен.

Сега имаме бутон **Delete**, но още няма да го натискаме.

Стъпка 5.25 — GET Delete

Първо ще направим страница за потвърждение:

„Сигурни ли сте, че искате да изтриете този студент?“

Това е по-добре от директно изтриване при натискане на линка.

1. Отвори

Controllers

└─ StudentsController.cs

Под Edit методите добави:

```
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
```

```
        return NotFound();
    }

    var student = await _context.Students.FirstOrDefault(m => m.Id == id);

    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}
```

Какво прави?

При натискане на:

/Students/Delete/1

методът получава:

id = 1

След това:

```
_context.Students.FirstOrDefault(m => m.Id == id)
```

търси конкретния студент в базата.

Ако го намери:

```
return View(student);
```

ще отвори:

Views/Students/Delete.cshtml

Там ще направим страницата за потвърждение.

Ако студентът не съществува, получаваме:

404 Not Found



Само това засега

1. Добави GET Delete.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

Не създавай Delete.cshtml още. Ще го направим след успешния Build.

1. Build started at 12:56 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 12:56 AM and took 00.910 seconds =====

2. ChatGPT каза:

Перфектно!  GET Delete е готов и Build-ът е успешен.

Стъпка 5.26 — създаваме Delete.cshtml

Сега ще направим страницата за потвърждение.

В **Solution Explorer**:

1. Десен бутон върху:
Views → Students
2. **Add → New Item...**
3. Избери:
Razor View - Empty
4. Име:

Delete.cshtml

5. Натисни **Add**.

Трябва да имаш:

Views

└─ Students
 ├─ Create.cshtml
 ├─ Delete.cshtml
 ├─ Edit.cshtml
 └─ Index.cshtml

Засега не пишем код

Само създай Delete.cshtml.

Когато го виждаш в Views → Students, напиши „**готово**“. После ще направим страницата за потвърждение.

3. Вие казахте:

готово

4. ChatGPT каза:

Чудесно.  Сега ще направим **само страницата за потвърждение**. Все още няма да изтриваме нищо.

Стъпка 5.27 — Delete.cshtml

Отвори:

Views → Students → Delete.cshtml

Изтрий всичко вътре и постави:

@model StudentManagement.Models.Student

<h1>Delete Student</h1>

<h3>Are you sure you want to delete this student?</h3>

<div>

<dl class="row">

<dt class="col-sm-2">First Name</dt>

<dd class="col-sm-10">@Model.FirstName</dd>

<dt class="col-sm-2">Last Name</dt>

<dd class="col-sm-10">@Model.LastName</dd>

<dt class="col-sm-2">Email</dt>

<dd class="col-sm-10">@Model.Email</dd>

<dt class="col-sm-2">Course</dt>

<dd class="col-sm-10">@Model.Course</dd>

<dt class="col-sm-2">Age</dt>

<dd class="col-sm-10">@Model.Age</dd>

</dl>

<form asp-action="Delete" method="post">

<input type="hidden" asp-for="Id" />

<button type="submit" class="btn btn-danger">Delete</button>

<a asp-action="Index" class="btn btn-secondary">Cancel

</form>

</div>

Какво прави тази страница?

Тук:

@model StudentManagement.Models.Student

казваме, че страницата получава **един конкретен студент**.

Например:

Иван

Петров

ivan.petrov@example.com

Software Engineering

20

След това показваме информацията, за да може потребителят да провери **кой студент ще бъде изтрил**.

Тази част:

```
<input type="hidden" asp-for="Id" />
```

запазва Id на студента, който ще бъде изпратен при потвърждението.

А:

```
<form asp-action="Delete" method="post">
```

означава, че бутонът **Delete** ще изпрати POST заявка към Delete action.

Важно

Все още **нямаме POST Delete в Controller-а**.

Следователно:

Students



Delete



Delete.cshtml



Delete button

ще покаже страницата, но още няма да изтрие записа.



Cera:

1. Постави кода.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

После ще направим последната част — реалното изтриване от SQL Server.

1. Build started at 1:00 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement ->

```
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagem
ent.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 1:00 AM and took 00.775 seconds =====
```

2. ChatGPT каза:

Перфектно! 

Build-ът е успешен и Delete.cshtml е наред.

Стъпка 5.28 — тестваме страницата за потвърждение

Преди да добавим реалното изтриване, ще проверим само дали страницата се отваря правилно.

1. Стартирай приложението.
2. Отиди на **Students**.
3. Намери тестовия студент.
4. Натисни **Delete**.

Трябва да се отвори страница приблизително така:

Delete Student

Are you sure you want to delete this student?

First Name Иван

Last Name Петров

Email ...

Course Software Engineering

Age 20

Delete Cancel



Много важно

Не натискай червения Delete бутон още.

Все още не сме добавили POST метода, който реално изтрива записа.

Провери само дали:

- страницата се отваря;
- правилният студент се показва;
- виждаш бутоните **Delete** и **Cancel**.

Ако всичко е наред, кажи ми „**работи**“ и ще направим последната стъпка от CRUD — реалното изтриване от базата.

1. работи

2. ChatGPT каза:

Супер! 🎉 Значи GET Delete и страницата за потвърждение работят.

Сега идва **последната част на CRUD — реалното изтриване.**

Стъпка 5.29 — POST Delete

Отвори:

Controllers

└─ StudentsController.cs

Под GET Delete метода добави:

[HttpPost]

[ValidateAntiForgeryToken]

```
public async Task<IActionResult> DeleteConfirmed(int id)
```

```
{
```

```
    var student = await _context.Students.FindAsync(id);
```

```
    if (student == null)
```

```
    {
```

```
        return NotFound();
```

```
    }
```

```
    _context.Students.Remove(student);
```

```
    await _context.SaveChangesAsync();
```

```
    return RedirectToAction(nameof(Index));
```

```
}
```

Но имаме една малка промяна в Delete.cshtml

В момента формата има:

```
<form asp-action="Delete" method="post">
```

Трябва да я променим на:

```
<form asp-action="DeleteConfirmed" method="post">
```

Така натискането на **Delete** ще извика точно:

```
DeleteConfirmed(int id)
```

Какво ще се случи?

Delete.cshtml

↓

DeleteConfirmed(id)



FindAsync(id)



Remove(student)



SaveChangesAsync()



SQL Server



студентът е изтрит



/Students

Remove() маркира студента за изтриване, а SaveChangesAsync() реално изпраща промяната към SQL Server.

Защо не използваме директно Delete()?

За да разделим ясно двете действия:

GET Delete



Показва потвърждението

POST DeleteConfirmed



Реално изтрива

Това е по-ясна структура за нашия проект.

Сера

1. Добави DeleteConfirmed в StudentsController.
2. В Delete.cshtml промени само:
3. asp-action="Delete"

на:

asp-action="DeleteConfirmed"

4. **Ctrl + S**
5. **Ctrl + Shift + B**
6. Изпрати ми Build резултата.

 **Не натискай Delete в браузъра още.** Първо ще проверим Build-a.

1. така изглежда Delete.cshtml @model StudentManagement.Models.Student <h1>Delete Student</h1> <h3>Are you sure you want to delete this student?</h3> <div> <dl class="row"> <dt class="col-sm-2">First Name</dt> <dd class="col-sm-10">@Model.FirstName</dd> <dt class="col-sm-2">Last Name</dt> <dd class="col-sm-10">@Model.LastName</dd> <dt class="col-sm-2">Email</dt> <dd class="col-sm-10">@Model.Email</dd> <dt class="col-sm-2">Course</dt> <dd class="col-sm-10">@Model.Course</dd> <dt class="col-sm-2">Age</dt> <dd class="col-sm-10">@Model.Age</dd> </dl> <form asp-action="DeleteConfirmed" method="post"> <input type="hidden" asp-for="Id" /> <button type="submit" class="btn btn-danger">Delete</button> <a asp-action="Index" class="btn btn-secondary">Cancel </form> </div>

2. ChatGPT каза:

Да, точно така трябва да изглежда. 

Особено важно е, че вече си променил:

```
<form asp-action="DeleteConfirmed" method="post">
```

Това е правилно и съвпада с метода, който добавихме в Controller-a.

Сера Delete.cshtml е готов.

Следващото

Ако вече си добавил и този метод в StudentsController:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    _context.Students.Remove(student);
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}
```

направи:

Ctrl + S → Ctrl + Shift + B

и ми изпрати резултата от Build.

Ако Build е успешен, ще направим **контролирано изтриване на тестовия студент** и ще проверим и в dbo.Students, както направихме при Edit.

3. Вие казахте:

```
StudentsController изглежда така using Microsoft.AspNetCore.Mvc; using
Microsoft.EntityFrameworkCore; using StudentManagement.Data; using
StudentManagement.Models; namespace StudentManagement.Controllers; public class
StudentsController : Controller { private readonly ApplicationDbContext _context; public
StudentsController(ApplicationDbContext context) { _context = context; } public async
Task<IActionResult> Index() { var students = await _context.Students.ToListAsync(); return
View(students); } public IActionResult Create() { return View(); } [HttpPost]
[ValidateAntiForgeryToken] public async Task<IActionResult> Create(Student student) { if
(!ModelState.IsValid) { return View(student); } _context.Students.Add(student); await
_context.SaveChangesAsync(); return RedirectToAction(nameof(Index)); } public async
Task<IActionResult> Edit(int? id) { if (id == null) { return NotFound(); } var student = await
_context.Students.FindAsync(id); if (student == null) { return NotFound(); } return View(student); }
[HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> Edit(int id, Student student)
{ if (id != student.Id) { return NotFound(); } if (!ModelState.IsValid) { return View(student); }
_context.Update(student); await _context.SaveChangesAsync(); return
RedirectToAction(nameof(Index)); } public async Task<IActionResult> Delete(int? id) { if (id == null) {
return NotFound(); } var student = await _context.Students.FirstOrDefaultAsync(m => m.Id == id); if
(student == null) { return NotFound(); } return View(student); } [HttpPost] [ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id) { var student = await
_context.Students.FindAsync(id); if (student == null) { return NotFound(); }
_context.Students.Remove(student); await _context.SaveChangesAsync(); return
RedirectToAction(nameof(Index)); } }
```

4. ChatGPT каза:

Да — StudentsController **ти е правилен**.  Няма нужда да променяме нищо в него на този етап.

Имаш вече трите основни операции:

- Create → добавя студент
- Edit → редактира студент
- Delete → изтрива студент

И DeleteConfirmed е свързан правилно с твоя Delete.cshtml.

Стъпка 5.30 — Build

Сега направи:

Ctrl + S

след това:

Ctrl + Shift + B

Ако получиш:

Build: 1 succeeded, 0 failed

ще направим теста.

След успешния Build

Ще стартираме приложението и ще направим следното:

4. Отваряме **Students**.
5. Натискаме **Delete** на тестовия студент.
6. Проверяваме дали се показва страницата за потвърждение.
7. Натискаме червения **Delete**.
8. Проверяваме, че студентът е изчезнал от таблицата.
9. Проверяваме и `dbo.Students` в SQL Server.

Така ще потвърдим, че **Delete работи от браузъра чак до SQL Server**.

Направи Build-а и ми изпрати резултата.

1. Build started at 1:12 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 1:13 AM and took 07.154 seconds =====

2. ChatGPT каза:

Перфектно!  Build-ът е успешен.

Сега правим **реалния тест на Delete**.

Стъпка 5.31 — изтриваме тестовия студент

Стартирай приложението и отвори:

`/Students`

Намери тестовия студент и:

1. Натисни **Delete**.
2. Ще се отвори страницата **Delete Student**.
3. Провери дали показва правилния студент.
4. Натисни червения **Delete**.

След това провери две неща

В сайта:

`/Students`

Студентът трябва да е изчезнал от таблицата.

След това в **SQL Server Management Studio**:

`StudentManagementDb`

→ `Tables`

→ `dbo.Students`

Изпълни **Select Top 1000 Rows**.

Студентът също трябва да е изчезнал от таблицата.

⚠ Ако имаш други студенти, **не ги изтривай** — изтрий само тестовия, който използвахме.

Кажете ми дали студентът е изчезнал **и от сайта, и от dbo.Students**.

1. да

2. ChatGPT каза:

Супер! 🎉 **CRUD-ът вече работи изцяло.**

Имаме:

CREATE → добавяне

READ → показване

UPDATE → редактиране

DELETE → изтриване

И проверихме всичко и директно в SQL Server. Това означава, че основната функционалност на Student Management System вече е налице. ✅

Какво имаме в проекта

StudentManagement

|

|— Controllers

| |— HomeController.cs

| |— StudentsController.cs

|

|— Data

| |— ApplicationDbContext.cs

|

|— Models

| |— Student.cs

|

|— Views

| |— Home

| |— Students

| | |— Create.cshtml

| | |— Delete.cshtml

| | |— Edit.cshtml

| | |— Index.cshtml

| └─ Shared

|

| └─ Migrations

| └─ InitialCreate

|

└─ appsettings.json

И базата:

StudentManagementDb

└─ dbo.Students

Следващата логична стъпка

Преди да добавяме нови функции, бих предложил да направим **Details** страница.

Тя ще позволява:

Students

↓

Details

↓

/Students/Details/1

и ще показва информацията за конкретния студент, без да може да я променя.

След това можем да подобрим интерфейса и да добавим например:

- търсене на студент;
- филтриране;
- сортиране;
- validation съобщения на български;
- по-добър Bootstrap дизайн;
- навигация;
- статистика за броя студенти.

Но ще продължим **една стъпка наведнъж**, както досега.

1. може ли преди да пристъпим към Details да направим една промяна само в таблицата да сложим пореден номер и да си оставим id-то и ако работим в таблицата при нужда ще го крием и ще го правим недостъпно за промяна

2. ChatGPT каза:

Да, разбира се. 👍 И според мен това е **по-добрият вариант**.

Нека запазим Id като вътрешен идентификатор на базата, а в таблицата на сайта да имаме отделна колона **No**, която е само за визуално подреждане.

Например:

No	First Name	Last Name	Email	Course	Age	Actions
1	Иван	Петров	...	...	20	Edit / Delete
2	Мария	Иванова	...	...	21	Edit / Delete

При това:

- **No** → визуален пореден номер за потребителя.
- **Id** → остава в базата и в модела Student.
- **Id не го показваме в таблицата.**
- **Id не може да се променя от потребителя.**
- При Edit и Delete продължаваме да използваме истинския Id вътрешно.
- Не променяме базата и **не ни трябва нова Migration.**

Как ще направим поредния номер?

В Index.cshtml можем да използваме брояч, например:

```
@{
    var number = 1;
}
```

и при всеки ред:

```
<td>@number</td>
```

след което увеличаваме:

```
number++;
```

Така No е само визуален номер и **не е част от базата.**

Това е важно, защото ако изтрием студент No2, например:

No 1 → Иван

No 2 → Мария

No 3 → Петър

след изтриването може да стане:

No 1 → Иван

No 2 → Петър

докато истинските Id стойности в базата могат да бъдат:

Id 1 → Иван

Id 3 → Петър

И това е напълно нормално. **Не трябва да променяме Id, за да поддържаме визуална номерация.**

Ако си съгласен, ще променим **само** Index.cshtml, без да пишем Controller, Model или SQL Server.

1. ок

2. ChatGPT каза:

Супер. 👍 Ще направим само промяната в Index.cshtml.

Стъпка 5.32 — добавяме пореден №

Отвори:

Views → Students → Index.cshtml

Първо ще ти покажа какво правим, без да променяме Student.cs или базата.

1. Добавяме брояча

В началото на файла, след @model, добави:

```
@{  
    var number = 1;  
}
```

2. Добавяме колоната №

В <thead> най-отпред добави:

```
<th>№</th>
```

3. Добавяме номера към всеки ред

Във foreach, преди името на студента, добави:

```
<td>@number</td>
```

4. Увеличаваме номера

В края на всеки ред, след Actions колоната, добави:

```
@{  
    number++;  
}
```

В крайна сметка

Твоята таблица трябва да изглежда приблизително така:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
@{  
    var number = 1;  
}
```

```
<h1>Students</h1>
```

```
<p>
```

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
</p>
```

```
<table class="table">
```

```
<thead>
```

```
<tr>
```

```
<th>No</th>
```

```
<th>First Name</th>
```

```
<th>Last Name</th>
```

```
<th>Email</th>
```

```
<th>Course</th>
```

```
<th>Age</th>
```

```
<th>Actions</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
@foreach (var student in Model)
```

```
{
```

```
<tr>
```

```
<td>@number</td>
```

```
<td>@student.FirstName</td>
```

```
<td>@student.LastName</td>
```

```
<td>@student.Email</td>
```

```
<td>@student.Course</td>
```

```
<td>@student.Age</td>
```

```
<td>
```

```
<a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
```

```
<a asp-action="Delete" asp-route-id="@student.Id" class="btn btn-danger">Delete</a>
```

```
</td>
```

```
</tr>
```

```
number++;
```

```
}
```

```
</tbody>
```

</table>

Важно

Тук:

<td>@number</td>

показваме **поредния номер**.

А тук:

asp-route-id="@student.Id"

продължаваме да използваме **истинското Id** за Edit/Delete.

Тоест двете са напълно отделни:

№ → само за показване

Id → вътрешен идентификатор

Не променяме Student.cs, ApplicationDbContext **или SQL Server**.

След като го направиш:

Ctrl + S → Ctrl + Shift + B

Изпрати ми Build резултата и ще проверим таблицата.

1. Build started at 10:50 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU ----- 1>StudentManagement -> D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 10:50 AM and took 06.531 seconds =====

2. ChatGPT каза:

Перфектно!  Build-ът е успешен.

Сега стартирай приложението и отвори **Students**.

Трябва да видиш:

№	First Name	Last Name	Email	Course	Age	Actions
1	...	...	...	...	...	Edit / Delete
2	...	...	...	...	...	Edit / Delete

Ако имаш само един студент, ще има:

№ 1

Проверка

Изтрий или добави студенти и ще видиш, че **№** се изчислява отново при зареждането на страницата.

Например ако базата има:

Id 1 → Иван

Id 3 → Петър

Id 7 → Мария

на сайта ще виждаш:

№ 1 → Иван

№ 2 → Петър

№ 3 → Мария

Точно това искаме. 👍

И най-важното — **не сме променили Id в базата.**

Когато потвърдиш, че таблицата изглежда правилно, продължаваме към **Details**.

Влизане

1. Вие казахте:

може ли оценка <https://github.com/esp-nv>

2. ChatGPT каза:

Да. Разгледах профила ти в GitHub: esp-nv / GitHub. В момента профилът е позициониран като **Junior Developer | C# / .NET |**

Web Development, а имаш 4 публични repository-та и 1 star.  GitHub

Оценка: 6.5/10 за Junior C#/.NET кандидат

Има **добра основа**, но в момента профилът изглежда повече като **учебно портфолио**, отколкото като портфолио на човек, готов за работа.

Какво е добре

- **Посоката е ясна** — C#, .NET, SQL Server, Entity Framework.
- Имаш отделено внимание на **OOP, SOLID, Clean Code и refactoring**, което е правилно за .NET позиция.  GitHub
- README-то е чисто и професионално за junior.
- Добре е, че показваш и **HTML/CSS/JavaScript**, защото това подсказва, че можеш да разбираш целия web stack.
- Харесва ми изречението, че използваш AI като помощник, но целта ти е да разбираш и прилагаш нещата самостоятелно. Това е много по-добре от това просто да показваш „AI-generated projects“.  GitHub

Какво те дърпа надолу

Основният проблем е **липсата на силен реален проект**.

Ако аз съм hiring manager и отворя профила, виждам:

„Уча C#, .NET, SQL Server, EF, OOP...”

но искам да видя:

„Ето приложение, което съм изградил от начало до край.”

Например:

ASP.NET Core Web API + EF Core + SQL Server + JWT Authentication + React/HTML/JS frontend + Docker + tests

дори едно такова добре направено приложение би променило доста впечатлението.

Как бих го направил

Бих подредил портфолиото така:

6. **1 голям проект** ☆☆☆☆☆

Например система за управление на задачи, inventory, booking или expense tracker.

7. **2–3 средни проекта** ☆☆☆☆☆

Всеки да демонстрира конкретно умение:

- EF Core + SQL
- REST API
- Authentication/Authorization
- LINQ
- async/await
- testing

8. **CSharp repository** като учебна база

Това е полезно, но не трябва да бъде главната звезда на профила.

9. **Software-Engineering repository** като допълнение

Добро е за показване на мислене и принципи, но работодателят по-скоро ще гледа реалния код.

Най-важното

Ти вече си направил нещо правилно: **показваш какво учиш**.

Следващата стъпка е да започнеш да показваш **какво можеш да построиш**.

Ако целта ти е Junior .NET работа, бих ти дал следния roadmap:

C# → OOP → LINQ → async/await → SQL → EF Core → ASP.NET Core Web API → Authentication/JWT → testing → Git → Docker → един сериозен проект

И тогава бих очаквал профилът ти да стигне около **8–9/10 за Junior**, ако проектът е качествен.

Ако искаш, мога да направя и **много по-подробен code review на самите 4 repository-та** — структура, C# код, naming, OOP, SOLID, EF/SQL, README-та и какво конкретно бих променил, за да изглеждат като портфолио за кандидатстване.

3. Вие казахте:

<https://github.com/esp-nv/CSharp/tree/main/06.CSharp-EntityFramework/03.Entity-Relations/Author-Solutions/01-Student-System> може ли с теб да изградим проект asp.net core mvc

4. ChatGPT каза:

Да — и според мен това е много добър проект за следващата ти стъпка. Видях конкретния Student-System в repository-то ти. Той в момента е EF Core console-style проект с Data и StartUp.cs, така че можем да вземем **модела и релациите като основа**, но да изградим истинско **ASP.NET Core MVC приложение от нулата**,

вместо просто да прехвърляме кода.  GitHub Microsoft също има актуална поредица за ASP.NET Core MVC + EF Core, включваща CRUD, relationships, migrations и работа с related



data. Microsoft Learn+1

Какво предлагам да направим

Да направим проект примерно:

Student Management System

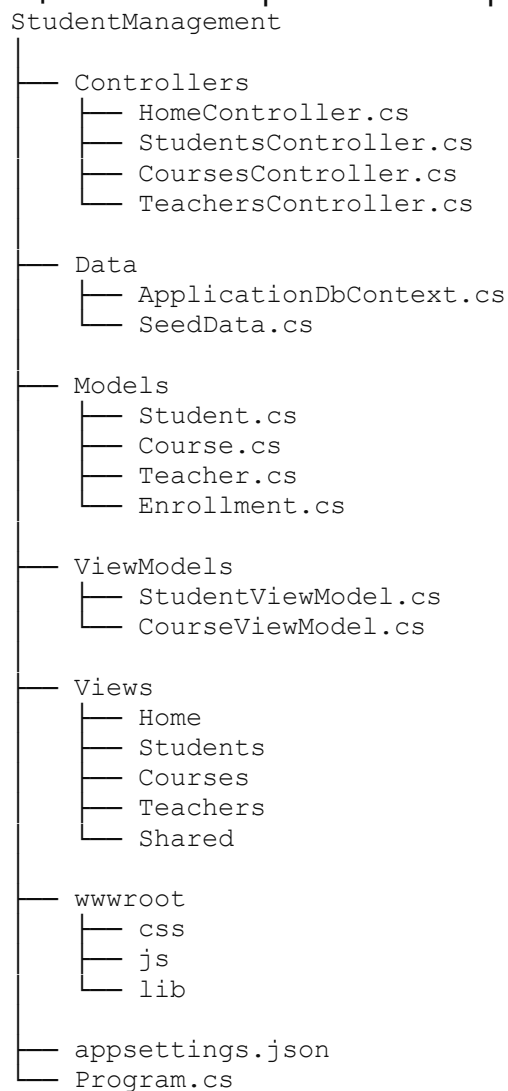
с:

- ASP.NET Core MVC
- C#
- Entity Framework Core
- SQL Server
- Razor Views
- Bootstrap
- Dependency Injection
- LINQ
- async/await
- CRUD
- Entity Relationships
- Validation
- Migrations
- ViewModels

- Search/Filtering
- Authentication/Authorization — на по-късен етап

Архитектура

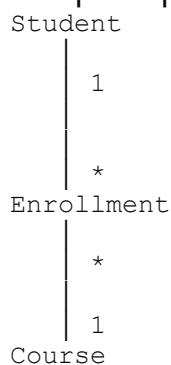
Ще започнем сравнително просто:



Релациите

Тук вече ще използваме това, което си учил в Student-System.

Например:



Тоест:

Student → Enrollment → Course

Това е много полезно, защото няма да учиш MVC отделно от EF Core — ще видиш **как EF Core relationships се използват в реално web приложение.**

Искам да го направим заедно, не да ти дам готов проект

Това според мен е най-важната част.

Можем да работим така:

Аз:

Създай Student.cs с тези properties...

Ти го пишеш.

После ми показваш кода.

Аз:

Проверявам го → обяснявам всяка грешка → подобряваме го.

След това:

1. Create project
- ↓
2. Models
- ↓
3. DbContext
- ↓
4. SQL Server
- ↓
5. Migrations
- ↓
6. Controllers
- ↓
7. Views
- ↓
8. CRUD
- ↓
9. Relationships
- ↓
10. ViewModels
- ↓
11. Validation
- ↓
12. Search
- ↓
13. Authentication
- ↓
14. Polish + GitHub

Така накрая няма просто да имаш проект, а ще можеш **да обясниш на интервю защо всяка част е написана по този начин.**

И още нещо важно

Не бих копирал директно стария Student-System.

Ще го използваме като **учебна основа**, но ще направим нов проект с актуален ASP.NET Core MVC. Това е по-смислено и за GitHub портфолиото ти.

Първата задача

Ако си готов, започваме **от нулата.**

Кажете ми само коя среда използвате:

- **Visual Studio 2022**
- **VS Code**
- **JetBrains Rider**

и коя версия на .NET имате (dotnet --version).

После ще направим **Стъпка 1: създаване на ASP.NET Core MVC проекта**, като вървим бавно и обяснявам *защо* правим всяко нещо.



5. Вие казахте:

стъпка по стъпка

6. ChatGPT каза:

Разбира се. Ще го направим **наистина стъпка по стъпка** — без да прескачаме и без да ти давам целия проект наведнъж.

Ще използваме ТВОЯ Student-System като основа за идеята, но ще изградим ново **ASP.NET Core MVC** приложение.

Стъпка 1 — Създаваме проекта

Първо ми трябва да знам с какво работиш:

Visual Studio 2022, VS Code или **Rider**?

Ако си с Visual Studio 2022, ще започнем така:

1. Отвори **Visual Studio 2022**.
2. Избери **Create a new project**.
3. В търсачката напиши:

ASP.NET Core Web App (Model-View-Controller)

4. Избери точно този template.

5. Натисни **Next**.

Настройки

На следващия екран например:

Project name:
StudentManagement

Location:
[папката, в която си държиш проектите]

Solution name:
StudentManagement

Натисни **Next**.

На следващия екран:

Framework: .NET 10.0
Authentication type: None
Configure for HTTPS: ✓
Enable Docker: X

Ако при теб няма .NET 10.0, **не избирай на сяпо** — кажи ми какви версии ти показва.

След това:

Create

След създаването

Трябва да получиш нещо подобно:

```
StudentManagement
├── Controllers
├── Models
├── Views
├── wwwroot
├── Properties
├── appsettings.json
├── Program.cs
└── StudentManagement.csproj
```

Това е нашата начална точка.

Не прави следващите стъпки още.

Кажи ми „**готово**“ и продължаваме със **Стъпка 2** —

разглеждаме какво

представяват Models, Views, Controllers **и** Program.cs, преди да напишем първия код.

7. Вие казахте:

Visual Studio 2022

8. ChatGPT каза:

Супер 👍 Ще работим с **Visual Studio 2022** и ще вървим бавно, стъпка по стъпка.

Стъпка 1 — Проверяваме .NET версията

Преди да създадем проекта, искам да проверим какви .NET версии имаш.

В **Visual Studio 2022**:

1. Отвори Visual Studio.

2. От менюто избери **Create a new project**.
3. В полето за търсене напиши:

ASP.NET Core Web App (Model-View-Controller)

4. Избери този template.
5. Натисни **Next**.

Важно

На следващия екран ще видиш поле **Framework**.

Кажи ми **точно какво ти показва там**.

Например може да е:

```
.NET 8.0  
.NET 9.0  
.NET 10.0
```

или няколко от тях.

Не създавай проекта още. Просто ми напиши какви Framework версии виждаш и оттам продължаваме.

9. Вие казахте:

<https://github.com/esp-nv/StudentManagement> той е направен само трябва да се довърши но може стъпка и аз ще ти казвам ако съм готова, но може ли с коментари за да уча

10. ChatGPT каза:

Да, **точно така е по-добре**. Видях repository-то StudentManagement в GitHub — то вече е създадено като **ASP.NET Core MVC Student Management system**, така че

няма смисъл да създаваме нов проект.  GitHub

Ще го **довършим заедно стъпка по стъпка**, а не просто ще ти давам готов код.

И да — **ще слагаме коментари в кода**, но не прекалено много.

Ще обяснявам важните неща, например:

```
// DbContext represents a session with the database.  
// Through this class EF Core allows us to query and save entities.  
public class ApplicationDbContext : DbContext  
{  
}
```

После ще ти обяснявам **защо** е нужен DbContext, какво прави DbSet, защо използваме async, защо имаме Controller/View/Model и т.н.

Как ще работим

Ще имаме следния принцип:

Аз давам само една стъпка → ти я правиш → казваш „готово“ → проверяваме → продължаваме.

Ако получиш грешка, **не я заобикаляме**. Показваш ми грешката и я разбираме.

Целта е накрая да можеш сама да кажеш на интервю:

„Използвам EF Core за достъп до базата, Controller обработва HTTP request-а, View отговаря за представянето, а ViewModel ми позволява да не подавам директно Entity към View.“

Това е много по-ценно от това просто проектът да работи.

Стъпка 1 — Отваряме съществуващия проект В Visual Studio 2022:

1. Отвори:
File → Open → Project/Solution
2. Намери папката:
StudentManagement
3. Отвори:
4. `StudentManagement.sln`
5. Изчакай Visual Studio да зареди проекта.
6. В **Solution Explorer** разгледай какво има вътре.

Не променяй нищо още.

Искам само да ми изпратиш **снимка на Solution Explorer-а** или да копираш тук структурата, която виждаш.

Например:

```
StudentManagement
├── Controllers
├── Models
├── Views
├── wwwroot
├── Data
├── Program.cs
└── appsettings.json
```

Оттам ще направим **Стъпка 2: разбираме твоята текуща структура и проверяваме какво вече е направено**, преди да пишем какъвто и да е код.



Източници

11. Вие казахте:

да

12. ChatGPT каза:

Разбира се 😊 Кажи ми какво искаш да направим.

13. Вие казахте:

тя е като репо

14. ChatGPT каза:

Ако имаш предвид „тя е като репо“ като израз — можеш да ми дадеш малко контекст какво искаш да кажеш с „репо“, за да ти отговоря точно.

15. Вие казахте:

<https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement>

16. ChatGPT каза:

Да, вече виждам кое имаш предвид. Това е репото `esp-nv/StudentManagement`.

В папката `StudentManagement` има стандартна **ASP.NET Core MVC** структура:

- Controllers
- Models
- Views
- wwwroot
- Program.cs
- appsettings.json
- StudentManagement.csproj  github.com

Ако това е проектът, за който каза „**тя е като репо**“, кажи ми какво точно искаш да направим с него — например да го **разгледаме, обясним как работи, поправим грешка или да добавим функционалност**.



Източници

17. Вие казахте:

да добавим функционалност.

18. ChatGPT каза:

Да 👍 Можем да добавим функционалност директно върху този проект. Прегледах структурата на репото — то е ASP.NET Core

MVC проект с Controllers, Models, Views, wwwroot, Program.cs и .csproj.  GitHub

Кажи ми **каква функционалност трябва да добавим**. Например:

- 🔍 търсене на студенти

-  оценки и изчисляване на среден успех
-  присъствие/отсъствия
-  профил на студент
-  добавяне/редактиране/изтриване на студенти
-  login/register и роли
-  предмети и записване на студенти към предмети
-  dashboard със статистики
-  генериране на справка/документ
- или нещо конкретно от заданието ти

Ако имаш задание, просто го изпрати тук (текст или снимка) и ще го направим **стъпка по стъпка върху точно това репо**, включително кои файлове да промениш и готовия код.



Източници

19. Вие казахте:

но стъпка по стъпка

20. ChatGPT каза:

Разбира се. Ще го правим **много бавно и стъпка по стъпка**, без да прескачаме. Ще променяме по нещо, ще го пускаш и ще проверяваме дали работи.

Вече потвърдих структурата на проекта —

ИМАМЕ Controllers, Models, Views, wwwroot, Program.cs И .csproj.  GitHub

Стъпка 1

Първо ми кажи **каква точно функционалност трябва да добавим**.

Ако имаш условие от преподавател/задание, **копирай го тук или изпрати снимка**.

След това ще ти кажа само:

- 1. кой файл да отвориш**
- 2. какво точно да добавиш**
- 3. как да го стартираш**
- 4. какво трябва да видиш**

И чак когато потвърдиш, че първата стъпка работи, продължаваме към **Стъпка 2**.



Източници

21. Вие казахте:

правим заедно с теб Student Management System

22. ChatGPT каза:

Да. Ще го направим **заедно от нулата, стъпка по стъпка**, като използваме точно твоето репо StudentManagement в GitHub.

В момента проектът има основната ASP.NET структура

— Controllers, Models, Views, wwwroot, Program.cs и Т.Н.  GitHub

Планът

Ще изградим **Student Management System**, като постепенно добавяме:

1.  Students — добавяне, преглед, редактиране и изтриване
2.  Search — търсене на студент
3.  Courses/Subjects — предмети
4.  Grades — оценки
5.  Average Grade — автоматично изчисляване на среден успех
6.  Attendance — присъствия
7.  Student Details — подробна страница за студент
8.  По-добър интерфейс
9.  Database — ако текущият проект още не е настроен правилно за нея
10.  Login/roles — ако искаме системата да има администратор и потребители

Но няма да правим всичко наведнъж. Първо ще направим една малка част, ще я стартираш и ще проверим.

Стъпка 1 — кажи ми само това

Отвори проекта в **Visual Studio** или **Visual Studio Code** и го стартирай с Ctrl + F5 / dotnet run.

След това ми кажи:

„Стартира“ — ако сайтът се отваря,
или

„Дава грешка“ — и ми копирай грешката.

Оттам започваме **Стъпка 2**.



Източници

23. Вие казахте:

ГОТОВО

24. ChatGPT каза:

Супер! 🚀 Започваме.

Стъпка 2 — да видим какво вече имаме

Преди да пишем нов код, ще разгледаме съществуващия проект, за да не счупим нещо.

В **Solution Explorer** намери папката:

Models

и ми напиши **какви файлове има вътре**.

Например:

```
Models
├── Student.cs
└── ...
```

Само ми изпрати имената на файловете в Models. След това ще направим първата функционалност.

25. Вие казахте:

[https://github.com/esp-](https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement)

[nv/StudentManagement/tree/main/StudentManagement](https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement) ето линк

26. ChatGPT каза:

Да, точно този линк ми трябва. 👍 Проверих го директно и виждам структурата на проекта.

В StudentManagement има Controllers, Models, Views, wwwroot, Program.cs и

останалите файлове.  GitHub

Нека **не гадаем какво има вътре**, а да започнем от реалния код.

Стъпка 2.1

Натисни в GitHub:

Models → Student.cs

ако има такъв файл.

Ако няма Student.cs, натисни Models и ми изпрати **какви файлове виждаш вътре**.

Може просто да ми копираш списъка, например:

```
Models
- Student.cs
- ErrorViewModel.cs
```

След това ще ти кажа **точно какво да направиш в първия файл**. Няма да прескачаме напред.



Източници

27. Вие казахте:

няма файл-а Student.cs

28.ChatGPT каза:

Точно така — тогава започваме от **модела** Student. Това е логичната първа стъпка за Student Management System. 👍

Стъпка 2 — създаваме Student.cs

Във Visual Studio:

1. В **Solution Explorer** намери папката **Models**.
2. Натисни **десен бутон** върху Models.
3. Избери **Add → Class...**
4. Име на файла:
5. Student.cs
6. Натисни **Add**.

В новия Student.cs ще има някакъв готов код. **Изтрий всичко и постави:**

```
using System.ComponentModel.DataAnnotations;

namespace StudentManagement.Models
{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string FirstName { get; set; }

        [Required]
        public string LastName { get; set; }

        [Required]
        [EmailAddress]
        public string Email { get; set; }

        public string Course { get; set; }

        public int Age { get; set; }
    }
}
```

Какво направихме?

Създадохме обект **Student**, който има:

- Id — уникален номер
- FirstName — име
- LastName — фамилия
- Email — имейл
- Course — курс/специалност
- Age — възраст

[Required] означава, че полето не трябва да бъде празно,
а [EmailAddress] проверява дали имейлът е във валиден формат.

Сега не прави нищо друго. Запази файла с Ctrl + S и ми напиши „**готово**“. След това продължаваме със следващата стъпка.

29. Вие казахте:

готово и е качен в gitHub

30. ChatGPT каза:

Супер! 🎉 Значи първата част е готова и вече имаме Student.cs в GitHub.

Стъпка 3 — създаваме базата с данни

Сега ще направим така, че приложението да може да **съхранява студентите**.

Първо трябва да проверим дали проектът вече има DbContext.

Направи само това:

В **Solution Explorer** потърси папка или файл с име:

Data

или файл:

ApplicationDbContext.cs

или:

StudentManagementContext.cs

Ако намериш такъв, **не го променяй**.

Просто ми кажи какво намери, например:

Имам папка Data и вътре ApplicationDbContext.cs

или

Нямам нищо такова.

После ще направим **само следващата стъпка**. 🚀

31. Вие казахте:

Нямам нищо такова.

32. ChatGPT каза:

Перфектно. 👍 Тогава ще създадем базата сами.

Стъпка 3.1 — инсталираме Entity Framework Core

Ще използваме **Entity Framework Core**, за да може нашият Student модел да се записва в база данни.

Ако си във **Visual Studio**:

1. От горното меню избири **Tools** → **NuGet Package Manager** → **Manage NuGet Packages for Solution**.

2. Отвори **Browse**.
3. Потърси:
4. `Microsoft.EntityFrameworkCore.SqlServer`
5. Инсталирай пакета в проекта **StudentManagement**.

След това потърси и инсталирай:

`Microsoft.EntityFrameworkCore.Tools`

Важно

Не инсталирай още други пакети и **не променяй кода**.

Когато и двата пакета са инсталирани, ми напиши:

„Инсталирани са“

и продължаваме със следващата малка стъпка. 

33. Вие казахте:

може ли десен бутон върху самия solution от там да избира
Manage NuGet Packages for Solution.

34. ChatGPT каза:

Да, **точно така**.  Даже това е удобен начин.

Направи следното:

1. В **Solution Explorer** натисни **десен бутон върху Solution** StudentManagement.
2. Избери:
Manage NuGet Packages for Solution...
3. В таба **Browse** потърси:
`Microsoft.EntityFrameworkCore.SqlServer`
4. Избери твоя проект StudentManagement и натисни **Install**.
5. После потърси:
`Microsoft.EntityFrameworkCore.Tools`
6. Отново избери проекта StudentManagement и натисни **Install**.

След инсталацията трябва да ги виждаш в:

Dependencies → Packages

като приблизително:

`Microsoft.EntityFrameworkCore.SqlServer`

`Microsoft.EntityFrameworkCore.Tools`

Когато ги инсталираш, кажи ми **„ГОТОВО“** и продължаваме. 

35. Вие казахте:

`Microsoft.EntityFrameworkCore.SqlServer` дава версия 10.0.11

36. ChatGPT каза:

Да, това е нормално — **но не я инсталирай още.** 😊

Версия **10.0.11** е от новото поколение EF Core и е важно да съобразим версията с **.NET версията на твоя проект.** Ако проектът е например на .NET 8, по-добре ще използваме EF Core 8; ако е .NET 9 — EF Core 9 и т.н.

Направи само това:

В Solution Explorer натисни десен бутон върху проекта:

StudentManagement → **Properties**

После намери:

Target Framework

и ми кажи какво пише там.

Например:

.NET 8.0

или

.NET 9.0

или

.NET 10.0

Не инсталирай пакета още. Кажи ми Target Framework-а и ще ти кажа точно коя версия да избереш.

37. Вие казахте:

8.0

38. ChatGPT каза:

Перфектно 👍 Проектът е на **.NET 8.0**, затова ще използваме **EF Core 8**, а не 10.

Стъпка 3.1 — правилната версия

В **Manage NuGet Packages for Solution**:

1. Потърси:

Microsoft.EntityFrameworkCore.SqlServer

2. Намери падащото меню за **Version**.

3. Избери най-новата версия, започваща с **8.0** — например 8.0.x.

4. Отбележи проекта StudentManagement.

5. Натисни **Install**.

След това потърси:

Microsoft.EntityFrameworkCore.Tools

и пак избери **8.0.x** и инсталирай.

Важно

Не избирай:

10.0.11 ✗

Избираме:

8.0.x ☒

След като и двата пакета са инсталирани, **не прави следващата стъпка още.**

Напиши ми „**ГОТОВО**“, когато приключиш.

39. Вие казахте:

8.0.30

40. ChatGPT каза:

Да! ☒ **8.0.30 е правилният избор** за проект на **.NET 8.0.**

Избери **8.0.30** за:

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

и ги инсталирай в проекта **StudentManagement**.

След това провери в:

Dependencies → Packages

да виждаш нещо подобно:

```
Microsoft.EntityFrameworkCore.SqlServer 8.0.30
Microsoft.EntityFrameworkCore.Tools      8.0.30
```

Като ги видиш и двата, кажи „**ГОТОВО**“. Тогава ще

създадем ApplicationDbContext.cs. 🚀

41. Вие казахте:

ГОТОВО

42. ChatGPT каза:

Супер! 🚀 Вече имаме модела Student и Entity Framework Core 8.0.30.

Стъпка 3.2 — създаваме ApplicationDbContext

Сега ще направим връзката между нашия Student клас и базата данни.

1. Създай папка Data

В **Solution Explorer**:

- десен бутон върху проекта **StudentManagement**
- **Add → New Folder**
- кръсти папката:

Data

2. Създай ApplicationDbContext.cs

Десен бутон върху новата папка **Data**:

Add → Class...

Име:

ApplicationDbContext.cs

3. Замени кода

Изтрий всичко във файла и постави:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Models;

namespace StudentManagement.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
        ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}
```

Какво направихме?

Този ред:

```
public DbSet<Student> Students { get; set; }
```

казва на Entity Framework:

„В базата данни искам таблица за студентите.“

Следващата стъпка ще бъде да **свържем този DbContext с SQL Server**, но засега спираме тук.

Запази с **Ctrl + S** и ми кажи „**готово**“.

43. Вие казахте:

може ли стъпка по стъпка за начинаещ и с коментари

44. ChatGPT каза:

Разбира се. 🙌 Оттук нататък ще работим **като с човек, който тепърва започва с ASP.NET**, и няма да прескачаме стъпки.

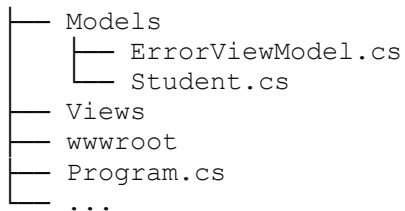
Ще ти обяснявам:

- **къде да натиснеш;**
- **какво да създадеш;**
- **какво да напишеш;**
- **какво прави всеки ред** чрез коментари;
- и след всяка малка стъпка ще спираме, за да провериш.

Стъпка 3.2 — папката Data

В момента трябва да имаш нещо подобно:

```
StudentManagement
├── Controllers
```



1. Създаваме папка

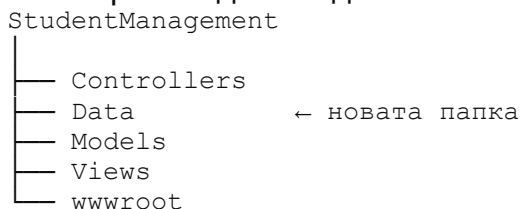
В **Solution Explorer**:

6. Намери проекта **StudentManagement**.
7. Натисни върху него с **десен бутон**.
8. Избери **Add** → **New Folder**.
9. Напиши:

Data

5. Натисни **Enter**.

Вече трябва да виждаш:



Спираме тук 🛑

Не създавай файла още.

Напиши ми само **„имам Data“**, когато папката е създадена. След това ще направим **само файла** ApplicationDbContext.cs и ще ти обясня ред по ред какво означава.

45. Вие казахте:

```

<Project Sdk="Microsoft.NET.Sdk.Web"> <PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings> </PropertyGroup>
  <ItemGroup> <PackageReference
    Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.30"
  /> <PackageReference
    Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.30">
    <PrivateAssets>all</PrivateAssets> <IncludeAssets>runtime; build;
    native; contentfiles; analyzers; buildtransitive</IncludeAssets>
  </PackageReference> </ItemGroup> <ItemGroup> <Folder
    Include="Data\" /> </ItemGroup> </Project>
  
```

46. ChatGPT каза:

Да — **точно така трябва да изглежда**. 👍 Виждам, че:

- проектът е на **.NET 8**;
- Microsoft.EntityFrameworkCore.SqlServer е **8.0.30**;
- Microsoft.EntityFrameworkCore.Tools е **8.0.30**;
- папката Data вече е създадена.

Тоест сме готови за следващата малка стъпка.

Стъпка 3.3 — създаваме ApplicationDbContext.cs

Сега **само това**:

5. В Solution Explorer намери папката **Data**.
6. Десен бутон върху Data.
7. Избери **Add** → **Class...**
8. Напиши име:

ApplicationDbContext.cs

5. Натисни **Add**.

Вътре ще се появи някакъв код.

Изтрий кода

Изтрий всичко, което Visual Studio е сложило автоматично.

След това постави този код:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Models;

namespace StudentManagement.Data
{
    // Този клас управлява връзката между нашето приложение
    // и базата данни.
    public class ApplicationDbContext : DbContext
    {
        // Конструкторът получава настройките за базата данни.
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        // Това представлява таблицата Students в базата данни.
        // Всеки запис в нея ще бъде един Student.
        public DbSet<Student> Students { get; set; }
    }
}
```

Какво означава най-важната част?

Това:

```
public DbSet<Student> Students { get; set; }
```

означава приблизително:

„Искам в базата данни да имам колекция/таблица, която съдържа Student обекти.“

Тоест по-късно ще получим нещо подобно:

Students

Id	FirstName	LastName	Email	Course	Age
1	Ivan	Petrov	...	IT	20
2	Maria	Ivanova	...	CS	21

Не се притеснявай, ако все още не виждаш тази таблица. На този етап само сме описали как трябва да изглежда тя.

Сега спираме. 🛑

Запази с **Ctrl + S**.

След това ми кажи само:

„ApplicationDbContext.cs е готов“

И чак тогава ще направим следващата стъпка — ще свържем ApplicationDbContext с приложението в Program.cs.

47. Вие казахте:

може ли едно уточнение

48. ChatGPT каза:

Разбира се. Кажи какво искаш да уточним. 😊

49. Вие казахте:

кода които ще слагаме там да няма коментари а те да са отвън

50. ChatGPT каза:

Да, разбира се. 📄 Даже за начинаещ е по-удобно така.

Оттук нататък ще правим така:

Във файла — само чист код:

```
using Microsoft.EntityFrameworkCore;  
using StudentManagement.Models;
```

```
namespace StudentManagement.Data  
{  
    public class ApplicationDbContext : DbContext  
    {  
        public ApplicationDbContext(  
            DbContextOptions<ApplicationDbContext> options)  
            : base(options)  
        {  
        }  
  
        public DbSet<Student> Students { get; set; }  
    }  
}
```

А извън кода — аз ще обяснявам подробно, например:

- using Microsoft.EntityFrameworkCore; — ПОЗВОЛЯВА НИ ДА ИЗПОЛЗВАМЕ Entity Framework Core.

- using StudentManagement.Models; — позволява на този файл да вижда класа Student.
- ApplicationDbContext — класът, който ще управлява връзката с базата данни.
- DbSet<Student> Students — казва, че ще имаме таблица за студентите.

Така кодът ще остане **чист и професионален**, а обясненията ще са отделно.

Ще спазваме това правило за **всички следващи стъпки**. 

51. Вие казахте:

така става

52. ChatGPT каза:

Да, точно така. 

Продължаваме по този начин:

1. Давам ти **една малка стъпка**.
2. Давам ти **чист код без коментари**.
3. След кода обяснявам **всеки важен ред на разбираем език**.
4. Ти го правиш и ми казваш „**готово**“.
5. Едва тогава минаваме към следващата стъпка.

В момента сме на ApplicationDbContext.cs. Ако вече си поставил кода и си го запазил, следващото ще бъде **свързването му с Program.cs**.
 Кажете „**готово**“ и продължаваме. 

53. Вие казахте:

може ли и още едо уточнение само ако се налага чрез `/// <summary> /// /// </summary>`

54. ChatGPT каза:

Да, разбира се.  Това даже е **добра практика** в C#.

Оттук нататък ще използваме `/// <summary>` **само когато наистина има смисъл** — например за класове, публични методи или свойства, които е полезно да имат документация.

Няма да слагаме излишни коментари на всеки ред.

Например:

```
/// <summary>
/// Представява връзката между приложението и базата данни.
/// </summary>
```

```
public class ApplicationDbContext : DbContext
{
    /// <summary>
    /// Създава нова инстанция на ApplicationDbContext.
    /// </summary>
    public ApplicationDbContext(
        DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

А когато не е необходимо описание, ще оставяме кода чист.

Нашето правило оттук нататък:

- ✗ Без // коментари вътре в кода, освен ако наистина са необходими.
- ✓ /// <summary> — само когато е полезно.
- ✓ Основните обяснения — **извън кода**, стъпка по стъпка.
- ✓ Няма да ти давам огромни парчета код наведнъж.

Готови сме да продължим с ApplicationDbContext.cs → Program.cs. 🚀

55. Вие казахте:

да

56. ChatGPT каза:

Супер. 👍 Продължаваме **бавно и само с една стъпка**.

Стъпка 3.4 — свързваме базата с Program.cs

Вече имаме:

```
Data
└─ ApplicationDbContext.cs
```

Сега трябва да кажем на ASP.NET Core:

„Използвай ApplicationDbContext и SQL Server.“

1. Отвори Program.cs

В **Solution Explorer** намери:

Program.cs

Отвори го.

Вероятно вътре вече имаш код, подобен на този:

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
// ...
```

Не изтривай целия файл.

2. Добавяме using

Най-отгоре на Program.cs добави:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

Това позволява на Program.cs да използва:

- EntityFrameworkCore
- НАШИЯ ApplicationDbContext

3. Добавяме регистрацията на базата

Намери този ред:

```
builder.Services.AddControllersWithViews();
```

Непосредствено преди него добави:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection"));
```

Така началото на Program.cs трябва да изглежда приблизително така:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection"));
```

```
builder.Services.AddControllersWithViews();
```

Какво направихме?

Тази част:

```
builder.Services.AddDbContext<ApplicationDbContext>
```

регистрира нашия ApplicationDbContext в ASP.NET Core.

А тази:

```
options.UseSqlServer(...)
```

казва:

„Ще използваме Microsoft SQL Server като база данни.“

А това:

```
builder.Configuration.GetConnectionString("DefaultConnection")
```

казва:

„Вземи настройките за връзка от appsettings.json, от connection string с име DefaultConnection.“

⚠ **Все още нямаме DefaultConnection в appsettings.json.** Това е нарочно — ще го направим в следващата стъпка.

Сега спираме. 🛑

Запази Program.cs с **Ctrl + S**.

Ако Visual Studio показва червени линии, **не се притеснявай** — възможно е да е заради липсващия connection string.

Кажи ми **„ГОТОВО“** и ще направим следващата стъпка: appsettings.json.

```
namespace StudentManagement { public class Program { public static
void Main(string[] args) { var builder =
WebApplication.CreateBuilder(args); // Add services to the container.
builder.Services.AddControllersWithViews(); var app = builder.Build();
// Configure the HTTP request pipeline. if
(!app.Environment.IsDevelopment()) {
app.UseExceptionHandler("/Home/Error"); // The default HSTS value is
30 days. You may want to change this for production scenarios, see
https://aka.ms/aspnetcore-hsts. app.UseHsts(); }
app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting();
app.UseAuthorization(); app.MapControllerRoute( name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}"); app.Run(); } }
```

ИМАМ ТОВА

58.ChatGPT каза:

Да 👍 Това е напълно нормален Program.cs за ASP.NET Core MVC.
Няма проблем, че при теб е с namespace StudentManagement И class Program.
Ще работим **върху твоя код**, без да го пренаписваме излишно.

Стъпка 3.4 — първо добавяме using

В самото начало на файла, преди:

```
namespace StudentManagement
```

добави:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

Така началото трябва да стане:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

```
namespace StudentManagement
{
    public class Program
    {
```

Защо?

Microsoft.EntityFrameworkCore ни дава достъп до Entity Framework Core.

StudentManagement.Data ни позволява да използваме нашия:

```
ApplicationDbContext
```

Стъпка 3.5 — добавяме връзката с базата

Сега намери:

```
builder.Services.AddControllersWithViews();
```

Непосредствено над него постави:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection"));
```

Следователно тази част от твоя Program.cs трябва да изглежда така:

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllersWithViews();
```

Какво означава това?

AddDbContext<ApplicationDbContext> казва на приложението:

Използвай нашия ApplicationDbContext.

UseSqlServer казва:

Базата данни ще бъде SQL Server.

GetConnectionString("DefaultConnection") казва:

Вземи информацията за връзката от appsettings.json, където по-късно ще създадем DefaultConnection.

Важно

При теб има готови коментари като:

```
// Add services to the container.
```

и:

```
// Configure the HTTP request pipeline.
```

Засега не ги пипай. Те са стандартни коментари, генерирани от Visual Studio. Когато стигнем до почистване на проекта, ще решим кои да оставим.

Също така **не променяй нищо друго** в Program.cs.

Направи само тези две промени и натисни **Ctrl + S**.

После ми покажи **целия** Program.cs, както го направи сега, за да го проверим заедно, преди да продължим.

59. Вие казахте:

```
using Microsoft.EntityFrameworkCore; using
StudentManagement.Data; namespace StudentManagement { public
class Program { public static void Main(string[] args) { var builder =
WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection"))); //
Add services to the container.
builder.Services.AddControllersWithViews(); var app = builder.Build();
// Configure the HTTP request pipeline. if
(!app.Environment.IsDevelopment()) {
app.UseExceptionHandler("/Home/Error"); // The default HSTS value is
30 days. You may want to change this for production scenarios, see
https://aka.ms/aspnetcore-hsts. app.UseHsts(); }
app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting();
```

```
app.UseAuthorization(); app.MapControllerRoute( name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}"); app.Run(); } }
```

60.ChatGPT каза:

Да — **кодът ти е правилен.** ✓ Няма грешка в добавената част.
Само бих оправил **подравняването**, за да бъде по-четим.
Функционално е същото.

Твоят Program.cs с по-добро форматиране

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

namespace StudentManagement
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            builder.Services.AddDbContext<ApplicationDbContext>(options =>
                options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")));

            // Add services to the container.
            builder.Services.AddControllersWithViews();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (!app.Environment.IsDevelopment())
            {
                app.UseExceptionHandler("/Home/Error");
                // The default HSTS value is 30 days. You may want to
change this for production scenarios, see https://aka.ms/aspnetcore-
hsts.

                app.UseHsts();
            }

            app.UseHttpsRedirection();
            app.UseStaticFiles();

            app.UseRouting();

            app.UseAuthorization();

            app.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}");

            app.Run();
        }
    }
}
```

Какво проверихме?

Имах правилно:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;
```

и:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection")));
```

Това е връзката между **ASP.NET Core** → **Entity Framework Core** → **SQL Server**.

⚠ Има само едно нещо: `DefaultConnection` **все още не съществува**.
Това ще го направим в `appsettings.json`.

Следваща стъпка

Не променяй нищо повече в `Program.cs`.

Запази файла с **Ctrl + S** и ми кажи „**готово**“.

После ще отворим `appsettings.json` и ще направим **само connection string-а**, като ще ти обясня какво представлява.

61. Вие казахте:

ГОТОВО

62. ChatGPT каза:

Супер! 👍 `Program.cs` е ГОТОВ.

Стъпка 3.6 — `appsettings.json`

Сега ще кажем на приложението **къде се намира SQL Server базата**.

1. Намери файла

В **Solution Explorer** намери:

`appsettings.json`

Обикновено е директно в проекта:

`StudentManagement`

```
|
|— Controllers
|— Data
|— Models
|— Views
|— wwwroot
|— appsettings.json ← този файл
|— Program.cs
|— StudentManagement.csproj
```

Отвори `appsettings.json`.

2. Покажи ми какво има вътре

Засега нищо не променяй.

Копирай съдържанието на `appsettings.json` и го изпрати тук.

Ще го погледнем заедно и **след това** ще добавим `DefaultConnection`.

Така няма да рискуваме да изтрием нещо, което вече е настроено.



63. Вие казахте:

```
{ "Logging": { "LogLevel": { "Default": "Information",  
"Microsoft.AspNetCore": "Warning" } }, "AllowedHosts": "*" }
```

64. ChatGPT каза:

Перфектно. 📁 Файлът е чист и можем спокойно да добавим connection string.

Стъпка 3.7 — добавяме ConnectionStrings

В ТВОЯ appsettings.json в момента имаш:

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*" }  
}
```

Ще добавим ConnectionStrings **между** Logging **и** AllowedHosts.

Замени съдържанието на файла с:

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "ConnectionStrings": {  
    "DefaultConnection":  
"Server=(localdb)\\mssqllocaldb;Database=StudentManagementDb;Trusted_  
Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=  
True"  
  },  
  "AllowedHosts": "*" }  
}
```

Какво направихме?

Тази част:

```
"ConnectionStrings": {
```

съдържа настройките за връзка с базата.

Тази част:

```
"DefaultConnection":
```

е името на връзката.

То е **същото име**, което използвахме в Program.cs:

```
builder.Configuration.GetConnectionString("DefaultConnection")
```

А това:

```
Server=(localdb)\\mssqllocaldb
```

казва, че ще използваме **SQL Server LocalDB**, който обикновено се използва за локална разработка с Visual Studio.

Това:

```
Database=StudentManagementDb
```

е името на нашата база данни.

И така:

```
ASP.NET Core
```

↓

ApplicationDbContext



SQL Server LocalDB



StudentManagementDb



Важна проверка

След като поставиш кода:

1. Натисни **Ctrl + S**.
2. Виж дали appsettings.json има **червени вълнообразни линии**.
3. Засега **не стартирай миграция и не създавай база**.

Кажи ми само „**готово**“, ако няма грешки, и следващата ни стъпка ще бъде да проверим дали проектът се **компилира успешно** преди да създаваме базата.

65. Вие казахте:

ГОТОВО

66. ChatGPT каза:

Супер! 🎉 Вече имаме трите основни части:

Student



ApplicationDbContext



Connection String

Сега **няма да правим база данни още**. Първо ще проверим дали всичко, което направихме досега, се компилира.

Стъпка 3.8 — проверяваме проекта

Във Visual Studio:

1. От менюто избери **Build**.
2. Натисни **Build Solution**.

Или използвай клавишната комбинация:

Ctrl + Shift + B

Какво трябва да видиш?

В долната част на Visual Studio ще се появи:

Build succeeded.

Ако видиш:

Build succeeded.

0 Error(s)

значи всичко дотук е наред. 

Ако има грешки

Не се опитвай да ги поправяш сам.

Копирай ми грешката **точно както я показва Visual Studio** и ще я разгледаме заедно.

Засега не прави нищо друго.

Направи Ctrl + Shift + B и ми кажи какво показва **Build**.

67. Вие казахте:

```
Build started at 9:28 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>D:\GitHub\StudentManagement\StudentManagement\Models\Stu
dent.cs(10,23,10,32): warning CS8618: Non-nullable property
'FirstName' must contain a non-null value when exiting constructor.
Consider adding the 'required' modifier or declaring the property as
nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Stu
dent.cs(13,23,13,31): warning CS8618: Non-nullable property
'LastName' must contain a non-null value when exiting constructor.
Consider adding the 'required' modifier or declaring the property as
nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Stu
dent.cs(17,23,17,28): warning CS8618: Non-nullable property 'Email'
must contain a non-null value when exiting constructor. Consider
adding the 'required' modifier or declaring the property as nullable.
1>D:\GitHub\StudentManagement\StudentManagement\Models\Stu
dent.cs(19,23,19,29): warning CS8618: Non-nullable property 'Course'
must contain a non-null value when exiting constructor. Consider
adding the 'required' modifier or declaring the property as nullable.
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll 1>Done building project
"StudentManagement.csproj". ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:28 PM and took 05.416 seconds =====
```

68. ChatGPT каза:

Перфектно! 🎉 **Проектът се компилира успешно.**

Най-важният ред е:

```
Build: 1 succeeded, 0 failed
```

Тоест **нямаме грешки**. Имаме само 4 предупреждения (warning).

Какви са предупрежденията?

Те са в Student.cs и са за:

- FirstName
- LastName
- Email
- Course

Причината е, че в проекта сме включили:

```
<Nullable>enable</Nullable>
```

Това кара C# да проверява дали string свойствата могат да бъдат null.

Например:

```
public string FirstName { get; set; }
```

C# казва:

„Ти ми казваш, че FirstName никога няма да бъде null, но не си дал начална стойност.“

Ще го поправим ли?

Да, но нека го направим сега, преди да продължим. Това е добра практика и ще изчистим предупрежденията.

Отвори:

```
Models  
└─ Student.cs
```

и ми изпрати **текущото съдържание на** Student.cs.

Ще го коригираме заедно, като запазим правилото ни: **чист код без обикновени коментари, а обясненията — извън кода.**

69. Вие казахте:

```
using System.ComponentModel.DataAnnotations; namespace  
StudentManagement.Models { public class Student { public int Id {  
get; set; } [Required] public string FirstName { get; set; } [Required]  
public string LastName { get; set; } [Required] [EmailAddress] public  
string Email { get; set; } public string Course { get; set; } public int Age  
{ get; set; } } }
```

70. ChatGPT каза:

Да, това е текущият ти Student.cs. 👍

Ще поправим предупрежденията по **най-лесния начин за начинаещ.**

Стъпка 3.9 — поправяме предупрежденията

Замени съдържанието на Student.cs с това:

```
using System.ComponentModel.DataAnnotations;  
  
namespace StudentManagement.Models  
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string FirstName { get; set; } = string.Empty;  
  
        [Required]  
        public string LastName { get; set; } = string.Empty;  
  
        [Required]
```

```

        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        public string Course { get; set; } = string.Empty;

        public int Age { get; set; }
    }
}

```

Какво променихме?

Например преди имяхме:

```
public string FirstName { get; set; }
```

Сега имаме:

```
public string FirstName { get; set; } = string.Empty;
```

string.Empty означава **празен текст** ("").

Така казваме на C#:

„Това свойство започва с празна стойност, а не с null.“

Направихме същото с:

- FirstName
- LastName
- Email
- Course

Атрибутите:

```
[Required]
```

и:

```
[EmailAddress]
```

остават. Те са важни за валидацията на данните.

Сега направи само това

5. Замени кода.
6. Натисни **Ctrl + S**.
7. Натисни **Ctrl + Shift + B**.
8. Провери резултата.

Очакваме:

```
Build: 1 succeeded, 0 failed
```

и предупрежденията от Student.cs да са изчезнали.

Изпрати ми резултата от Build, както направи преди. После продължаваме. 🚀

71. Вие казахте:

```

Build started at 9:33 PM... ===== Build: 0 succeeded, 0 failed,
1 up-to-date, 0 skipped ===== Build
completed at 9:33 PM and took 00.033 seconds =====

```

72. ChatGPT каза:

Перфектно! ☒ Това също е **успешен резултат**.

Този път Visual Studio казва:

Build: 0 succeeded, 0 failed, 1 up-to-date

Какво означава up-to-date?

Visual Studio е проверил проекта и е установил:

„Нищо не се е променило, което да изисква ново компилиране.“

Тоест:

-  0 failed → **нямаме грешки**
-  1 up-to-date → проектът вече е компилиран и няма нужда от повторно компилиране
-  всичко е наред

Следваща стъпка — проверяваме с Rebuild

За да сме напълно сигурни след промяната в Student.cs, направи:

Build → Rebuild Solution

или натисни:

Ctrl + Alt + F7

Това ще компилира проекта **отначало**, вместо да използва вече компилираните файлове.

После ми изпрати резултата.

Очакваме нещо подобно:

Build: 1 succeeded, 0 failed

Ако няма warnings — още по-добре. 

73. Вие казахте:

```
using System.ComponentModel.DataAnnotations; namespace
StudentManagement.Models { public class Student { public int Id {
get; set; } [Required] public string FirstName { get; set; } =
string.Empty; [Required] public string LastName { get; set; } =
string.Empty; [Required] [EmailAddress] public string Email { get; set; }
= string.Empty; public string Course { get; set; } = string.Empty; public
int Age { get; set; } } }
```

74. ChatGPT каза:

Да — Student.cs **вече е напълно наред.** 

Точно този код искаме да имаме:

- Id — идентификатор на студента.
- FirstName — име.
- LastName — фамилия.
- Email — имейл.
- Course — курс/специалност.
- Age — възраст.

- [Required] — задължително поле.
- [EmailAddress] — проверява формата на имейла.
- = string.Empty — премахва предупрежденията за null.

Не променяй нищо повече в този файл.

Следващата ни цел

Вече имаме:

```
Student.cs
  ↓
ApplicationDbContext.cs
  ↓
Program.cs
  ↓
appsettings.json
```

Следващата важна стъпка е да **създадем базата данни чрез Migration**.

Но преди това ще направим една кратка проверка, за да сме сигурни, че ApplicationDbContext.cs е точно както трябва.

📩 Изпрати ми **текущия код от** ApplicationDbContext.cs и ще го проверим заедно.

75. Вие казахте:

```
Build started at 9:35 PM... ===== Build: 0 succeeded, 0 failed,
1 up-to-date, 0 skipped ===== Build
completed at 9:35 PM and took 00.029 seconds =====
```

76. ChatGPT каза:

Да, това е **ОК**. 

```
Build: 0 succeeded, 0 failed, 1 up-to-date
```

Най-важното е:

- **0 failed** → няма грешки.
- **1 up-to-date** → проектът вече е компилиран и Visual Studio няма какво ново да компилира.

Значи можем да продължим. 

Следваща стъпка — проверяваме ApplicationDbContext.cs

Преди да създадем базата, искам да проверим последната част от настройката.

Отвори:

```
Data
└─ ApplicationDbContext.cs
```

и ми копирай **целия код от файла** тук.

Ще го проверим ред по ред, и ако всичко е наред, ще преминем към **Migration**. 

77. Вие казахте:

```
using Microsoft.EntityFrameworkCore; using
StudentManagement.Models; namespace StudentManagement.Data {
public class ApplicationDbContext : DbContext { public
ApplicationDbContext( DbContextOptions<ApplicationDbContext>
options) : base(options) { } public DbSet<Student> Students { get; set;
} } }
```

78. ChatGPT каза:

Да, точно това искаме. ☒ ApplicationDbContext.cs е правилен.
Само ще направим **една малка подобрявка**, за да избегнем
евентуално предупреждение за Students.

Стъпка 3.10 — малка поправка

Промени:

```
public DbSet<Student> Students { get; set; }
```

на:

```
public DbSet<Student> Students { get; set; } = null!;
```

Целият файл ще бъде:

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Models;

namespace StudentManagement.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; } = null!;
    }
}
```

Защо = null!?

DbSet ще бъде създаден и зададен от **Entity Framework Core**.
C# обаче не знае това предварително и може да ни покаже
предупреждение за null.

null! означава:

„Знам какво правя — не предупреждавай компилатора за тази
стойност.“

Това е често използван подход при DbSet в Entity Framework Core.

Сега:

1. Направи промяната.
2. Натисни **Ctrl + S**.
3. Направи **Build → Rebuild Solution**.
4. Изпрати ми резултата.

След това, ако всичко е чисто, ще преминем към **първата Migration** и ще започнем реално да създаваме базата данни. 🚀

79. Вие казахте:

ctr+shift+b

80. ChatGPT каза:

Да 👍 Ctrl + Shift + B е **Build Solution**.
Направи сега:

1. Натисни **Ctrl + Shift + B**.
2. Изчакай да приключи.
3. Копирай резултата отдолу и ми го изпрати.

Ако получиш:

Build: 1 succeeded, 0 failed

или:

0 failed

значи сме добре. ✅

Не прави Migration още. Първо ще проверим резултата от Build.

81. Вие казахте:

```
Build started at 9:40 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:40 PM and took 01.057 seconds =====
```

82. ChatGPT каза:

Перфектно! 🎉

Това е точно резултатът, който искаме:

Build: 1 succeeded, 0 failed

И най-хубавото — **нямаме никакви warnings.** ✅

Дотук сме направили:

Student.cs

↓

ApplicationDbContext.cs

↓

Program.cs

↓

appsettings.json

↓

Build успешно ✅

Стъпка 4 — създаваме базата данни

Сега ще използваме **Migration**.

Migration е начинът, по който Entity Framework Core превръща моделите ни (Student) в структура на база данни.

Първата Migration ще бъде нещо като:

```
Student модел
    ↓
EF Core Migration
    ↓
Students таблица
    ↓
SQL Server база
```

Първо само отваряме конзолата

Във Visual Studio отиди на:

Tools → NuGet Package Manager → Package Manager Console

Ще се отвори прозорец в долната част на Visual Studio.

⚠ **Не въвеждай команда още.**

Когато видиш конзолата с нещо подобно на:

```
PM>
```

ми напиши „**отворих я**“.

След това ще ти дам **само първата команда** и ще ти обясня какво прави.

83. Вие казахте:

отворих я

84. ChatGPT каза:

Супер. 👍 Сега ще направим **само първата Migration**.

Стъпка 4.1 — създаваме първата Migration

В **Package Manager Console** трябва да виждаш:

```
PM>
```

До него напиши точно:

```
Add-Migration InitialCreate
```

и натисни **Enter**.

Какво прави тази команда?

Add-Migration казва на Entity Framework Core:

„Виж какви модели имам в проекта и създай описание на промените, които трябва да направим в базата.“

InitialCreate е просто **името на Migration-а**.

Можехме да го наречем и CreateDatabase, но InitialCreate е стандартно и ясно име за първата Migration.

Какво трябва да се случи?

Ако всичко е наред, Visual Studio ще създаде нова папка:

```
Migrations
```

с няколко файла вътре.

Например:

Migrations

```
├─ 20260829xxxxxx_InitialCreate.cs
├─ 20260829xxxxxx_InitialCreate.Designer.cs
└─ ApplicationDbContextModelSnapshot.cs
```

Не променяй тези файлове ръчно.

🔒 **След командата**

Не изпълнявай друга команда.

Копирай ми **целия резултат**, който се появи в Package Manager Console след:

```
Add-Migration InitialCreate
```

Ако има грешка — няма проблем. Ще я разгледаме заедно.

85. Вие казахте:

може ли да настроим и Microsoft SQL Server Manager - аз използвам 22 и connection string-a , server name->DESKTOP-O6FIRML\SQLEXPRESS022

86. ChatGPT каза:

Да, **може** — и според мен е по-добре да го настроим с твоя **SQL Server**, вместо с LocalDB. 👍

Твоят SQL Server instance е:

```
DESKTOP-O6FIRML\SQLEXPRESS022
```

И понеже използваш SQL Server Management Studio 22, ще настроим приложението да използва този SQL Server.

Важно преди да продължим

Не изпълнявай Add-Migration **още**. Първо ще сменя connection string-a.

Стъпка 4.1 — променяме appsettings.json

Отвори:

```
appsettings.json
```

В момента имаш нещо подобно:

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=StudentManagementDb;Trusted_
Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=
True"
}
```

Замени **само стойността на** DefaultConnection с:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=DESKTOP-
O6FIRML\SQLEXPRESS022;Database=StudentManagementDb;Trusted_Connectio
n=True;TrustServerCertificate=True"
}
```

Какво означава?

```
Server=DESKTOP-O6FIRML\SQLEXPRESS022
```

казва на приложението:

Свържи се с SQL Server instance SQLEXPRESS022 на моя компютър.

Database=StudentManagementDb

казва:

Базата, която ще използваме, се казва StudentManagementDb.

Trusted_Connection=True

означава:

Използвай Windows удостоверяването, с което си влязъл в Windows.

TrustServerCertificate=True

позволява на локалната връзка да приеме сертификата на SQL Server.

Защо има \\?

В JSON единичната обратна наклонена черта \ е специален символ.

Затова:

DESKTOP-O6FIRML\SQLEXPRESS022

се записва в JSON като:

DESKTOP-O6FIRML\\SQLEXPRESS022

Но реалното име на SQL Server остава:

DESKTOP-O6FIRML\SQLEXPRESS022

Засега спираме тук

Не изпълнявай Add-Migration още.

Направи промяната в appsettings.json, запази с **Ctrl + S** и ми кажи: „готово“

След това ще направим **проверка на връзката към** DESKTOP-O6FIRML\SQLEXPRESS022, преди да създаваме Migration-a.

87. Вие казахте:

за сега ще е local-но

88. ChatGPT каза:

Да, разбира се. 👍 **Оставаме с LocalDB засега.** Това е дори по-лесно за началото и няма нужда да настройваме SQL Server Express.

Значи **не променяме нищо** от текущия appsettings.json.

Оставаме с:

```
"ConnectionStrings": {  
  "DefaultConnection":  
    "Server=(localdb)\\mssqllocaldb;Database=StudentManagementDb;Trusted_  
Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=  
True"  
}
```

Това означава:

- LocalDB → локална база на твоя компютър.
- StudentManagementDb → името на нашата база.

- `Trusted_Connection=True` → използва Windows удостоверяване.
- Не е необходимо да стартираш SQL Server Management Studio, за да работи приложението.

Следващото

Вече сме готови за командата:

`Add-Migration InitialCreate`

Но ще я направим **само когато ти кажеш**.

Ако си оставил `appsettings.json` както беше с LocalDB, напиши „**ГОТОВО**“ и ще продължим с Migration-а. 🚀

89. Вие казахте:

DESKTOP-O6FIRML\SQLEXPRESS02 ми е сървара

90. ChatGPT каза:

Да, разбрах. 📖 Тогава имаме **SQL Server Express**, чийто Server Name е:

DESKTOP-O6FIRML\SQLEXPRESS02

Но щом каза, че **засега искаш да работим локално**, има два различни варианта:

- **LocalDB** — `(localdb)\mssqllocaldb`
- **Твоят SQL Server Express** — `DESKTOP-O6FIRML\SQLEXPRESS02`

И двата са локални на твоя компютър. Ако искаш базата да се вижда директно в **SQL Server Management Studio 22**, тогава е по-добре да използваме **твоя SQL Server Express**.

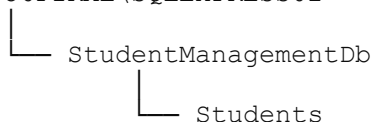
Ако изберем SQL Server Express

В `appsettings.json` ще бъде:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=DESKTOP-O6FIRML\SQLEXPRESS02;Database=StudentManagementDb;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "AllowedHosts": "*"
}
```

Това ще създаде/използва база:

DESKTOP-O6FIRML\SQLEXPRESS02



Моят съвет: щом вече имаш SQL Server Express и искаш да използваш SSMS 22, нека използваме него. Така после ще можеш да отваряш StudentManagementDb директно в SQL Server Management Studio и да виждаш таблицата Students.

Ако си съгласен, **следващата ни стъпка е само да сменим connection string-а** — няма да пускаме Migration още.

91. Вие казахте:

```
смених го { "Logging": { "LogLevel": { "Default": "Information",  
"Microsoft.AspNetCore": "Warning" } }, "ConnectionStrings": {  
"DefaultConnection": "Server=DESKTOP-  
O6FIRML\\SQLEXPRESS02;Database=StudentManagementDb;Trusted  
_Connection=True;TrustServerCertificate=True" }, "AllowedHosts": "*" }
```

92. ChatGPT каза:

Да.  Точно така трябва да изглежда appsettings.json.

Връзката вече е:

```
ASP.NET Core  
↓  
Entity Framework Core  
↓  
DESKTOP-O6FIRML\\SQLEXPRESS02  
↓  
StudentManagementDb
```

Нека проверим нещо важно

Преди да създаваме Migration, искам да се уверим, че **SQL Server instance-ът наистина се казва точно:**

```
DESKTOP-O6FIRML\\SQLEXPRESS02
```

В SQL Server Management Studio 22 при свързване трябва да имаш:

- **Server type:** Database Engine
- **Server name:** DESKTOP-O6FIRML\\SQLEXPRESS02
- **Authentication:** това, с което обичайно се свързваш

Ако вече си се свързал успешно с този Server Name в SSMS — супер. 

Следващата малка стъпка

Върни се във Visual Studio → **Package Manager Console**.

Провери най-долу полето:

Default project

То трябва да е:

```
StudentManagement
```

 **Все още не изпълнявай команда.**

Кажи ми какво пише при **Default project**.

93. Вие казахте:

Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any dependencies. Package Manager Console Host Version 6.14.3.1 Type 'get-help NuGet' to see all available NuGet commands.
PM>

94. ChatGPT каза:

Да, това е **нормално**. 👍 Това е стандартният текст, който Visual Studio показва при отваряне на **Package Manager Console**.
Най-важното е, че долу виждаш:

PM>

Това означава, че конзолата е готова за команда.

Стъпка 4.1 — проверяваме Default project

В горната част на **Package Manager Console** трябва да има падащо меню:

Default project:

Провери дали е избрано:

StudentManagement

Ако е избрано — **не въвеждай нищо в PM> още**.

Кажете ми само:

„StudentManagement е избран“

и тогава ще изпълним първата Migration команда. 🚀

95. Вие казахте:

избран

96. ChatGPT каза:

Супер! 👍 Всичко е настроено правилно.

Стъпка 4.2 — създаваме първата Migration

В **Package Manager Console**, където пише:

PM>

въведи:

Add-Migration InitialCreate

и натисни **Enter**.

Какво ще направи командата?

Entity Framework Core ще вземе нашия модел:

Student.cs

и ще създаде инструкции за базата данни.

Ще се появи папка:

Migrations

с файлове вътре.

Не променяй тези файлове ръчно.

🛑 **След командата**

Не изпълнявай друга команда.

Копирай ми **целия резултат от Package Manager Console**, който се появи след:

Add-Migration InitialCreate

Ако има грешка — спокойно. Ще я разгледаме стъпка по стъпка.

97. Вие казахте:

PM> Add-Migration InitialCreate Build started... Build succeeded. To undo this action, use Remove-Migration. PM>

98. ChatGPT каза:

Перфектно! 🎉 **Migration-ът е създаден успешно.**

Това:

Build succeeded.

To undo this action, use Remove-Migration.

означава, че Entity Framework Core е успял да анализира Student модела и да създаде Migration-а.

Какво имаме вече?

В Solution Explorer би трябвало да се е появила папка:

StudentManagement

```
— Controllers
— Data
  └─ ApplicationDbContext.cs
— Migrations ← нова папка
  └─ ...InitialCreate.cs
  └─ ...InitialCreate.Designer.cs
  └─ ApplicationDbContextModelSnapshot.cs
— Models
  └─ Student.cs
— Views
— wwwroot
— appsettings.json
— Program.cs
```

Не променяй нищо в Migrations.

Какво следва?

В момента Migration-ът е само **описание на промените**. Базата данни все още трябва да бъде създадена.

Следващата команда ще бъде:

Update-Database

Тя ще каже на Entity Framework:

„Изпълни Migration-а върху SQL Server и създай StudentManagementDb.“

Но **още не я изпълнявай**. 🛑

Първо отвори **SQL Server Management Studio 22** и провери дали можеш да се свържеш към:
DESKTOP-06FIRML\SQLEXPRESS02
Ако вече си свързан — кажи ми „свързан съм“ и ще направим Update-Database заедно.

99. Вие казахте:

ОК

100. ChatGPT каза:

Супер. 🙌 Нека продължим **само с една стъпка**.

Стъпка 4.3 — Update-Database

Върни се във Visual Studio → **Package Manager Console**.

Трябва да виждаш:

PM>

Въведи:

Update-Database

и натисни **Enter**.

Какво ще направи?

Тази команда ще вземе Migration-a:

InitialCreate

и ще го приложи към твоя SQL Server:

DESKTOP-06FIRML\SQLEXPRESS02

Ще се създаде:

StudentManagementDb

а вътре ще има таблица за студентите.

 Важно

След като натиснеш Enter, **не изпълнявай друга команда**.

Копирай ми целия резултат от:

PM> Update-Database

Ако е успешно — ще проверим базата в **SQL Server Management Studio 22**. Ако има грешка — ще я оправим заедно.

101. Вие казахте:

PM> Update-Database Build started... Build succeeded.

Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (242ms) [Parameters=[], CommandType='Text', CommandTimeout='60'] CREATE DATABASE [StudentManagementDb];

Microsoft.EntityFrameworkCore.Database.Command[20101] Executed DbCommand (54ms) [Parameters=[], CommandType='Text', CommandTimeout='60'] IF SERVERPROPERTY('EngineEdition') <> 5 BEGIN ALTER DATABASE [StudentManagementDb] SET

```

READ_COMMITTED_SNAPSHOT ON; END;
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (5ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] SELECT 1
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (9ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] CREATE TABLE [__EFMigrationsHistory] (
[MigrationId] nvarchar(150) NOT NULL, [ProductVersion] nvarchar(32)
NOT NULL, CONSTRAINT [PK__EFMigrationsHistory] PRIMARY KEY
([MigrationId]) );
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (0ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] SELECT 1
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (9ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] SELECT
OBJECT_ID(N'[__EFMigrationsHistory]');
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (8ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] SELECT [MigrationId], [ProductVersion]
FROM [__EFMigrationsHistory] ORDER BY [MigrationId];
Microsoft.EntityFrameworkCore.Migrations[20402] Applying migration
'20260829190300_InitialCreate'. Applying migration
'20260829190300_InitialCreate'.
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (6ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] CREATE TABLE [Students] ( [Id] int NOT NULL
IDENTITY, [FirstName] nvarchar(max) NOT NULL, [LastName]
nvarchar(max) NOT NULL, [Email] nvarchar(max) NOT NULL, [Course]
nvarchar(max) NOT NULL, [Age] int NOT NULL, CONSTRAINT
[PK_Students] PRIMARY KEY ([Id]) );
Microsoft.EntityFrameworkCore.Database.Command[20101] Executed
DbCommand (3ms) [Parameters=[], CommandType='Text',
CommandTimeout='30'] INSERT INTO [__EFMigrationsHistory]
([MigrationId], [ProductVersion]) VALUES
(N'20260829190300_InitialCreate', N'8.0.30'); Done. PM>

```

102. ChatGPT каза:



Перфектно! Базата е създадена успешно.

Точно това искахме да получим.

Какво се случи?

Най-важните редове са:

```
CREATE DATABASE [StudentManagementDb];
```

Това означава, че SQL Server е създад базата:

StudentManagementDb

След това Entity Framework е създад таблицата:

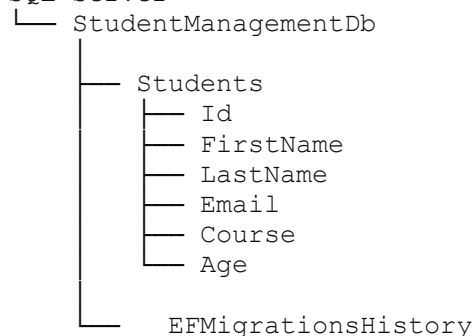
```
CREATE TABLE [Students]
```

И таблицата съдържа:

```
Id
FirstName
LastName
Email
Course
Age
```

Тоест в момента имаме:

SQL Server



__EFMigrationsHistory е вътрешна таблица на Entity Framework Core, която следи кои Migration-и вече са приложени.

А това:

Done.

означава, че всичко е приключило успешно. ✓

Следващата стъпка

Сега **няма да пишем нов код**.

Първо ще отворим **SQL Server Management Studio 22** и ще проверим с очите си дали:

```
StudentManagementDb
├── Tables
│   └── dbo.Students
```

наистина съществува.

Това е важно, защото така ще видиш връзката:

С# модел → EF Core → Migration → SQL Server → таблица

Направи само това

В SQL Server Management Studio 22:

1. Намери твоя сървър:
DESKTOP-O6FIRML\SQLEXPRESS02
2. Отвори **Databases**.
3. Потърси:
StudentManagementDb
4. Разгъни я.
5. Разгъни **Tables**.

Кажете ми какво виждаш вътре в Tables.

Очаквам да има:

`dbo.Students`

`dbo.__EFMigrationsHistory`

И после ще направим **първата истинска функционалност на системата — добавяне на студент.** 🧑

103. Вие казахте:

ОК

104. ChatGPT каза:

Супер. 👍

Тогава направи само проверката в **SQL Server Management Studio 22:**

Databases → StudentManagementDb → Tables

Трябва да виждаш:

- `dbo.Students`
- `dbo.__EFMigrationsHistory`

Ако ги виждаш, напиши „**виждам ги**“.

После започваме с първата функционалност: **страница за добавяне и управление на студенти.** 🧑

105. Вие казахте:

виждам ги

106. ChatGPT каза:

Супер! 🎉 Значи базата работи и връзката е успешна.

Сега започваме **първата реална функционалност** на нашия Student Management System:

Стъпка 5 — Students

Искаме потребителят да може да:

- вижда всички студенти;
- добавя студент;
- редактира студент;
- изтрива студент;
- разглежда подробности за студент.

За това ще използваме **MVC Controller + Views.**

Стъпка 5.1 — създаваме `StudentsController` В **Solution Explorer**:

6. Намери папката:

`Controllers`

2. Натисни **десен бутон** върху нея.
3. Избери:

Add → Controller...

4. Ще се появи прозорец **Add New Scaffolded Item**.
5. Избери:

MVC Controller - Empty

6. Натисни **Add**.
7. За име въведи:

`StudentsController`

8. Натисни **Add**.

Трябва да получиш:

```
Controllers
├── HomeController.cs
└── StudentsController.cs
```

Какво представлява Controller?

В MVC имаме:

```
Model
  ↓
Controller
  ↓
View
```

При нас:

```
Student.cs
  ↓
StudentsController.cs
  ↓
HTML страниците
```

`StudentsController` ще отговаря за действията със студентите.

Например по-късно ще имаме:

```
Index    → списък със студенти
Create   → добавяне
Edit     → редактиране
Details  → подробности
Delete   → изтриване
```

Засега спираме

Не пишем код още.

Създай `StudentsController` и ми изпрати **какво има вътре в него**.

Така ще работим върху твоя реален код и ще добавяме нещата едно по едно.

```
using Microsoft.AspNetCore.Mvc; namespace
StudentManagement.Controllers; public class StudentsController :
Controller { public IActionResult Index() { return View(); } }
```

108. ChatGPT каза:

Да, перфектно. 📖 Това е стандартният празен MVC controller. И понеже искаме да го разбереш, няма да добавяме всичко наведнъж.

Стъпка 5.2 — свързваме Controller-а с базата

В момента StudentsController знае само как да показва Index:

```
public IActionResult Index()
{
    return View();
}
```

Но той **още не знае за** ApplicationDbContext.

Ще го свържем с него.

1. Добавяме using

Най-отгоре добави:

```
using StudentManagement.Data;
```

Получаваш:

```
using Microsoft.AspNetCore.Mvc;
using StudentManagement.Data;
```

```
namespace StudentManagement.Controllers;
```

```
public class StudentsController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

2. Добавяме ApplicationDbContext

Сега вътре в класа, **преди** Index(), добави:

```
private readonly ApplicationDbContext _context;
```

След това добавяме конструктор:

```
public StudentsController(ApplicationDbContext context)
{
    _context = context;
}
```

Целият файл вече трябва да изглежда така:

```
using Microsoft.AspNetCore.Mvc;
using StudentManagement.Data;

namespace StudentManagement.Controllers;

public class StudentsController : Controller
{
    private readonly ApplicationDbContext _context;

    public StudentsController(ApplicationDbContext context)
    {
        _context = context;
    }
}
```

```

        public IActionResult Index()
        {
            return View();
        }
    }

```

Какво направихме?

Това:

```
private readonly ApplicationDbContext _context;
```

създава променлива `_context`, чрез която Controller-ът ще разговаря с базата.

А това:

```

public StudentsController(ApplicationDbContext context)
{
    _context = context;
}

```

казва:

„Когато ASP.NET създаде `StudentsController`, дай ми `ApplicationDbContext`.“

ASP.NET Core знае как да го направи, защото по-рано

в `Program.cs` написахме:

```
builder.Services.AddDbContext<ApplicationDbContext>(...)
```

Това се нарича **Dependency Injection**.



Спираме тук

Не добавяй още `Create`, `Edit`, `Delete` и т.н.

Постави кода, запази с **Ctrl + S** и направи:

Ctrl + Shift + B

После ми изпрати резултата от Build.

109. Вие казахте:

```

Build started at 10:24 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 10:24 PM and took 04.105 seconds =====

```

110. ChatGPT каза:

Перфектно! 

```
Build: 1 succeeded, 0 failed
```

Това означава, че `StudentsController` вече е успешно свързан с `ApplicationDbContext` и проектът се компилира без грешки.

Стъпка 5.3 — взимаме студентите от базата

Сега ще направим **само едно нещо**: `Index()` да прочете студентите от базата.

В момента имаш:

```
public IActionResult Index()
```

```
{  
    return View();  
}
```

Замени **само този метод** с:

```
public async Task<IActionResult> Index()  
{  
    var students = await _context.Students.ToListAsync();  
  
    return View(students);  
}
```

Но ще ни трябва още един using.

Най-отгоре на файла добави:

```
using Microsoft.EntityFrameworkCore;
```

Така целият StudentsController.cs ще бъде:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using StudentManagement.Data;  
  
namespace StudentManagement.Controllers;  
  
public class StudentsController : Controller  
{  
    private readonly ApplicationDbContext _context;  
  
    public StudentsController(ApplicationDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task<IActionResult> Index()  
    {  
        var students = await _context.Students.ToListAsync();  
  
        return View(students);  
    }  
}
```

Какво прави новият код?

```
public async Task<IActionResult> Index()
```

async означава, че методът може да изчаква операция, без да блокира приложението.

```
var students = await _context.Students.ToListAsync();
```

Това е най-важният ред.

Казваме:

„Отиди в таблицата Students, вземи всички записи и ги превърни в списък.“

Получаваме:

```
Database  
↓  
Students  
↓  
ToListAsync()  
↓  
List<Student>
```

А:

```
return View(students);
```

подава получения списък към Index.cshtml.

Но има един проблем

Все още **нямаме View**, което да може да покаже този списък.

Това е следващата ни стъпка.

🛑 Засега:

1. Постави кода.
2. Ctrl + S
3. Ctrl + Shift + B
4. Изпрати ми резултата от Build.

Не създавай View още. Ще го направим заедно на следващата стъпка.

111. Вие казахте:

```
Build started at 10:27 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 10:27 PM and took 01.164 seconds =====
```

112. ChatGPT каза:

Перфектно! 🎉 Отново:

```
Build: 1 succeeded, 0 failed
```

Значи кодът е правилен и вече можем да продължим.

Стъпка 5.4 — създаваме View за студентите

В StudentsController имаме:

```
return View(students);
```

Това означава, че ASP.NET ще търси View на следния адрес:

```
Views/Students/Index.cshtml
```

В момента тази папка вероятно **не съществува**.

1. Създаваме папка Students

В **Solution Explorer**:

1. Намери папката:

```
Views
```

2. Десен бутон върху Views.
3. Избери **Add** → **New Folder**.
4. Кръсти я:

```
Students
```

Трябва да получиш:

Views

├ Home
├ Shared
└ Students

2. Създаваме `Index.cshtml`

Сега:

2. Десен бутон върху новата папка `Students`.
3. **Add → New Item...**
4. Потърси **Razor View**.
5. Име:

`Index.cshtml`

5. Натисни **Add**.

Ако Visual Studio ти предложи шаблон, можеш да избереш **Empty**.

🛑 **Спираме тук**

Не пишем HTML още.

Искам първо да сме сигурни, че имаш:

Views

├ Students
└ `Index.cshtml`

Когато го създадеш, кажи ми „**готово**“.

След това ще направим първата ни страница, която ще показва студентите от SQL Server. 🧑

113. Вие казахте:

имам Razor Component

114. ChatGPT каза:

Да — **спри там**, не избирай Razor Component. 👍

За нашия проект използваме **MVC с Razor Views**,

защото `StudentsController` връща:

```
return View(students);
```

Razor Component е различно нещо.

Стъпка 5.4 — правилният начин

Понеже Visual Studio при теб показва Razor Component, ще направим файла ръчно. Това е напълно нормално.

1. В папката `Views/Students`

Направи:

десен бутон върху `Students` → **Add → New Item...**

2. Потърси

В полето за търсене напиши:

Razor View

Ако **няма** Razor View, не избирай Razor Component.

3. Ако пак не го намира

Тогава ще създадем файла ръчно:

десен бутон върху Students → **Add** → **New Item...**

Потърси:

Text File

и го кръсти точно:

Index.cshtml

Трябва да получим:

Views

└─ Students

 └─ Index.cshtml

Защо .cshtml?

.cshtml е Razor View файлът, който ASP.NET Core MVC използва за генериране на HTML страница.

Нашият:

StudentsController

има:

Index()

и когато посетим:

/Students

MVC ще търси:

Views/Students/Index.cshtml

 **Засега не добавяй код в** Index.cshtml.

Първо само създай файла Index.cshtml.

Ако **не намираш нито** Razor View, **нито** Text File, кажи ми какви опции ти показва **Add** → **New Item...**, и ще те насоча точно според твоя Visual Studio.

115. Вие казахте:

има Text Editor > Razor(Asp.net core)>general; scroll bars и tabs

116. ChatGPT каза:

Да — това, което виждаш:

Text Editor → **Razor (ASP.NET Core)** → **General / Scroll Bars / Tabs**
е **настройка на редактора**, а не шаблон за създаване на файл.

Няма проблем. 👍

Нека го направим по най-сигурния начин.

Стъпка 5.4 — създаваме Index.cshtml ръчно

1. Намери папката Students

В **Solution Explorer** трябва да имаш:

Views

└─ Home

└─ Shared

└─ Students

2. Десен бутон върху Students

Избери:

Add → New Item...

3. В прозореца Add New Item

Горе вдясно има поле за търсене.

Напиши:

Text File

Ако намериш **Text File**, избери го.

Името на файла трябва да бъде:

Index.cshtml

След това **Add**.

4. Провери

В Solution Explorer трябва да получиш:

Views

└─ Students

└─ Index.cshtml

⚠ Ако Visual Studio създаде:

Index.cshtml.txt

това **не е правилно**. Кажи ми, ако стане така, и ще го поправим.

🛑 **Засега не пиши код**

Само създай:

Index.cshtml

Когато го виждаш в Views → Students, напиши „**имам Index.cshtml**“.

После ще сложим **първия Razor код** и ще ти обясня как работи.

117. Вие казахте:

още няколко неща

118. ChatGPT каза:

Разбира се. 🙌 Кажи ми всички уточнения наведнъж.

Ще ги спазвам оттук нататък, за да не се налага да променяме начина, по който работим.

119. Вие казахте:

asp.net core web app(razor pages); razor class library; use the new razor formating engine in the ide (require restart); formating(Text Editor → Razor (ASP.NET Core) →Advanced)

120. ChatGPT каза:

Да, разбрах. 🙌 Това са **настройки/шаблони на Visual Studio**, които виждаш, и е важно да ги разграничим от самото създаване на MVC View.

Нашият проект е **ASP.NET Core MVC**, а не Razor Pages. Затова:

- **✗ ASP.NET Core Web App (Razor Pages)** — не го избираме.
- **✗ Razor Class Library** — не ни трябва.
- **✗ Razor Component** — не го избираме.
- **⚙ Use the new Razor formatting engine in the IDE** — това е настройка на Visual Studio и няма отношение към създаването на Index.cshtml.
- **⚙ Text Editor → Razor (ASP.NET Core) → Advanced** — това са настройки за форматиране/редактиране на Razor файловете, не типът на View.

Защо е важно?

Нашата структура е MVC:

Controller



View



HTML

Имаме:

Controllers

└─ StudentsController.cs

и ТОЙ ИЗПОЛЗВА:

```
return View(students);
```

Затова View-то трябва да бъде:

Views

└─ Students

└─ Index.cshtml

а не:

Pages

└─ Students

което би било характерно за Razor Pages.

Нека не правим нищо на сяпо

Понеже при твоя Visual Studio не виждаме стандартния шаблон **Razor View**, най-сигурно е първо да видим **точно какво ти предлага менюто**.

Направи:

десен бутон върху Views → Students → **Add** → **New Item...**

и ми изпрати **снимка на прозореца** Add New Item.

Така ще ти кажа **точно кое да натиснеш при твоята версия на Visual Studio**, вместо да те карам да избираш нещо на случаен принцип. 👍

121. Вие казахте:

стандартния шаблон Razor View как мога да си го добавя

122. ChatGPT каза:

Да. 👍 И всъщност при **ASP.NET Core MVC** е най-добре да имаш шаблона **Razor View**, за да можем удобно да създаваме `Index.cshtml`, `Create.cshtml`, `Edit.cshtml` и т.н.

При теб вероятно липсва компонентът за **ASP.NET/Web development scaffolding** във Visual Studio.

Стъпка 1 — отваряме Visual Studio Installer

Затвори **Visual Studio** или го остави отворен.

След това:

1. Отвори **Visual Studio Installer**.
2. Намери своята инсталация на Visual Studio.
3. Натисни **Modify**.

Стъпка 2 — проверяваме Workloads

В таба **Workloads** потърси:

ASP.NET and web development

Трябва да е отметнато:

☒ ASP.NET and web development

Това е основният workload, който ни трябва за ASP.NET Core MVC.

Ако не е отметнато:

4. Отметни го.
5. Натисни **Modify / Install**.
6. Изчакай инсталацията да приключи.

Стъпка 3 — проверяваме Individual components

АКО ASP.NET and web development вече е инсталиран, **не инсталирай нищо на сяпо**.

Отиди на:

Individual components

и потърси:

scaffolding

или:

ASP.NET

Ако видиш компоненти, свързани с **ASP.NET Core scaffolding**, ще ги проверим.

Важно

Не ти препоръчвам да променяш настройки като:

Use the new Razor formatting engine in the IDE

Това **няма да добави Razor View шаблона**. То е само за начина, по който Visual Studio форматира Razor кода.

Но има и по-лесен вариант

Дори ако шаблонът **Razor View** не се появи, можем спокойно да създадем:

`Views/Students/Index.cshtml`

като обикновен файл. **Не е задължително да имаш шаблона, за да работи MVC View.**

Аз лично бих предпочел първо да проверим дали **ASP.NET and web development** е инсталиран, преди да променяме каквото и да е.

🔗 Отвори **Visual Studio Installer** → **Modify** → **Workloads** и ми кажи дали **ASP.NET and web development** е отменено.

123. Вие казахте:

има отметка

124. ChatGPT каза:

Да, тогава **не пипаме Workload-а**. 👍 Той вече е инсталиран. Има още една важна подробност: за MVC scaffolding в Visual Studio понякога липсват отделни компоненти, въпреки че **ASP.NET and web development** е отменено.

Стъпка 2 — проверяваме Individual components

В **Visual Studio Installer** → **Modify**:

1. Натисни **Individual components**.
2. В полето за търсене напиши:

`scaffolding`

3. Кажи ми **какви резултати ти показва**.

🛑 Засега **не слагай нови отметки**.

Ще видим точно какво имаш и ще ти кажа кое ни трябва.

125. Вие казахте:

има катинари навсякъде

126. ChatGPT каза:

Да, това е нормално. 👍 Катарите означават, че тези компоненти са **вече инсталирани** или са управлявани от избора workload/инсталацията.

След като **ASP.NET and web development** е отменен и компонентите са заключени, **не е нужно да инсталираме нищо допълнително.**

И тук има една важна корекция от мен: **не е необходимо да имаме шаблона** Razor View, **за да продължим.** Можем да създадем .cshtml файла ръчно и той ще работи абсолютно нормално като MVC View.

Да се върнем към проекта

Затвори Visual Studio Installer и се върни в проекта.

В:

```
Views
└── Students
```

направи:

Right click → Add → New Item...

Ако не намиращ Razor View, избири **Text File** и го кръсти:

```
Index.cshtml
```

Това е напълно достатъчно.

Важно

Не избирай:

- ASP.NET Core Web App (Razor Pages)
- Razor Class Library
- Razor Component

Те са за други сценарии.

Нашият проект е:

```
ASP.NET Core MVC
```

и ще използваме:

```
StudentsController
```

↓

```
Views/Students/Index.cshtml
```

👉 Ако вече си създад Index.cshtml, кажи „**имам го**“. Ако още не си, направи го като Text File и ми кажи какво се появи в Solution Explorer.

127. Вие казахте:

имам razor page- empty; razor view - empty; razor layout; razor view start; razor view imports

128. ChatGPT каза:

А, това е точно което ни трябва! 👍

Имаш:

Razor View - Empty

Това е правилният шаблон за нашия MVC проект.

Стъпка 5.4 — създаваме Index.cshtml

В Views → Students:

1. Десен бутон върху папката **Students**.
2. **Add → New Item...**
3. Избери:

Razor View - Empty

4. Натисни **Add**.
5. За име въведи:

Index.cshtml

6. Натисни **Add**.

Трябва да получиш:

```
Views
├── Students
│   └── Index.cshtml
```

Важно

Избери точно:

Razor View - Empty

а не:

- Razor Page - Empty
- Razor Layout
- Razor View Start
- Razor View Imports

За нашия StudentsController правилният избор е **Razor View - Empty**.



Засега **не слагай код в** Index.cshtml.

Когато файлът е създаден и го виждаш под:

Views → Students → Index.cshtml

напиши „**ГОТОВО**“.

После ще направим първата View, която ще прочете и покаже студентите от базата. 🚀

129. Вие казахте:

готово и ето кода там @* For more information on enabling MVC for empty projects, visit <https://go.microsoft.com/fwlink/?LinkID=397860>

*@ @{ }

130. ChatGPT каза:

Да, това е напълно нормално. 👍 Visual Studio е създал празен **Razor View** и е добавил стандартния си шаблон. Тъй като се разбрахме **да няма обикновени коментари вътре в кода**, ще изчистим този шаблон.

Стъпка 5.5 — подготвяме Index.cshtml

Изтрий ВСИЧКО в Index.cshtml и постави само:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

Какво направихме?

Първият ред:

```
@model IEnumerable<StudentManagement.Models.Student>
```

казва на View-то:

„Тази страница ще получи списък от обекти Student.“

Това съответства на нашия Controller:

```
var students = await _context.Students.ToListAsync();
```

```
return View(students);
```

Тоест:

Database



_context.Students



ToListAsync()



List<Student>



Index.cshtml

А:

```
<h1>Students</h1>
```

засега просто ще покаже заглавието **Students**.

Важно

Няма да правим таблицата още. Искам първо да проверим, че **Controller** → **View** връзката работи правилно.

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Изпрати ми резултата.

След това ще стартираме приложението и ще отворим:

```
/Students
```

Тогава ще видим първата ни страница от Student Management System. 🚀

131. Вие казахте:

```
Build started at 11:37 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
```

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 11:37 PM and took 07.575 seconds =====

132. ChatGPT каза:

Перфектно! 

Build: 1 succeeded, 0 failed

Нямаме грешки. Вече имаме работещата връзка:

Student.cs

↓

ApplicationDbContext

↓

SQL Server

↓

StudentsController

↓

Index.cshtml

Стъпка 5.6 — стартираме приложението

Сега за първи път ще проверим страницата в браузъра.

В Visual Studio натисни зеления бутон ► за стартиране на проекта.

Може да пише нещо подобно:

https

или:

StudentManagement

Ще се отвори браузър с началната страница.

След това

В адресната лента добави:

/Students

Например ако браузърът е отворил:

https://localhost:7xxx/

отвори:

https://localhost:7xxx/Students

Какво очакваме?

Трябва да видиш:

Students

Това ще докаже, че:

/Students

↓

StudentsController

↓

Index()

↓

Index.cshtml

↓

<h1>Students</h1>

работи.

 **Засега не добавяме таблица и бутони.**

Стартирай проекта и отвори /Students.

Кажи ми **какво се показва в браузъра.**

133. Вие казахте:

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"
/> <meta name="viewport" content="width=device-width, initial-
scale=1.0" /> <title>Home Page - StudentManagement</title> <link
rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.min.css" />
<link rel="stylesheet"
href="/css/site.css?v=pAGv4ietcJNk_EwsQZ5BN9-K4MuNYS2a9wl4Jw-
q9D0" /> <link rel="stylesheet"
href="/StudentManagement.styles.css?v=J8HllgePW9NlpQzb3tFrnxtS
-y0GhZ_fh9sumvG8ORY" /> </head> <body> <header b-
y9obw7rtys> <nav b-y9obw7rtys class="navbar navbar-expand-sm
navbar-togglerable-sm navbar-light bg-white border-bottom box-
shadow mb-3"> <div b-y9obw7rtys class="container-fluid"> <a
class="navbar-brand" href="/">StudentManagement</a> <button b-
y9obw7rtys class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target=".navbar-collapse" aria-
controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation"> <span b-y9obw7rtys class="navbar-
toggler-icon"> </span> </button> <div b-y9obw7rtys class="navbar-
collapse collapse d-sm-inline-flex justify-content-between"> <ul b-
y9obw7rtys class="navbar-nav flex-grow-1"> <li b-y9obw7rtys
class="nav-item"> <a class="nav-link text-dark" href="/">Home</a>
</li> <li b-y9obw7rtys class="nav-item"> <a class="nav-link text-
dark" href="/Home/Privacy">Privacy</a> </li> </ul> </div> </div>
</nav> </header> <div b-y9obw7rtys class="container"> <main b-
y9obw7rtys role="main" class="pb-3"> <div class="text-center"> <h1
class="display-4">Welcome</h1> <p>Learn about <a
href="https://learn.microsoft.com/aspnet/core">building Web apps
with ASP.NET Core</a>.</p> </div> </main> </div> <footer b-
y9obw7rtys class="border-top footer text-muted"> <div b-
y9obw7rtys class="container"> &copy; 2026 - StudentManagement -
<a href="/Home/Privacy">Privacy</a> </div> </footer> <script
src="/lib/jquery/dist/jquery.min.js"> </script> <script
src="/lib/bootstrap/dist/js/bootstrap.bundle.min.js"> </script> <script
src="/js/site.js?v=hRQyftXiu1lX2P9Ly9xa4gHJgLeR1uGN5qegUobtGo
"> </script> <!-- Visual Studio Browser Link --> <script
type="text/javascript" src="/_vs/browserLink" async="async"
id="__browserLink_initializationData" data-
requestId="6c9234b0695a4aebbde5603f5ad8053e" data-
requestMappingFromServer="false" data-
```

```
connectUrl="http://localhost:61289/107dce9254d640adaf13a9f080ab7503/browserLink"></script> <!-- End Browser Link --> <script src="/_framework/aspnetcore-browser-refresh.js"></script> </body> </html>
```

134. ChatGPT каза:

Да, виждам какво се е случило. 📁 **Приложението работи**, но в момента си отворил **Home** страницата, а не Students.

HTML-ът, който ми изпрати, съдържа:

```
<title>Home Page - StudentManagement</title>
```

и:

```
<h1 class="display-4">Welcome</h1>
```

Това е Home/Index, не нашата Students/Index.

Стъпка 5.7 — отваряме Students

В брауъра, който се е отворил от Visual Studio, погледни адреса.

Вероятно е нещо като:

```
https://localhost:xxxx/
```

След последната / добави:

```
Students
```

Така трябва да стане например:

```
https://localhost:xxxx/Students
```

Не копирай xxxx **буквално** — остави номера на порта, който вече имаш.

Какво трябва да видиш?

Трябва да се появи:

```
Students
```

Това е заглавието от нашия:

```
Views
```

```
└─ Students
```

```
    └─ Index.cshtml
```

и доказва, че:

```
/Students
```

↓

```
StudentsController
```

↓

```
Index()
```

↓

```
Index.cshtml
```

работи. ✅

🛑 Засега **не променяме нищо по кода**.

Отвори /Students и ми кажи какво виждаш.

135. Вие казахте:

само главно меню без линк към страницата, но
<https://localhost:7295/Students> съществува

136. ChatGPT каза:

Точно така. 👍 Това всъщност е **добър знак**.

<https://localhost:7295/Students> съществува, което означава, че **маршрутът и страницата работят**. Просто в главното меню още няма линк към Students.

Стъпка 5.8 — добавяме Students в главното меню

Главното меню, което виждаш, идва от:

```
Views
├── Shared
│   └── _Layout.cshtml
```

Този файл съдържа навигацията:

Home | Privacy

Ще добавим:

Home | Students | Privacy

1. Отвори _Layout.cshtml

В Solution Explorer:

```
Views
├── Shared
│   └── _Layout.cshtml
```

Отвори файла.

2. Намери тази част

Трябва да има нещо подобно:

```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Index">Home</a>
</li>
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Privacy">Privacy</a>
</li>
```

3. Между Home и Privacy добавяме

```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Students" asp-action="Index">Students</a>
</li>
```

Получава се:

```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Index">Home</a>
</li>
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Students" asp-action="Index">Students</a>
</li>
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Privacy">Privacy</a>
</li>
```

Какво означава това?

Тук:

asp-controller="Students"

казваме:

Линкът води към StudentsController.

А:

asp-action="Index"

казва:

Използвай метода `Index()`.

Така:

`Students → StudentsController → Index()`

и браузърът отива към:

`/Students`



След това

1. Запази с **Ctrl + S**.
2. Не променяй нищо друго.
3. Върни се в браузъра и презареди страницата с **Ctrl + F5**.
4. Трябва да се появи:

Home Students Privacy

Натисни **Students**.

Ако те отведе към `https://localhost:7295/Students` и покаже **Students**, значи навигацията вече работи. ✓

137. Вие казахте:

ОК

138. ChatGPT каза:

Супер! 👍

Значи вече имаме работеща основа:

- ✓ Student модел
- ✓ ApplicationDbContext
- ✓ SQL Server връзка
- ✓ InitialCreate Migration
- ✓ StudentManagementDb
- ✓ Students таблица
- ✓ StudentsController
- ✓ Students/Index.cshtml
- ✓ линк **Students** в главното меню

Следваща стъпка — показваме студентите като таблица

В момента `/Students` вероятно показва само:

`Students`

Следващото, което ще направим, е да превърнем страницата в нещо подобно:

Id First Name Last Name Email Course Age

— — — — —

След това ще добавим бутон **Create Student**, с който ще можем да запишем първия студент в SQL Server.

Но ще вървим **една малка стъпка наведнъж**.

👉 Кажете ми, когато си на страницата:

<https://localhost:7295/Students>

и виждаш **Students**. Тогава ще направим таблицата.

139. Вие казахте:

ГОТОВО

140. ChatGPT каза:

Чудесно. 👉 Сега ще направим **само таблицата**, без добавяне, редактиране или изтриване още.

Стъпка 5.9 — показваме студентите в таблица

Отвори:

```
Views
├── Students
│   └── Index.cshtml
```

Изтрий текущото съдържание:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

и постави това:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

```
<table class="table">
  <thead>
    <tr>
      <th>First Name</th>
      <th>Last Name</th>
      <th>Email</th>
      <th>Course</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var student in Model)
    {
      <tr>
        <td>@student.FirstName</td>
        <td>@student.LastName</td>
        <td>@student.Email</td>
        <td>@student.Course</td>
        <td>@student.Age</td>
      </tr>
    }
  </tbody>
</table>
```

Какво прави кодът?

Тази част:

```
@model IEnumerable<StudentManagement.Models.Student>
```

казва, че View-то получава **списък от студенти**.

Тази част:

```
@foreach (var student in Model)
```

обикаля през всички студенти, които Controller-ът е получил от базата.

За всеки студент създаваме един `<tr>` — един ред в таблицата.

Например:

```
<td>@student.FirstName</td>
```

показва името на студента.

Важното в момента

Нашата база данни все още е празна.

Затова след зареждането на `/Students` ще видиш заглавията:

```
First Name | Last Name | Email | Course | Age
```

но **няма да има редове със студенти.**

Това е правилно. Следващата ни задача ще бъде да направим **Create Student** и да добавим първия запис.

Сега

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Ако Build е успешен, отвори `/Students` или презареди страницата.

Кажете ми какво виждаш.

141. Вие казахте:

празна таблица

142. ChatGPT каза:

Точно така! ☒ **Празната таблица е правилният резултат.**

Това означава, че всичко по веригата работи:

SQL Server



StudentManagementDb



Students



ApplicationDbContext



StudentsController



Index.cshtml



Празна таблица

Просто още **нямаме записан студент.**

Стъпка 5.10 — добавяме бутон „Add Student“

Сега ще направим само бутона. След това ще направим формата.

Отвори:

Views

└─ Students

Най-горе имаш:

```
<h1>Students</h1>
```

Веднага **под него** добави:

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

Така началото на файла ще изглежда:

```
@model IEnumerable<StudentManagement.Models.Student>
```

```
<h1>Students</h1>
```

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

```
<table class="table">
```

Какво прави този ред?

```
<a asp-action="Create" class="btn btn-primary">Add Student</a>
```

Това създава Bootstrap бутон **Add Student**.

asp-action="Create" казва:

Когато натисна бутона, отиди към Create action в текущия controller

— StudentsController.

Тоест очакваме:

Add Student

↓

StudentsController

↓

Create()

⚠ **Но** Create() **все още не съществува**.

Затова, ако натиснеш бутона сега, ще получиш грешка/404. Това е нормално и точно това ще направим в следващата стъпка.

Сега само:

1. Добави реда.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Отвори /Students.
5. Провери дали виждаш син бутон **Add Student**.

Не натискай бутона още. 😊

Кажи ми „**бутонът е там**“, когато го видиш.

143. Вие казахте:

бутонът е там

144. ChatGPT каза:

Супер! 🙌 Бутонът е там и вече имаме връзката към бъдещата форма.

Стъпка 5.11 — създаваме Create action

В момента бутонът сочи към:

/Students/Create

НО StudentsController все още няма Create() МЕТОД.

Сега ще добавим **само GET версията** на Create. Тя ще показва празната форма.

Отвори:

```
Controllers
└── StudentsController.cs
```

Под метода Index() добави:

```
public IActionResult Create()
{
    return View();
}
```

Така StudentsController трябва да изглежда:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

namespace StudentManagement.Controllers;

public class StudentsController : Controller
{
    private readonly ApplicationDbContext _context;

    public StudentsController(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        var students = await _context.Students.ToListAsync();

        return View(students);
    }

    public IActionResult Create()
    {
        return View();
    }
}
```

Какво прави Create()?

Когато натиснем:

Add Student

ASP.NET ще намери:

StudentsController

↓

Create()

↓

View()

и ще търси View:

Views

└── Students

 └── Create.cshtml

Този файл още няма да съществува.

Сега само проверяваме Controller-a

1. Добави метода.
2. **Ctrl + S**
3. **Ctrl + Shift + B**

4. Изпрати ми резултата от Build.

Не създавай Create.cshtml **още**. След успешния Build ще го направим като Razor View - Empty, както направихме с Index.cshtml.

145. Вие казахте:

```
Build started at 11:58 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 11:59 PM and took 00.865 seconds =====
```

146. ChatGPT каза:

Перфектно. ☒ Create() е добавен и проектът се компилира без грешки.

Стъпка 5.12 — създаваме Create.cshtml

Сега ще създадем View-то за формата.

В **Solution Explorer**:

1. Десен бутон върху:
Views → Students
2. Избери **Add → New Item...**
3. Избери:

Razor View - Empty

4. Име:

Create.cshtml

5. Натисни **Add**.

Трябва да имаш:

```
Views
├── Students
│   ├── Create.cshtml
│   └── Index.cshtml
```

Засега не пишем код

Само създай Create.cshtml.

Когато го видиш в Solution Explorer, кажи **„готово“**.

След това ще направим формата **поле по поле**, за да е ясно как всяко нещо работи.

147. Вие казахте:

148. ChatGPT каза:

Супер! 🙌 Сега ще направим **само основата на формата**. После ще добавим запазването в базата.

Стъпка 5.13 — моделът на формата

Отвори:

```
Views
├── Students
│   └── Create.cshtml
```

Изтрий всичко вътре и постави:

```
@model StudentManagement.Models.Student
```

```
<h1>Add Student</h1>
```

```
<form asp-action="Create" method="post">
```

```
    <div class="mb-3">
        <label asp-for="FirstName" class="form-label"></label>
        <input asp-for="FirstName" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="LastName" class="form-label"></label>
        <input asp-for="LastName" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Course" class="form-label"></label>
        <input asp-for="Course" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Age" class="form-label"></label>
        <input asp-for="Age" class="form-control" />
    </div>
```

```
    <button type="submit" class="btn btn-primary">Save</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
```

```
</form>
```

Какво направихме?

Първият ред:

```
@model StudentManagement.Models.Student
```

казва:

Тази форма работи с един обект Student.

След това:

```
<form asp-action="Create" method="post">
```

казва:

При натискане на **Save** изпрати данните към Create action чрез POST.

Например:

```
<input asp-for="FirstName" class="form-control" />
```

автоматично се свързва с:

```
Student.FirstName
```

Същото правим за:

```
FirstName
```

```
LastName
```

```
Email
```

```
Course
```

```
Age
```

Save изпраща формата, а Cancel ни връща към списъка.

⚠ Важно

В момента формата **ще се покаже**, но натискането на Save няма да запише студента, защото все още нямаме POST Create action.

Това ще бъде **следващата малка стъпка**.

Засега:

1. Запази с **Ctrl + S**.
2. Направи **Ctrl + Shift + B**.
3. Ако Build е успешен, стартирай приложението.
4. Отиди на /Students.
5. Натисни **Add Student**.

Трябва да видиш формата с:

```
Add Student
```

```
First Name
```

```
Last Name
```

```
Email
```

```
Course
```

```
Age
```

```
Save Cancel
```

Не натискай Save още.

Кажи ми как изглежда формата.

149. Вие казахте:

както при теб

150. ChatGPT каза:

Чудесно! 👍 Значи **View-то работи правилно** и формата се показва.

Сега идва важната част: ще направим така, че натискането на **Save** действително да запише студента в SQL Server.

Стъпка 5.14 — POST Create

В момента имаме:

```
Create.cshtml
```

↓

```
Save
```

↓

```
/Students/Create
```

Но Controller-ът няма метод, който да приеме изпратените данни.

Отвори:

```
Controllers  
└─ StudentsController.cs
```

Под сегашния Create() метод добави:

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> Create(Student student)  
{  
    if (!ModelState.IsValid)  
    {  
        return View(student);  
    }  
  
    _context.Students.Add(student);  
    await _context.SaveChangesAsync();  
  
    return RedirectToAction(nameof(Index));  
}
```

⚠ Ще имаме една малка промяна

За да разпознае Student, Controller-ът трябва да има:

```
using StudentManagement.Models;
```

Затова най-отгоре трябва да имаш:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using StudentManagement.Data;  
using StudentManagement.Models;
```

Целият StudentsController.cs ще изглежда така:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using StudentManagement.Data;  
using StudentManagement.Models;  
  
namespace StudentManagement.Controllers;  
  
public class StudentsController : Controller  
{  
    private readonly ApplicationDbContext _context;  
  
    public StudentsController(ApplicationDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task<IActionResult> Index()  
    {  
        var students = await _context.Students.ToListAsync();  
  
        return View(students);  
    }  
  
    public IActionResult Create()  
    {  
        return View();  
    }  
  
    [HttpPost]  
    [ValidateAntiForgeryToken]  
    public async Task<IActionResult> Create(Student student)  
    {  
        if (!ModelState.IsValid)  
        {
```

```

        return View(student);
    }

    _context.Students.Add(student);
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}

```

Какво прави POST методът?

Когато натиснеш **Save**, формата изпраща:

```

FirstName
LastName
Email
Course
Age

```

към ТОЗИ МЕТОД:

```
Create(Student student)
```

ASP.NET автоматично създава Student обект от въведените данни.

После:

```
_context.Students.Add(student);
```

казва на Entity Framework:

„Искам този студент да бъде добавен към таблицата Students.“

След това:

```
await _context.SaveChangesAsync();
```

е моментът, в който промяната реално се записва в SQL Server.

Накрая:

```
return RedirectToAction(nameof(Index));
```

връща потребителя обратно към списъка със студенти.

А какво прави [ValidateAntiForgeryToken]?

Това е стандартна защита при POST форми в ASP.NET Core MVC.

Тя предпазва формата от определен тип атака, наречена **Cross-Site Request Forgery (CSRF)**.

Razor формата ни автоматично ще добави необходимия anti-forgery token.



Сега само това

1. Добави using StudentManagement.Models;
2. Добави POST Create.
3. **Ctrl + S**
4. **Ctrl + Shift + B**
5. Изпрати ми резултата от Build.

Още не натискай Save във формата. Първо проверяваме, че Controller-ът се компилира.

Build started at 12:10 AM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 12:10 AM and took 01.032 seconds =====

152. ChatGPT каза:

Перфектно! 

Build: 1 succeeded, 0 failed

Значи POST Create е добавен правилно.

Сега идва **първият ни истински тест** — ще запишем реален студент в SQL Server.

Стъпка 5.15 — добавяме първия студент

Стартирай приложението и отвори:

<https://localhost:7295/Students>

Натисни:

Add Student

Във формата въведи например:

First Name: Иван

Last Name: Петров

Email: ivan.petrov@example.com

Course: Computer Science

Age: 20

След това натисни:

Save

Какво трябва да се случи?

След натискането на Save:

Create.cshtml

↓

POST Create(Student student)

↓

_context.Students.Add(student)

↓

SaveChangesAsync()

↓

SQL Server

↓

Students таблица

↓

/Students

И трябва да се върнеш автоматично на страницата **Students**.

Този път таблицата вече трябва да съдържа:

First Name	Last Name	Email	Course	Age
Иван	Петров	ivan.petrov@example.com	Computer Science	20

Направи само този тест

Добави **един** студент и ми кажи какво се случи след натискането на **Save**.

Ако се появи грешка, **не променяй нищо** — изпрати ми текста на грешката и ще я оправим заедно.

153. Вие казахте:

ОК

154. ChatGPT каза:

Супер. 👍

Направи теста:

1. **Students**
2. **Add Student**
3. Попълни петте полета.
4. Натисни **Save**.
5. Провери дали студентът се появява в таблицата.

После ми кажи само едно от двете:

- „добави се” ✓
- или „има грешка” ✗ — и ми изпрати грешката.

Засега не правим други промени.

155. Вие казахте:

добави се

156. ChatGPT каза:

Страхотно! 🎉 Това е важен момент — **първият студент вече се записва реално в SQL Server.**

Вече сме доказали, че целият поток работи:

Форма

↓

StudentsController

↓

Entity Framework Core

↓

SQL Server

↓

Students таблица

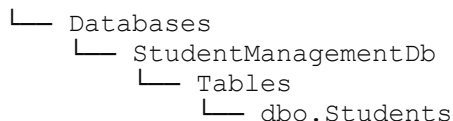
Стъпка 5.16 — проверяваме базата

Преди да добавяме нова функционалност, нека видим дали студентът наистина е в SQL Server.

Отвори **SQL Server Management Studio (SSMS)**.

В Object Explorer намери:

DESKTOP-06FIRML\SQLEXPRESS02



Ако dbo.Students е там:

1. Десен бутон върху dbo.Students.
2. Избери **Select Top 1000 Rows**.

Трябва да видиш записания студент.

Защо правим тази проверка?

Защото вече знаем, че страницата го показва, но сега ще потвърдим директно в SQL Server:

StudentManagementDb



dbo.Students



Иван Петров

Това е полезно и за теб като начинаещ — ще виждаш връзката между **C# кода и реалната база данни**.



Направи само тази проверка.

Кажи ми дали виждаш студента в dbo.Students.

157. Вие казахте:

записа го има

158. ChatGPT каза:

Перфектно! 🎉 Значи всичко работи както трябва.

Вече имаме **реално работещ CRUD проект**, но засега сме направили само **Create** и **Read**.

Какво имаме досега

- ☒ Student модел
- ☒ Entity Framework Core 8.0.30
- ☒ SQL Server Express
- ☒ ApplicationDbContext
- ☒ Connection String
- ☒ Migration InitialCreate
- ☒ База StudentManagementDb
- ☒ Таблица Students
- ☒ StudentsController
- ☒ Списък със студенти
- ☒ Добавяне на студент
- ☒ Записване в SQL Server
- ☒ Проверка директно в SSMS

Следваща стъпка — Edit

Сега ще направим възможност да **редактираме съществуващ студент**.

Например:

Иван Петров
ivan.petrov@example.com
Computer Science
20

да можем да променим на:

Иван Петров
ivan.petrov@example.com
Software Engineering
21

Ще направим това отново **стъпка по стъпка**, без да прескачаме.

Стъпка 5.17

Първо ще добавим в таблицата един бутон:

Edit

който ще води към:

/Students/Edit/1

Но **още няма да правим Edit Controller-а и формата**.

Ако си готов, следващата ни малка стъпка е да добавим само **Edit** **линк** към всеки студент.

159. **Вие казахте:**

да

160. **ChatGPT каза:**

Чудесно. 👍 Правим **само Edit бутона** към всеки студент.

Стъпка 5.17 — добавяме Edit

Отвори:

```
Views
├── Students
│   └── Index.cshtml
```

В момента в таблицата имаш нещо подобно:

```
@foreach (var student in Model)
{
    <tr>
        <td>@student.FirstName</td>
        <td>@student.LastName</td>
        <td>@student.Email</td>
        <td>@student.Course</td>
        <td>@student.Age</td>
    </tr>
}
```

1. Добавяме нова колона

В `<thead>` след Age добави:

```
<th>Actions</th>
```

Получаваш:

```
<thead>
    <tr>
```

```

        <th>First Name</th>
        <th>Last Name</th>
        <th>Email</th>
        <th>Course</th>
        <th>Age</th>
        <th>Actions</th>
    </tr>
</thead>

```

2. Добавяме Edit към всеки ред

В <tr> след:

```
<td>@student.Age</td>
```

добави:

```

<td>
    <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-
warning">Edit</a>
</td>

```

Така целият foreach ще бъде:

```

@foreach (var student in Model)
{
    <tr>
        <td>@student.FirstName</td>
        <td>@student.LastName</td>
        <td>@student.Email</td>
        <td>@student.Course</td>
        <td>@student.Age</td>
        <td>
            <a asp-action="Edit" asp-route-id="@student.Id"
class="btn btn-warning">Edit</a>
        </td>
    </tr>
}

```

Какво означава това?

Тази част:

```
asp-action="Edit"
```

казва:

Отиди към Edit action.

А:

```
asp-route-id="@student.Id"
```

подава **Id** на конкретния студент.

Например ако Иван има:

```
Id = 1
```

бутонът ще води към:

```
/Students/Edit/1
```

Ако друг студент има Id = 2:

```
/Students/Edit/2
```

Това е важно, защото по този начин знаем **кой точно студент** искаме да редактираме.

🔒 **Сега само това**

1. Добави колоната Actions.
2. Добави Edit бутона.
3. Ctrl + S
4. Ctrl + Shift + B
5. Отвори /Students.

Не натискай Edit още, защото Edit() в Controller-а все още не съществува.

Изпрати ми Build резултата и ще продължим.

161. Вие казахте:

```
Build started at 12:24 AM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 12:24 AM and took 07.311 seconds =====
```

162. ChatGPT каза:

Перфектно! ☒ Build-ът е успешен.

Сега трябва да имаш таблица приблизително така:

First Name Last Name Email Course Age Actions

Иван Петров 20 **Edit**

Има само едно нещо, което **няма да работи още**: натискането на **Edit**. Това е очаквано, защото още нямаме Edit() в Controller-а.

Стъпка 5.18 — създаваме GET Edit

Отвори:

```
Controllers
└── StudentsController.cs
```

Под Create методите добави:

```
public async Task<IActionResult> Edit(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}
```

Какво прави този метод?

Когато натиснем например:

```
/Students/Edit/1
```

стойността:

```
id = 1
```

се подава на метода.

След това:

```
var student = await _context.Students.FindAsync(id);
```

търси студента с това id в базата.

Ако не е намерен:

```
return NotFound();
```

връщаме **404 Not Found**.

Ако е намерен:

```
return View(student);
```

подаваме конкретния студент към бъдещото:

```
Views/Students/Edit.cshtml
```

 **Засега само това**

Добави метода, запази и направи:

Ctrl + Shift + B

Не създавай Edit.cshtml още и не натискай Edit бутона.

Изпрати ми Build резултата.

163. Вие казахте:

Build started at 12:26 AM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0

failed, 0 up-to-date, 0 skipped ===== Build

completed at 12:26 AM and took 01.009 seconds =====

164. ChatGPT каза:

Перфектно! ☒ Отново:

Build: 1 succeeded, 0 failed

GET Edit вече е ГОТОВ.

Стъпка 5.19 — създаваме Edit.cshtml

Сега ще създадем View-то, което ще покаже данните на избрания студент.

В **Solution Explorer**:

1. Десен бутон върху:

Views → Students

2. **Add → New Item...**

3. Избери:

Razor View - Empty

4. Име:

Edit.cshtml

5. Натисни **Add**.

Трябва да имаш:

```
Views
├── Students
│   ├── Create.cshtml
│   ├── Edit.cshtml
│   └── Index.cshtml
```

🛑 Засега не добавяй код

Само създай Edit.cshtml.

Когато го видиш в Views → Students, кажи „**ГОТОВО**“ и ще направим формата за редактиране, отново стъпка по стъпка.

165. Вие казахте:

ГОТОВО

166. ChatGPT каза:

Чудесно. 👍 Сега ще направим формата за редактиране.

Стъпка 5.20 — Edit.cshtml

Отвори:

Views → Students → Edit.cshtml

Изтрий всичко вътре и постави:

```
@model StudentManagement.Models.Student
```

```
<h1>Edit Student</h1>
```

```
<form asp-action="Edit" method="post">
```

```
    <input type="hidden" asp-for="Id" />
```

```
    <div class="mb-3">
        <label asp-for="FirstName" class="form-label"></label>
        <input asp-for="FirstName" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="LastName" class="form-label"></label>
        <input asp-for="LastName" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Course" class="form-label"></label>
        <input asp-for="Course" class="form-control" />
    </div>
```

```
    <div class="mb-3">
        <label asp-for="Age" class="form-label"></label>
        <input asp-for="Age" class="form-control" />
    </div>
```

```
    <button type="submit" class="btn btn-primary">Save</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
```

```
</form>
```

Какво е новото тук?

Най-важният ред, който не сме имали в Create, е:

```
<input type="hidden" asp-for="Id" />
```

Той запазва Id на студента във формата.

Например:

```
Id = 1
```

Когато редактираме Иван, формата ще знае:

Редактираме студент №1.

Останалите полета се попълват автоматично от:

```
return View(student);
```

който направихме в Edit(int? id).

Така потокът вече е:

```
Students
```

↓

```
Edit
```

↓

```
Edit/1
```

↓

```
FindAsync(1)
```

↓

```
Edit.cshtml
```

↓

Показва данните на Иван



Засега спираме тук

Не добавяме POST Edit още.

Направи:

1. **Ctrl + S**
2. **Ctrl + Shift + B**
3. Изпрати ми Build резултата.

След това ще проверим дали при натискане на **Edit** се отваря формата с вече попълнените данни.

167. Вие казахте:

Build started at 12:34 AM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0

failed, 0 up-to-date, 0 skipped ===== Build

completed at 12:34 AM and took 00.826 seconds =====

168. ChatGPT каза:

Перфектно!

Build-ът е успешен.

Стъпка 5.21 — тестваме Edit формата

Сега стартирай приложението и:

1. Отвори **Students** от главното меню.
2. Трябва да видиш записания от нас студент.
3. Натисни **Edit** срещу него.

Трябва да се отвори:

Edit Student

First Name	Иван
Last Name	Петров
Email	...
Course	...
Age	20

Save Cancel

Важно

Не натискай Save още.

На този етап Edit само **зарежда данните**. Все още не сме написали POST метода, който да запише промените.

Провери само дали формата се отваря и дали полетата са попълнени с данните на студента.

Ако всичко е наред, кажи ми „**работи**“ и ще направим следващата малка стъпка — записването на промените в SQL Server.

169. Вие казахте:

работи

170. ChatGPT каза:

Супер! 🎉 Това означава, че **GET Edit** работи правилно — намираме конкретния студент по id и зареждаме данните му във формата.

Стъпка 5.22 — записваме промените

Сега ще добавим **POST Edit** в StudentsController.

Отвори:

```
Controllers
└─ StudentsController.cs
```

Под съществуващия Edit(int? id) метод добави:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Student student)
{
    if (id != student.Id)
    {
        return NotFound();
    }

    if (!ModelState.IsValid)
    {
        return View(student);
    }
}
```

```

        _context.Update(student);
        await _context.SaveChangesAsync();

        return RedirectToAction(nameof(Index));
    }

```

Какво прави?

Когато натиснем **Save** във формата:

Edit.cshtml

↓
POST

↓
Edit(int id, Student student)

Първо проверяваме:

```
if (id != student.Id)
```

дали id от URL адреса съвпада с id на изпратения студент.

След това:

```
_context.Update(student);
```

казваме на Entity Framework:

Този студент вече съществува. Искам да актуализирам данните му.

A:

```
await _context.SaveChangesAsync();
```

записва промяната реално в SQL Server.

Накрая:

```
return RedirectToAction(nameof(Index));
```

връщаме потребителя обратно към списъка.



Сега само това

1. Добави POST метода.
2. Запази с **Ctrl + S**.
3. Направи **Ctrl + Shift + B**.
4. Изпрати ми Build резултата.

Още **не променяй данните**. Първо ще проверим, че кодът се компилира.

171. Вие казахте:

може ли да скрием <https://localhost:7295/Students/Edit/1> -> да скрием от лентата и да пише например името и фамилията за по-голяма сигурност

172. ChatGPT каза:

Да, можем — и е добра идея да го направим, но има едно важно уточнение:

скриването на id в адресната лента не е реална защита само по себе си. Например </Students/Edit/1> не разкрива чувствителна

информация, но можем да направим URL-а по-удобен и да не показваме числовия id.

За начинаещ бих ти предложил първо да направим URL от типа:
`/Students/Edit/Ivan-Petrov`

или:

`/Students/Edit/Ivan/Petrov`

Така вместо:

`https://localhost:7295/Students/Edit/1`

ще виждаш например:

`https://localhost:7295/Students/Edit/Ivan/Petrov`

Но има още по-добър вариант

Ако целта е **сигурност**, можем да оставим id вътрешно в приложението, но да не го показваме в URL-а. Например:

`/Students/Edit/Ivan-Petrov`

Controller-ът ще намери студента по името, а id ще си остане само вътрешно.

Само че има един проблем: двама студенти могат да се казват „Иван Петров“. Затова име + фамилия не е уникален идентификатор.

По-правилният професионален вариант е да използваме **GUID/случаен идентификатор** в URL-а или да използваме route, който не показва вътрешния database id.

Но понеже правим проекта **стъпка по стъпка**, предлагам да не променяме това точно сега, докато не завършим базовия CRUD. След като направим:

- ☒ Create
- ☒ Read
- ☒ Edit
- ☐ Delete
- ☐ Details

ще направим отделна малка стъпка за **по-сигурни и чисти URL адреси** и ще обсъдим защо избраният вариант е безопасен.

Така няма да счупим работещия Edit в момента.

173. Вие казахте:

за сега няма да го променяме

174. ChatGPT каза:

Разбрах. 👍 Засега **не променяме URL адресите**.

Продължаваме по плана и първо завършваме основния CRUD.

В момента сме на **Edit**. Следва:

1. Проверяваме POST Edit с реална промяна.
2. След това правим **Delete**.
3. Накрая ще имаме основния CRUD напълно работещ.

Сега

Имаш вече добавения:

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> Edit(int id, Student student)
```

Направи **Ctrl + Shift + B** и ми изпрати резултата, ако още не си го направил след добавянето му.

После ще променим например курса или възрастта на Иван и ще проверим дали промяната влиза и в SQL Server.

175. Вие казахте:

```
Build started at 12:45 AM... 1>----- Build started: Project:  
StudentManagement, Configuration: Debug Any CPU -----  
1>StudentManagement ->  
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net  
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0  
failed, 0 up-to-date, 0 skipped ===== Build  
completed at 12:45 AM and took 07.334 seconds =====
```

176. ChatGPT каза:

Перфектно! ☒ POST Edit се компилира без грешки.

Стъпка 5.23 — тестваме редактирането

Сега ще проверим целия процес с реална промяна.

1. Стартирай приложението.
2. Отвори **Students**.
3. Натисни **Edit** на Иван Петров.
4. Промени само **Course**, например от:

```
Computer Science  
на:  
Software Engineering
```

5. Натисни **Save**.

Какво трябва да се случи?

След Save трябва автоматично да се върнеш на:

```
/Students
```

и таблицата трябва да показва:

First Name	Last Name	Course
Иван	Петров	Software Engineering

След това можем да направим още една важна проверка в **SQL Server Management Studio** — дали `dbo.Students` също съдържа променения курс.

🔍 Направи само тази промяна и ми кажи:
„**промени се**“ или „**има грешка**“.

177. Вие казахте:

промени се и в `dbo.Students` също

178. ChatGPT каза:

Отлично! 🎉 Това е точно резултатът, който искахме.

Вече имаме напълно работещи **Create + Read + Update** операции:

Create → добавя студент
Read → показва студентите
Update → променя студент

И най-важното — проверихме, че промяната стига **реално до SQL Server**.

Стъпка 5.24 — Delete

Следва последната основна CRUD операция:

Delete → изтрива студент

Ще го направим внимателно, защото не искам случайно да изтрием студента, който използваме за тестове.

Първо ще добавим само **Delete бутон** към таблицата.

Отвори:

Views → Students → Index.cshtml

В колоната Actions, където в момента имаш:

```
<td>
  <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
</td>
```

замени я с:

```
<td>
  <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit</a>
  <a asp-action="Delete" asp-route-id="@student.Id" class="btn btn-danger">Delete</a>
</td>
```

Така всеки студент ще има два бутона:

Edit Delete

Какво прави Delete линкът?

Например за студент с `Id = 1`:

`/Students/Delete/1`

Точно както направихме при Edit.

⚠ **Все още няма да работи**, защото още нямаме Delete() в Controller-а. Това е нарочно — ще го изградим на отделни стъпки.
Сега

1. Промени само Index.cshtml.
2. Запази.
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

После ще направим Delete метода.

179. **Вие казахте:**

```
Build started at 12:52 AM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 12:52 AM and took 01.010 seconds =====
```

180. **ChatGPT каза:**

Перфектно! ☒ Build-ът е успешен.
Сега имаме бутон **Delete**, но още няма да го натискаме.

Стъпка 5.25 — GET Delete

Първо ще направим страница за потвърждение:
„Сигурни ли сте, че искате да изтриете този студент?“
Това е по-добре от директно изтриване при натискане на линка.

1. Отвори

Controllers
└─ StudentsController.cs

Под Edit методите добави:

```
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var student = await _context.Students.FirstOrDefaultAsync(m =>
m.Id == id);

    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}
```

Какво прави?

При натискане на:

`/Students/Delete/1`

методът получава:

`id = 1`

След това:

`_context.Students.FirstOrDefaultAsync(m => m.Id == id)`

търси конкретния студент в базата.

Ако го намери:

`return View(student);`

ще отвори:

`Views/Students/Delete.cshtml`

Там ще направим страницата за потвърждение.

Ако студентът не съществува, получаваме:

`404 Not Found`



Само това засега

1. Добави GET Delete.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

Не създавай Delete.cshtml още. Ще го направим след успешния Build.

181. Вие казахте:

Build started at 12:56 AM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0

failed, 0 up-to-date, 0 skipped ===== Build

completed at 12:56 AM and took 00.910 seconds =====

182. ChatGPT каза:

Перфектно! ☒ GET Delete е готов и Build-ът е успешен.

Стъпка 5.26 — създаваме Delete.cshtml

Сега ще направим страницата за потвърждение.

В **Solution Explorer**:

1. Десен бутон върху:
Views → Students
2. **Add → New Item...**
3. Избери:

Razor View - Empty

4. Име:

Delete.cshtml

5. Натисни **Add**.

Трябва да имаш:

```
Views
├── Students
│   ├── Create.cshtml
│   ├── Delete.cshtml
│   ├── Edit.cshtml
│   └── Index.cshtml
```

Засега не пишем код

Само създай Delete.cshtml.

Когато го виждаш в Views → Students, напиши „**ГОТОВО**“. После ще направим страницата за потвърждение.

183. **Вие казахте:**

ГОТОВО

184. **ChatGPT каза:**

Чудесно. 🙌 Сега ще направим **само страницата за потвърждение**. Все още няма да изтриваме нищо.

Стъпка 5.27 — Delete.cshtml

Отвори:

Views → Students → Delete.cshtml

Изтрий всичко вътре и постави:

```
@model StudentManagement.Models.Student
```

```
<h1>Delete Student</h1>
```

```
<h3>Are you sure you want to delete this student?</h3>
```

```
<div>
```

```
<dl class="row">
```

```
<dt class="col-sm-2">First Name</dt>
```

```
<dd class="col-sm-10">@Model.FirstName</dd>
```

```
<dt class="col-sm-2">Last Name</dt>
```

```
<dd class="col-sm-10">@Model.LastName</dd>
```

```
<dt class="col-sm-2">Email</dt>
```

```
<dd class="col-sm-10">@Model.Email</dd>
```

```
<dt class="col-sm-2">Course</dt>
```

```
<dd class="col-sm-10">@Model.Course</dd>
```

```
<dt class="col-sm-2">Age</dt>
```

```
<dd class="col-sm-10">@Model.Age</dd>
```

```
</dl>
```

```
<form asp-action="Delete" method="post">
```

```
<input type="hidden" asp-for="Id" />
```

```

        <button type="submit" class="btn btn-danger">Delete</button>
        <a asp-action="Index" class="btn btn-secondary">Cancel</a>
    </form>
</div>

```

Какво прави тази страница?

Тук:

```
@model StudentManagement.Models.Student
```

казваме, че страницата получава **един конкретен студент**.

Например:

```

Иван
Петров
ivan.petrov@example.com
Software Engineering
20

```

След това показваме информацията, за да може потребителят да провери **кой студент ще бъде изтрит**.

Тази част:

```
<input type="hidden" asp-for="Id" />
```

запазва Id на студента, който ще бъде изпратен при потвърждението.

А:

```
<form asp-action="Delete" method="post">
```

означава, че бутонът **Delete** ще изпрати POST заявка към Delete action.

Важно

Все още **нямаме POST Delete в Controller-a**.

Следователно:

```

Students
↓
Delete
↓
Delete.cshtml
↓
Delete button

```

ще покаже страницата, но още няма да изтрие записа.

🔗 Сега:

1. Постави кода.
2. **Ctrl + S**
3. **Ctrl + Shift + B**
4. Изпрати ми Build резултата.

После ще направим последната част — реалното изтриване от SQL Server.

185. Вие казахте:

```

Build started at 1:00 AM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->

```

```
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 1:00 AM and took 00.775 seconds =====
```

186. ChatGPT каза:

Перфектно! 

Build-ът е успешен и Delete.cshtml е наред.

Стъпка 5.28 — тестваме страницата за потвърждение

Преди да добавим реалното изтриване, ще проверим само дали страницата се отваря правилно.

1. Стартирай приложението.
2. Отиди на **Students**.
3. Намери тестовия студент.
4. Натисни **Delete**.

Трябва да се отвори страница приблизително така:

Delete Student

Are you sure you want to delete this student?

First Name	Иван
Last Name	Петров
Email	...
Course	Software Engineering
Age	20

Delete Cancel

 **Много важно**

Не натискай червения Delete бутон още.

Все още не сме добавили POST метода, който реално изтрива записа.

Провери само дали:

- страницата се отваря;
- правилният студент се показва;
- виждаш бутоните **Delete** и **Cancel**.

Ако всичко е наред, кажи ми „**работи**“ и ще направим последната стъпка от CRUD — реалното изтриване от базата.

187. Вие казахте:

работи

188. ChatGPT каза:

Супер! 🎉 Значи GET Delete и страницата за потвърждение работят.

Сега идва **последната част на CRUD — реалното изтриване.**

Стъпка 5.29 — POST Delete

Отвори:

```
Controllers  
└─ StudentsController.cs
```

Под GET Delete метода добави:

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> DeleteConfirmed(int id)  
{  
    var student = await _context.Students.FindAsync(id);  
  
    if (student == null)  
    {  
        return NotFound();  
    }  
  
    _context.Students.Remove(student);  
    await _context.SaveChangesAsync();  
  
    return RedirectToAction(nameof(Index));  
}
```

Но имаме една малка промяна в Delete.cshtml

В момента формата има:

```
<form asp-action="Delete" method="post">
```

Трябва да я променим на:

```
<form asp-action="DeleteConfirmed" method="post">
```

Така натискането на **Delete** ще извика точно:

```
DeleteConfirmed(int id)
```

Какво ще се случи?

```
Delete.cshtml  
↓  
DeleteConfirmed(id)  
↓  
FindAsync(id)  
↓  
Remove(student)  
↓  
SaveChangesAsync()  
↓  
SQL Server  
↓  
студентът е изтрит  
↓  
/Students
```

Remove() маркира студента за изтриване, а SaveChangesAsync() реално изпраща промяната към SQL Server.

Защо не използваме директно Delete()?

За да разделим ясно двете действия:

```
GET Delete  
↓  
Показва потвърждението
```

POST DeleteConfirmed

↓

Реално изтрива

Това е по-ясна структура за нашия проект.

Сега

1. **Добави** DeleteConfirmed **в** StudentsController.
2. **В** Delete.cshtml **промени** само:
3. `asp-action="Delete"`

на:

`asp-action="DeleteConfirmed"`

4. **Ctrl + S**
5. **Ctrl + Shift + B**
6. Изпрати ми Build резултата.

 **Не натискай Delete в браузъра още.** Първо ще проверим Build-a.

189. Вие казахте:

така изглежда Delete.cshtml @model
StudentManagement.Models.Student <h1>Delete Student</h1>
<h3>Are you sure you want to delete this student?</h3> <div> <dl
class="row"> <dt class="col-sm-2">First Name</dt> <dd class="col-
sm-10">@Model.FirstName</dd> <dt class="col-sm-2">Last
Name</dt> <dd class="col-sm-10">@Model.LastName</dd> <dt
class="col-sm-2">Email</dt> <dd class="col-sm-
10">@Model.Email</dd> <dt class="col-sm-2">Course</dt> <dd
class="col-sm-10">@Model.Course</dd> <dt class="col-sm-
2">Age</dt> <dd class="col-sm-10">@Model.Age</dd> </dl>
<form asp-action="DeleteConfirmed" method="post"> <input
type="hidden" asp-for="Id" /> <button type="submit" class="btn
btn-danger">Delete</button> <a asp-action="Index" class="btn
btn-secondary">Cancel </form> </div>

190. ChatGPT каза:

Да, **точно така трябва да изглежда.** 

Особено важно е, че вече си променил:

`<form asp-action="DeleteConfirmed" method="post">`

Това е правилно и съвпада с метода, който добавихме в Controller-a.

Сега Delete.cshtml е ГОТОВ.

Следващото

Ако вече си добавил и този метод в StudentsController:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    _context.Students.Remove(student);
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}
```

направи:

Ctrl + S → Ctrl + Shift + B

и ми изпрати резултата от Build.

Ако Build е успешен, ще направим **контролирано изтриване на тестовия студент** и ще проверим и в `dbo.Students`, както направихме при Edit.

191. Вие казахте:

```
StudentsController изглежда така using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore; using
StudentManagement.Data; using StudentManagement.Models;
namespace StudentManagement.Controllers; public class
StudentsController : Controller { private readonly
ApplicationDbContext _context; public
StudentsController(ApplicationDbContext context) { _context =
context; } public async Task<IActionResult> Index() { var students =
await _context.Students.ToListAsync(); return View(students); } public
IActionResult Create() { return View(); } [HttpPost]
[ValidateAntiForgeryToken] public async Task<IActionResult>
Create(Student student) { if (!ModelState.IsValid) { return
View(student); } _context.Students.Add(student); await
_context.SaveChangesAsync(); return
RedirectToAction(nameof(Index)); } public async Task<IActionResult>
Edit(int? id) { if (id == null) { return NotFound(); } var student = await
_context.Students.FindAsync(id); if (student == null) { return
NotFound(); } return View(student); } [HttpPost]
[ValidateAntiForgeryToken] public async Task<IActionResult> Edit(int
id, Student student) { if (id != student.Id) { return NotFound(); } if
(!ModelState.IsValid) { return View(student); }
_context.Update(student); await _context.SaveChangesAsync(); return
```

```
RedirectToAction(nameof(Index)); } public async Task<IActionResult>
Delete(int? id) { if (id == null) { return NotFound(); } var student =
await _context.Students.FirstOrDefaultAsync(m => m.Id == id); if
(student == null) { return NotFound(); } return View(student); }
[HttpPost] [ValidateAntiForgeryToken] public async
Task<IActionResult> DeleteConfirmed(int id) { var student = await
_context.Students.FindAsync(id); if (student == null) { return
NotFound(); } _context.Students.Remove(student); await
_context.SaveChangesAsync(); return
RedirectToAction(nameof(Index)); } }
```

192. ChatGPT каза:

Да — StudentsController **ти е правилен.**  Няма нужда да променяме нищо в него на този етап.

Имаш вече трите основни операции:

- Create → добавя студент
- Edit → редактира студент
- Delete → изтрива студент

И DeleteConfirmed е свързан правилно с ТВОЯ Delete.cshtml.

Стъпка 5.30 — Build

Сега направи:

Ctrl + S

след това:

Ctrl + Shift + B

Ако получиш:

Build: 1 succeeded, 0 failed

ще направим теста.

След успешния Build

Ще стартираме приложението и ще направим следното:

4. Отваряме **Students**.
5. Натискаме **Delete** на тестовия студент.
6. Проверяваме дали се показва страницата за потвърждение.
7. Натискаме червения **Delete**.
8. Проверяваме, че студентът е изчезнал от таблицата.
9. Проверяваме и `dbo.Students` в SQL Server.

Така ще потвърдим, че **Delete работи от браузъра чак до SQL Server.**

Направи Build-а и ми изпрати резултата.

193. Вие казахте:

```
Build started at 1:12 AM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 1:13 AM and took 07.154 seconds =====
```

194. ChatGPT каза:

Перфектно!  Build-ът е успешен.
Сега правим **реалния тест на Delete**.

Стъпка 5.31 — изтриваме тестовия студент

Стартирай приложението и отвори:

/Students

Намери тестовия студент и:

1. Натисни **Delete**.
2. Ще се отвори страницата **Delete Student**.
3. Провери дали показва правилния студент.
4. Натисни червения **Delete**.

След това провери две неща

В сайта:

/Students

Студентът трябва да е изчезнал от таблицата.

След това в **SQL Server Management Studio**:

StudentManagementDb

→ Tables

→ dbo.Students

Изпълни **Select Top 1000 Rows**.

Студентът също трябва да е изчезнал от таблицата.

 Ако имаш други студенти, **не ги изтривай** — изтрий само тестовия, който използвахме.

Кажи ми дали студентът е изчезнал **и от сайта, и от** dbo.Students.

195. Вие казахте:

да

196. ChatGPT каза:

Супер!  **CRUD-ът вече работи изцяло.**

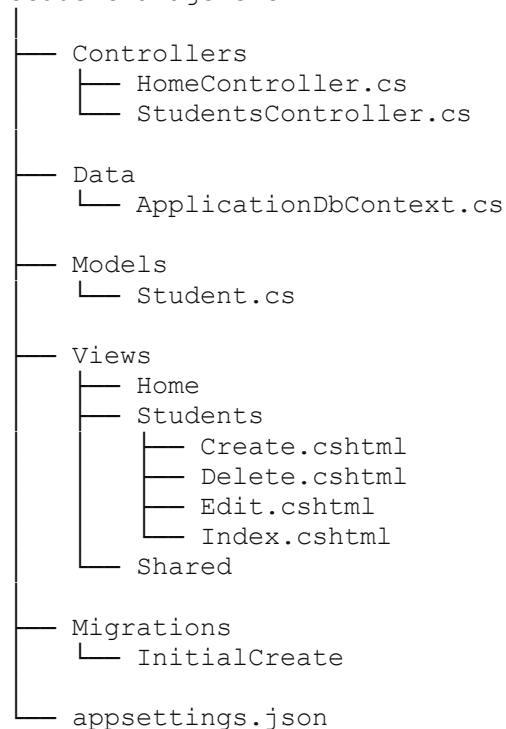
Имаме:

CREATE → добавяне
READ → показване
UPDATE → редактиране
DELETE → изтриване

И проверихме всичко и директно в SQL Server. Това означава, че основната функционалност на Student Management System вече е налице. ✓

Какво имаме в проекта

StudentManagement



И базата:

StudentManagementDb
└─ dbo.Students

Следващата логична стъпка

Преди да добавяме нови функции, бих предложил да направим **Details** страница.

Тя ще позволява:

Students

↓

Details

↓

/Students/Details/1

и ще показва информацията за конкретния студент, без да може да я променя.

След това можем да подобрим интерфейса и да добавим например:

- търсене на студент;
- филтриране;
- сортиране;
- validation съобщения на български;
- по-добър Bootstrap дизайн;

- навигация;
- статистика за броя студенти.

Но ще продължим **една стъпка наведнъж**, както досега.

197. Вие казахте:

може ли преди да пристъпим към Details да направим една промяна само в таблицата да сложим пореден номер и да си оставим id-то и ако работим в таблицата при нужда ще го крием и ще го правим недостъпно за промяна

198. ChatGPT каза:

Да, разбира се. 👍 И според мен това е **по-добрият вариант**. Нека запазим id като вътрешен идентификатор на базата, а в таблицата на сайта да имаме отделна колона **№**, която е само за визуално подреждане.

Например:

№	First Name	Last Name	Email	Course	Age	Actions
1	Иван	Петров	...	...	20	Edit / Delete
2	Мария	Иванова	...	...	21	Edit / Delete

При това:

- **№** → визуален пореден номер за потребителя.
- Id → остава в базата и в модела Student.
- Id **не го показваме в таблицата**.
- Id **не може да се променя от потребителя**.
- При Edit и Delete продължаваме да използваме истинския Id вътрешно.
- Не променяме базата и **не ни трябва нова Migration**.

Как ще направим поредния номер?

В Index.cshtml можем да използваме брояч, например:

```
@{
    var number = 1;
}
```

и при всеки ред:

```
<td>@number</td>
```

след което увеличаваме:

```
number++;
```

Така № е само визуален номер и **не е част от базата**.

Това е важно, защото ако изтрием студент №2, например:

№ 1 → Иван

№ 2 → Мария

№ 3 → Петър

след изтриването може да стане:

№ 1 → Иван
№ 2 → Петър

докато истинските Id стойности в базата могат да бъдат:

Id 1 → Иван
Id 3 → Петър

И това е напълно нормално. **Не трябва да променяме Id, за да поддържаме визуална номерация.**

Ако си съгласен, ще променим **само** Index.cshtml, без да пипаме Controller, Model или SQL Server.

199. Вие казахте:

ОК

200. ChatGPT каза:

201. Вие казахте:

съгласен съм

202. ChatGPT каза:

203. Вие казахте:

ето кода в Index.cshtml @model
IEnumerable<StudentManagement.Models.Student>
<h1>Students</h1> <a asp-action="Create" class="btn btn-primary">Add Student <table class="table"> <thead> <tr>
<th>First Name</th> <th>Last Name</th> <th>Email</th>
<th>Course</th> <th>Age</th> <th>Actions</th> </tr> </thead>
<tbody> @foreach (var student in Model) { <tr>
<td>@student.FirstName</td> <td>@student.LastName</td>
<td>@student.Email</td> <td>@student.Course</td>
<td>@student.Age</td> <td> <a asp-action="Edit" asp-route-id="@student.Id" class="btn btn-warning">Edit <a asp-action="Delete" asp-route-id="@student.Id" class="btn btn-danger">Delete </td> </tr> } </tbody> </table>

204. ChatGPT каза:

205. Вие казахте:

ОК

206. ChatGPT каза:

Супер. 👍 Ще направим само промяната в Index.cshtml.

Стъпка 5.32 — добавяме пореден №

Отвори:

Views → Students → Index.cshtml

Първо ще ти покажа какво правим, без да променяме Student.cs или базата.

1. Добавяме брояча

В началото на файла, след @model, добави:

```
@{
    var number = 1;
}
```

2. Добавяме колоната №

В <thead> най-отпред добави:

```
<th>№</th>
```

3. Добавяме номера към всеки ред

Във foreach, преди името на студента, добави:

```
<td>@number</td>
```

4. Увеличаваме номера

В края на всеки ред, след Actions колоната, добави:

```
@{
    number++;
}
```

В крайна сметка

Твоята таблица трябва да изглежда приблизително така:

```
@model IEnumerable<StudentManagement.Models.Student>

@{
    var number = 1;
}

<h1>Students</h1>

<p>
    <a asp-action="Create" class="btn btn-primary">Add Student</a>
</p>

<table class="table">
    <thead>
        <tr>
            <th>№</th>
            <th>First Name</th>
            <th>Last Name</th>
            <th>Email</th>
            <th>Course</th>
            <th>Age</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var student in Model)
        {
            <tr>
                <td>@number</td>
                <td>@student.FirstName</td>
                <td>@student.LastName</td>
                <td>@student.Email</td>
                <td>@student.Course</td>
                <td>@student.Age</td>
                <td>
                    <a asp-action="Edit" asp-route-id="@student.Id"
class="btn btn-warning">Edit</a>
                    <a asp-action="Delete" asp-route-id="@student.Id"
class="btn btn-danger">Delete</a>
                </td>
            </tr>
        }
```

```

        </tr>

        number++;
    }
</tbody>
</table>

```

Важно

Тук:

```
<td>@number</td>
```

показваме **пореждния номер**.

А тук:

```
asp-route-id="@student.Id"
```

продължаваме да използваме **истинското** Id за Edit/Delete.

Тоест двете са напълно отделни:

№ → само за показване

Id → вътрешен идентификатор

Не променяме Student.cs, ApplicationDbContext **или SQL Server**.

След като го направиш:

Ctrl + S → Ctrl + Shift + B

Изпрати ми Build резултата и ще проверим таблицата.

207. Вие казахте:

```

Build started at 10:50 AM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 10:50 AM and took 06.531 seconds =====

```

208. ChatGPT каза:

Перфектно! ☒ Build-ът е успешен.

Сега стартирай приложението и отвори **Students**.

Трябва да видиш:

№	First Name	Last Name	Email	Course	Age	Actions
1	...	...	...	...	...	Edit / Delete
2	...	...	...	...	...	Edit / Delete

Ако имаш само един студент, ще има:

№ 1

Проверка

Изтрий или добави студенти и ще видиш, че **№** се изчислява отново при зареждането на страницата.

Например ако базата има:

Id 1 → Иван

Id 3 → Петър

Id 7 → Мария

на сайта ще виждаш:

№ 1 → Иван

№ 2 → Петър

№ 3 → Мария

Точно това искаме. 👍

И най-важното — **не сме променили Id в базата.**

Когато потвърдиш, че таблицата изглежда правилно, продължаваме към **Details**.

209. Вие казахте:

ок. Може ли да питам нещо преди да продължим

210. ChatGPT каза:

Разбира се. 👍 Питай спокойно, преди да продължим към **Details**.

211. Вие казахте:

може ли да обясниш в детайли кода

212. ChatGPT каза:

Разбира се. 👍 И даже е много подходящ момент да го направим, защото вече имаме работещ проект и можем да разгледаме **как всяка част си взаимодейства**.

Ще обясня кода **като за начинаещ**, без да приемам, че вече знаеш ASP.NET Core или Entity Framework.

Ще започнем от най-важното: **какво се случва, когато отвориш /Students**.

1. Как работи целият поток

Когато напишеш:

`https://localhost:7295/Students`

се случва приблизително това:

Браузър



StudentsController



ApplicationDbContext



SQL Server



Student обекти



Index.cshtml



HTML



Браузър

Нека разгледаме всяка част.

2. Student.cs — какво е Student?

Нашият модел е:

```
using System.ComponentModel.DataAnnotations;

namespace StudentManagement.Models
{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string FirstName { get; set; } = string.Empty;

        [Required]
        public string LastName { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        public string Course { get; set; } = string.Empty;

        public int Age { get; set; }
    }
}
```

Какво е class?

```
public class Student
```

Това е описание на един студент.

Можеш да си го представиш като шаблон:

```
Student
├── Id
├── FirstName
├── LastName
├── Email
├── Course
└── Age
```

От този шаблон можем да създаваме обекти.

Например:

```
Student
Id: 1
FirstName: Иван
LastName: Петров
Email: ivan@example.com
Course: Software Engineering
Age: 20
```

3. Id

```
public int Id { get; set; }
```

Това е уникалният идентификатор на студента.

Entity Framework Core разпознава Id като **primary key**.

Например:

```
Id  FirstName
1   Иван
2   Мария
3   Петър
```

Тези стойности се съхраняват в SQL Server.

И тук идва нашето предишно решение:

№ → визуален пореден номер

Id → реален идентификатор

Например:

№ 1 → Id 17 → Иван

№ 2 → Id 25 → Мария

№ 3 → Id 31 → Петър

Затова **№** и **Id** не са едно и също нещо.

4. [Required]

Например:

[Required]

```
public string FirstName { get; set; } = string.Empty;
```

[Required] казва:

FirstName е задължително поле.

Ако потребителят изпрати празно име, ModelState ще бъде невалиден.

Точно това проверяваме в Controller-а:

```
if (!ModelState.IsValid)
{
    return View(student);
}
```

5. [EmailAddress]

[EmailAddress]

```
public string Email { get; set; } = string.Empty;
```

Това казва:

Email трябва да има валиден формат за имейл адрес.

Например:

ivan@example.com

е валиден формат.

6. ApplicationDbContext

Нашият клас е:

```
public class ApplicationDbContext : DbContext
```

Той е **връзката между C# приложението и SQL Server.**

Имаме:

```
public DbSet<Student> Students { get; set; }
```

Това е особено важно.

DbSet<Student> представлява колекцията от студенти, с която Entity Framework работи.

Можеш да си го представиш така:

```
C#
ApplicationDbContext
    ↓
DbSet<Student>
    ↓
```

SQL Server
↓
dbo.Students

7. StudentsController

Това е контролерът, който управлява студентите.

Имаме:

```
public class StudentsController : Controller
```

Името StudentsController е важно.

ASP.NET Core MVC автоматично го свързва с:

```
/Students
```

8. _context

В Controller-а имаме:

```
private readonly ApplicationDbContext _context;
```

Това е променлива, чрез която Controller-ът използва базата.

В конструктора:

```
public StudentsController(ApplicationDbContext context)
{
    _context = context;
}
```

ASP.NET Core автоматично ни предоставя ApplicationDbContext.

Това се нарича **Dependency Injection**.

Не е необходимо сега да запомняш термина. Важно е да разбереш идеята:

```
StudentsController
    ↓
получава ApplicationDbContext
    ↓
може да работи с базата
```

9. Index()

Имаме:

```
public async Task<IActionResult> Index()
{
    var students = await _context.Students.ToListAsync();

    return View(students);
}
```

Това е кодът, който зарежда списъка.

```
_context.Students
_context.Students
```

казва:

Работи с колекцията Students.

```
ToListAsync()
await _context.Students.ToListAsync();
```

казва:

Вземи студентите от базата и ги превърни в списък.

Получаваме:

List<Student>

10. Какво означава `async` и `await`?

Имаме:

```
public async Task<IActionResult> Index()
```

и:

```
await _context.Students.ToListAsync();
```

Това позволява операцията към базата да бъде **асинхронна**.

С други думи, приложението не трябва да блокира нишката, докато SQL Server изпълнява заявката.

За нашия проект просто запомни:

```
ToListAsync()
```

↓

операция към базата

↓

```
await
```

↓

изчакваме резултата

11. `return View(students)`

Това:

```
return View(students);
```

казва:

Използвай View-то и му подай списъка със студенти.

ASP.NET MVC намира:

Views

└─ Students

 └─ Index.cshtml

Тоест:

```
Index()
```

↓

Views/Students/Index.cshtml

12. `Index.cshtml`

В началото имаме:

```
@model IEnumerable<StudentManagement.Models.Student>
```

Това казва:

Този Razor страница получава колекция от Student.

Тоест:

```
Index.cshtml
```

↑

```
List<Student>
```

13. Нашият `number`

Имаме:

```
@{  
    var number = 1;  
}
```

Това е Razor блок.
В него можем да пишем C#.
Тук създаваме променлива:
`number = 1`

14. foreach

След това:
`@foreach (var student in Model)`

Това означава:

За всеки студент в списъка Model изпълни следващия код.

Ако имаме:

Иван
Мария
Петър

цикълът ще се изпълни три пъти.

Първият път:

`student = Иван`

вторият:

`student = Мария`

третият:

`student = Петър`

15. Поредният номер

Имаме:

`<td>@number</td>`

Първия път:

`number = 1`

затова показва:

1

След това:

`number++;`

означава:

`number = number + 1`

Следващият студент получава:

2

и така нататък.

16. Защо @student.FirstName?

Имаме:

`<td>@student.FirstName</td>`

student е текущият обект.

А:

`student.FirstName`

означава:

Вземи FirstName от този Student.

Например:

```
student
├── Id = 1
├── FirstName = Иван
├── LastName = Петров
├── Email = ...
├── Course = ...
└── Age = 20
```

Тогава:

```
@student.FirstName
```

показва:

Иван

17. Edit бутонът

Имаме:

```
<a asp-action="Edit"
    asp-route-id="@student.Id"
    class="btn btn-warning">
    Edit
</a>
```

Това е много важна част.

```
asp-action
asp-action="Edit"
```

казва:

Използвай action-а Edit.

```
asp-route-id
asp-route-id="@student.Id"
```

казва:

Изпрати Id на конкретния студент.

Ако:

```
student.Id = 17
```

URL-ът ще бъде:

```
/Students/Edit/17
```

Затова можем да намерим точно този студент.

18. GET Edit

В Controller-а имаме:

```
public async Task<IActionResult> Edit(int? id)
```

Тук:

```
int? id
```

означава, че id може да има стойност или да бъде null.

След това:

```
var student = await _context.Students.FindAsync(id);
```

търсим студента по Id.

Например:

```
FindAsync(17)
```

търси:

```
Id = 17
```

19. Edit.cshtml

След като намерим студента:

```
return View(student);
```

изпращаме го към:

```
Edit.cshtml
```

Там имаме:

```
@model StudentManagement.Models.Student
```

Следователно Model е **един Student**, а не списък.

Затова можем да правим:

```
@Model.FirstName
```

20. Hidden Id

В Edit формата имаме:

```
<input type="hidden" asp-for="Id" />
```

Това е много важно.

Id не се показва като нормално поле, но се изпраща заедно с формата.

Например:

```
Id = 17
```

```
FirstName = Иван
```

```
LastName = Петров
```

```
...
```

Така POST методът знае кой студент редактираме.

21. POST Edit

След натискане на Save се изпълнява:

```
[HttpPost]
```

```
[ValidateAntiForgeryToken]
```

```
public async Task<IActionResult> Edit(int id, Student student)
```

Тук вече имаме:

```
id
```

```
+
```

```
student
```

ASP.NET MVC автоматично взема данните от формата и ги поставя

в Student.

22. ModelState.IsValid

Имаме:

```
if (!ModelState.IsValid)
```

```
{
```

```
    return View(student);
```

```
}
```

Това проверява дали данните отговарят на правилата в Student.

Например:

```
FirstName → задължително
```

```
LastName → задължително
```

```
Email → email формат
```

Ако не са валидни:

```
ModelState.IsValid = false
```

и формата се показва отново.

23. Update

След това:

```
_context.Update(student);
```

казва:

Този Student трябва да бъде актуализиран.

И:

```
await _context.SaveChangesAsync();
```

изпраща промяната към SQL Server.

24. Create

Create работи по сходен начин.

GET:

```
public IActionResult Create()
{
    return View();
}
```

показва празната форма.

POST:

```
public async Task<IActionResult> Create(Student student)
```

получава въведените данни.

След това:

```
_context.Students.Add(student);
```

казва:

Добави този Student.

И:

```
await _context.SaveChangesAsync();
```

записва го в базата.

25. Delete

При Delete имаме две стъпки.

Първо:

```
public async Task<IActionResult> Delete(int? id)
```

намира студента и показва страницата за потвърждение.

След натискане на Delete:

```
public async Task<IActionResult> DeleteConfirmed(int id)
```

намира студента:

```
var student = await _context.Students.FindAsync(id);
```

и го премахва:

```
_context.Students.Remove(student);
```

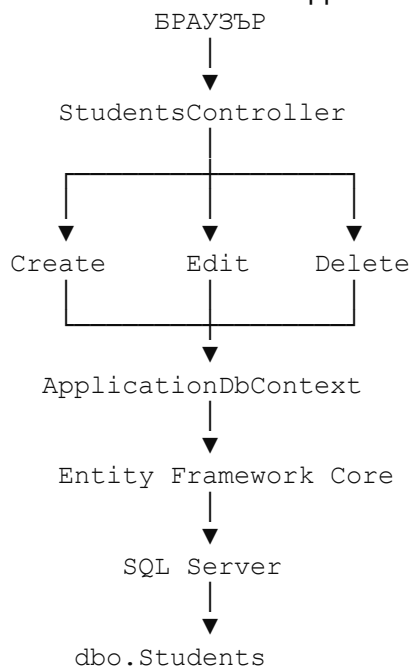
След това:

```
await _context.SaveChangesAsync();
```

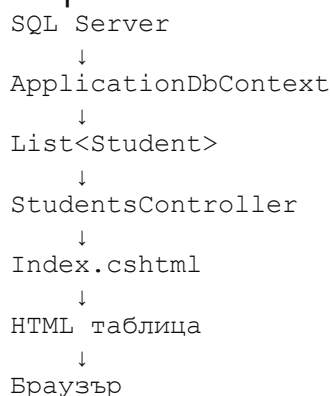
реално го изтрива от SQL Server.

26. Най-важната картина

Ако запомниш само една схема от всичко това, нека бъде тази:



А при показване:



И още нещо много важно

В момента вече имаш **ясно разделение на отговорностите**:

Models

→ Какви данни има Student

Data

→ Как приложението говори с базата

Controllers

→ Какво да се случи при дадена заявка

Views

→ Какво вижда потребителят

Това е основната идея на **ASP.NET Core MVC**.

И според мен е добре да задържим темпото точно така — **не просто да копираме код, а да разбираме какво прави всяка част**. Следващия път, когато добавяме Details, мога да обяснявам всеки нов ред по същия начин.

1. може ли да отделим студент на отделни неща-1/ полета и 2/методи както в ооп-то и всяко нещо да стои в отделна файл/папка нещо като това. Отделна папка да има за грешки и там ако се налага да ги извикваме ако се налага ето какво имам предвид <https://github.com/esp-nv/CSharp/tree/main/04.CSharpOOP>

2. ChatGPT каза:

Да — **може**, и всъщност идеята ти е много добра, ако целта е да учиш ООП, докато изграждаме приложението. Разгледах структурата, която си посочил в GitHub: тя действително разделя темите по ООП като **Encapsulation, Inheritance, Interfaces and Abstraction, Polymorphism, Reflection and Attributes** и **SOLID**. G



GitHub

Само бих направил едно важно уточнение:

Не бих разделял всяко едно поле на отделен файл. Това би направило ASP.NET MVC проекта прекалено раздробен. Вместо това можем да приложим ООП принципите на правилните места.

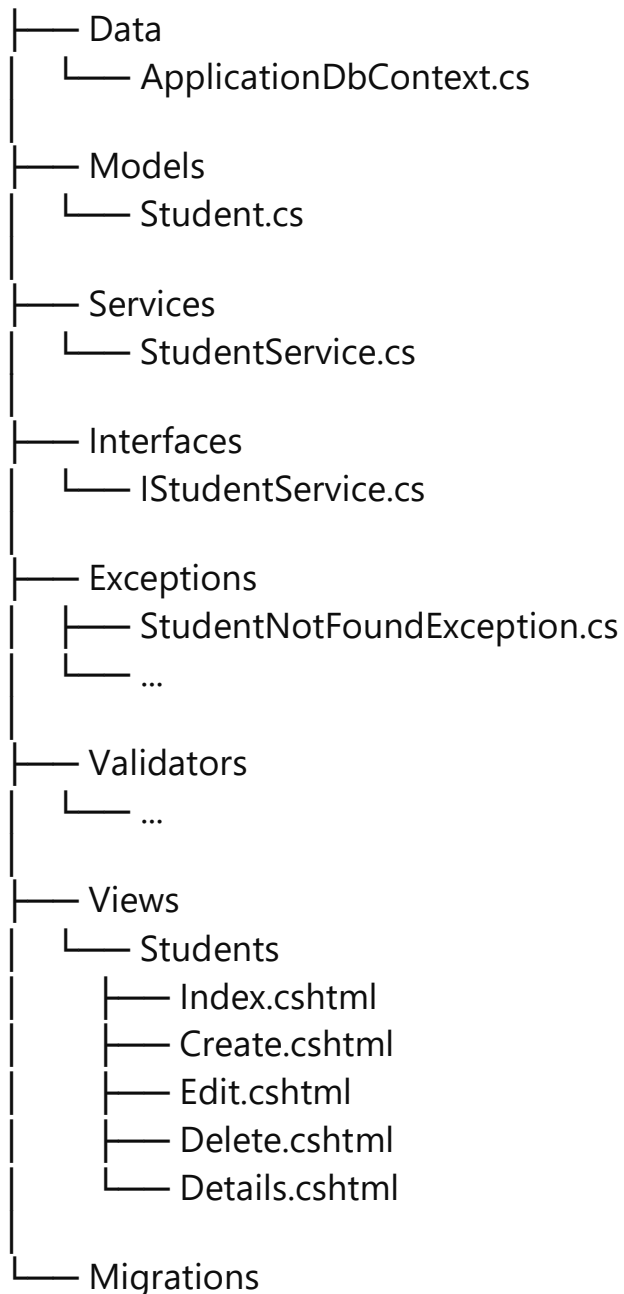
Какво предлагам за нашия проект

В момента имаме:

```
StudentManagement
├── Controllers
│   └── StudentsController.cs
├── Data
│   └── ApplicationDbContext.cs
├── Models
│   └── Student.cs
├── Views
│   ├── Students
│   │   ├── Index.cshtml
│   │   ├── Create.cshtml
│   │   ├── Edit.cshtml
│   │   └── Delete.cshtml
└── Migrations
```

Можем постепенно да го превърнем в нещо по-структурирано:

```
StudentManagement
├── Controllers
│   └── StudentsController.cs
```



Това вече започва да прилича на истинско приложение, а не просто на CRUD упражнение.

Но има нещо много важно

Не искам да променяме всичко наведнъж.

В момента имаме **работещ CRUD**. Това е нашата стабилна основа.

Можем да направим следващите стъпки така:

1. Student

↓

2. Encapsulation

↓

3. StudentService

↓

4. Interface

↓

5. Exceptions

↓

6. Validation

↓

7. Details

↓

8. SOLID

И на всяка стъпка ще ти обяснявам **защо** създаваме новия файл/папка и какво печелим от това.

1. Полетата на Student

В момента имаме:

```
public int Id { get; set; }
```

```
public string FirstName { get; set; } = string.Empty;
```

```
public string LastName { get; set; } = string.Empty;
```

```
public string Email { get; set; } = string.Empty;
```

```
public string Course { get; set; } = string.Empty;
```

```
public int Age { get; set; }
```

Това са **properties**, а не просто обикновени полета.

И тук вече можем да започнем да учим **Encapsulation**.

Например вместо всяко нещо да може да се променя отвсякъде:

```
student.Age = -50;
```

можем постепенно да въведем правила:

Age

↓

трябва да бъде валидна възраст

↓

ако не е → грешка

Това е истинската идея на encapsulation:

Обектът контролира собственото си състояние.

2. Методите

Тук твоето предложение е особено интересно.

В момента Controller-ът съдържа:

```
Create()
```

```
Edit()
```

```
Delete()
```

```
Index()
```

Но тези методи всъщност са **HTTP действия**.

Те не е задължително да съдържат цялата бизнес логика.

Например в бъдеще:

StudentsController



StudentService



ApplicationDbContext



SQL Server

Тогава Controller-ът ще казва:

"Искам да създам студент"

а Service ще се занимава с логиката:

"Може ли този студент да бъде създаден?"

"Как се записва?"

"Има ли вече такъв?"

Това е много по-близо до добрия ООП дизайн.

3. Отделна папка Exceptions

Това също **може и има смисъл**, но не трябва да хвърляме exception за всяка грешка.

Например можем да имаме:

Exceptions

└─ StudentNotFoundException.cs

и:

```
public class StudentNotFoundException : Exception
```

```
{
```

```
    public StudentNotFoundException(int studentId)
```

```
        : base($"Student with ID {studentId} was not found.")
```

```
    {
```

```
    }
```

```
}
```

Тогава:

StudentService



не намира Student



StudentNotFoundException



обработване на грешката

Това вече е истински пример за **ООП + Exceptions**.

4. Интерфейси

После можем да имаме:

Interfaces

└─ IStudentService.cs

с:

```
public interface IStudentService
{
    Task<List<Student>> GetAllAsync();

    Task<Student?> GetByIdAsync(int id);

    Task CreateAsync(Student student);

    Task UpdateAsync(Student student);

    Task DeleteAsync(int id);
}
```

А след това:

IStudentService

↑
| implements

StudentService

Това вече ни въвежда в:

- abstraction;
- interfaces;
- dependency injection;
- SOLID.

5. А Controller-ът ще стане много по-чист

В момента той знае прекалено много за базата:

_context.Students.FindAsync(...)

_context.Students.Remove(...)

_context.SaveChangesAsync()

В бъдеще можем да стигнем до:

```
public class StudentsController : Controller
{
    private readonly IStudentService _studentService;

    public StudentsController(IStudentService studentService)
    {
        _studentService = studentService;
    }
}
```

```

    }
}
И например:
public async Task<ActionResult> Index()
{
    var students = await _studentService.GetAllAsync();

    return View(students);
}

```

Това е много по-чисто:

Controller

↓

Service

↓

Data

↓

Database

6. Има обаче едно нещо, което НЕ бих направил

Не бих направил:

Student

```

├── FirstName.cs
├── LastName.cs
├── Email.cs
├── Course.cs
└── Age.cs

```

Това **не е добра практика**.

По-добре:

Student.cs

да съдържа състоянието на Student:

Id

FirstName

LastName

Email

Course

Age

и свързаното с него поведение, когато има смисъл.

Така Student остава един цялостен обект.

7. И още нещо — /// <summary>

Тук можем да използваме точно уточнението, което ми даде по-рано:

/// <summary> само ако се налага.

Съгласен съм.

Няма да правим:

/// <summary>

/// Gets the student.

/// </summary>

public Student GetStudent()

за всеки метод само за да има коментар.

Ще използваме XML документация **само когато има реална нужда от уточнение**, например при по-сложна бизнес логика, публичен интерфейс или нестандартно поведение.

И най-важното

Не бих започнал Details още сега.

Ти повдигна много добър въпрос и според мен е по-добре първо да решим **архитектурата**, която искаме да учим.

Можем да направим този проект не просто:

„ASP.NET CRUD приложение“

а:

ASP.NET Core MVC + Entity Framework Core + SQL Server + OOP + Encapsulation + Abstraction + Interfaces + Exceptions + SOLID като при това **не усложняваме проекта без причина**.

И ще запазим вече работещото — няма да чупим CRUD-а, а ще го **рефакторираме постепенно**, за да видиш защо всяка промяна е необходима.

1. може ли още нещо
2. **ChatGPT каза:**

Разбира се. 😊 Кажи какво имаш предвид. Ще го разгледаме спокойно, преди да продължим с проекта.

3. Вие казахте:

тъй като планът е този Планът - Student Management System Ще изградим Student Management System, като постепенно добавяме:

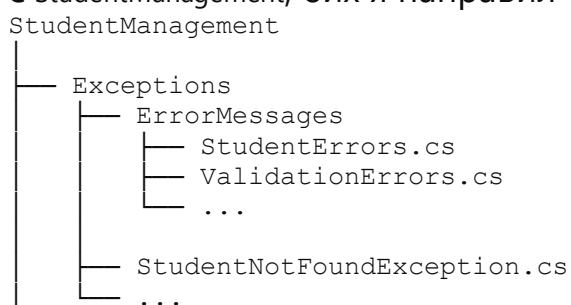
1. 👤 Students — добавяне, преглед, редактиране и изтриване
2. 🔍 Search — търсене на студент
3. 📖 Courses/Subjects — предмети
4. 📊 Grades — оценки
5. 📈 Average Grade — автоматично изчисляване на среден успех
6. 📅 Attendance — присъствия
7. 👤 Student Details — подробна страница за студент
8. 😊 По-добър интерфейс
9. 🗄 Database — ако текущият проект още не е настроен правилно за нея
10. 🚪 Login/roles — ако

искаме системата да има администратор и потребители
Exceptions-а да е с името на базата, да има
ErrorMessages/примерно име/ да трупаме там да не са като
магически числа и при нужда ще ги добавяме и извикваме чрез
имената им

4. ChatGPT каза:

Да — и според мен това е **много добра промяна в плана**. Даже
бих я заложил още сега като архитектурно правило за проекта.
Само едно уточнение: ако под „Exceptions-а да е с името на
базата“ имаш предвид **папката за грешки да бъде свързана**

с StudentManagement, бих я направил така:



Но **не бих слагал самите съобщения в отделен файл за всяка грешка**. По-добре да имаме централизирани класове с именувани константи.

Например:

```
namespace StudentManagement.Exceptions.ErrorMessage;

public static class StudentErrors
{
    public const string NotFound = "Student was not found.";
    public const string InvalidId = "Student ID is invalid.";
    public const string EmailAlreadyExists = "A student with this email already exists.";
}
```

После вместо:

```
throw new Exception("Student was not found.");
```

ще използваме:

```
throw new StudentNotFoundException(StudentErrors.NotFound);
```

Така текстовете **не са разпръснати из проекта**.

За „магическите числа“

Тук съм напълно съгласен с теб.

Например не искаме навсякъде:

```
if (student.Age < 18)
```

ако 18 има специално значение за бизнес логиката.

Можем да имаме:

```
Constants
└── StudentConstants.cs
```

например:

```
public static class StudentConstants
{
```

```

        public const int MinimumAge = 18;
        public const int MaximumAge = 100;
    }

    и после:
    if (student.Age < StudentConstants.MinimumAge)
    {
        ...
    }

```

Това прави кода много по-разбираем.

Но бих разделил три различни неща

Не искам да сложим абсолютно всичко в Exceptions.

1. Constants

За стойности, които имат име и значение:

```

Constants
└─ StudentConstants.cs

```

2. ErrorMessages

За текстовете на грешките:

```

Exceptions
└─ ErrorMessages
   └─ StudentErrors.cs

```

3. Exceptions

За самите exception класове:

```

Exceptions
├─ StudentNotFoundException.cs
├─ StudentValidationException.cs
└─ ...

```

Така имаме:

```

Стойност
  ↓
Constants

Съобщение
  ↓
ErrorMessages

Exception
  ↓
Exceptions

```

Това е много по-чисто.

И бих променил плана ни малко

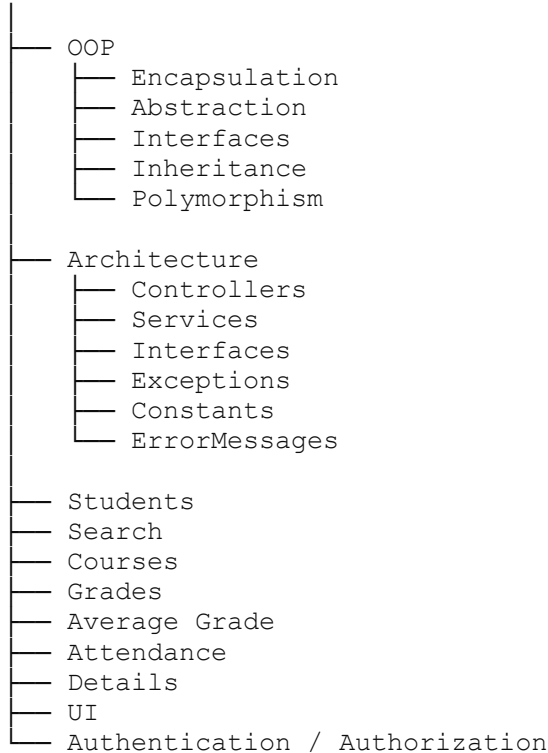
Твоят план остава:

1.  Students
2.  Search
3.  Courses/Subjects
4.  Grades
5.  Average Grade
6.  Attendance
7.  Student Details
8.  Interface

- 9.  Database
- 10.  Login/Roles

Но бих добавил **архитектурата като постоянна част от всяка стъпка**, а не като отделна функция:

Student Management System



Това е особено подходящо за теб, защото така няма просто да учим „**как се прави ASP.NET приложение**“, а ще виждаш как принципите от C# ООП се използват в реален проект.

И още нещо важно: **няма да добавяме папки и класове само заради самото добавяне**. Ако дадена абстракция няма реална полза на конкретния етап, ще ти кажа защо не я правим.

Така проектът ще остане чист, но постепенно ще стане доста по-професионален.

1. Вие казахте:

не забравяме и за solid принципите, но има нещо което не ми харесва

2. ChatGPT каза:

Разбира се — **SOLID остава задължително част от плана**.  Ще ги прилагаме постепенно, когато стигнем до подходящите места, а не просто да ги добавим теоретично.

Кажи ми какво не ти харесва. Ще го обсъдим първо и **няма да променяме кода**, докато не уточним как искаш да бъде организиран проектът.

3. Вие казахте:

може ли да е така <https://github.com/KrlsKa7a/CSharp-Web-Feb2026/tree/main/src> няма да е като по професионален проект

4. ChatGPT каза:

Да — разгледах `src` папката на проекта ти. В момента структурата е доста близка до типичен учебен ASP.NET Core проект, но **може да се направи по-професионално**, без да се усложнява излишно. 

GitHub

Виждам, че вече имаш отделени:

- Data
- Data.Models
- Data.Common
- Services.Core
- Services.Models
- Services.AutoMapping
- Web.Infrastructure
- Web.ViewModels
- тестови проекти

Това всъщност е **добра основа**. Подобно разделяне на слоеве се

използва и в реални C# проекти.  GitHub+1

Как бих го направил аз

Ако целта е да изглежда като **професионален проект за**

CV/GitHub, бих се стремил към нещо от този тип:

CSharp-Web-Feb2026/

```
├── src/
│   ├── CinemaApp.Web/
│   │   ├── Controllers/
│   │   ├── Views/
│   │   ├── ViewModels/
│   │   ├── Areas/
│   │   ├── Filters/
│   │   ├── Middleware/
│   │   ├── wwwroot/
│   │   └── Program.cs
│   ├── CinemaApp.Application/
│   │   ├── Services/
│   │   ├── Interfaces/
│   │   ├── DTOs/
│   │   └── Mapping/
│   ├── CinemaApp.Domain/
│   │   ├── Entities/
│   │   ├── Enums/
│   │   └── Constants/
│   └── CinemaApp.Infrastructure/
│       └── Data/
```

```

├── Repositories/
├── Configurations/
├── CinemaApp.Common/
│   ├── Constants/
│   ├── Extensions/
│   └── Helpers/
├── tests/
│   ├── CinemaApp.UnitTests/
│   ├── CinemaApp.IntegrationTests/
│   └── CinemaApp.Web.Tests/
├── .github/
│   └── workflows/
├── README.md
├── .gitignore
└── CinemaApp.sln

```

Това е по-близо до **Clean Architecture / layered architecture**, отколкото просто да имаш много проекти с Data, Services И Web.

Но има една важна уговорка

Не бих ти препоръчал просто да преименуваш сегашните проекти, само за да изглеждат „enterprise“.

Например сегашното:

```

CinemaApp.Services.Core
CinemaApp.Services.Common
CinemaApp.Services.Models
CinemaApp.Services.AutoMapping

```

може да стане прекалено раздробено.

За сравнително нормално ASP.NET Core приложение по-професионално може да е:

```

CinemaApp.Domain
CinemaApp.Application
CinemaApp.Infrastructure
CinemaApp.Web
CinemaApp.Common

```

и вътре да има ясни папки.

Например:

```

CinemaApp.Application
├── Interfaces/
│   ├── IMovieService.cs
│   ├── IActorService.cs
│   └── IGenreService.cs
├── Services/
│   ├── MovieService.cs
│   ├── ActorService.cs
│   └── GenreService.cs
├── DTOs/
│   ├── MovieDto.cs
│   └── ActorDto.cs
└── Mapping/
    └── MappingProfile.cs

```

A:

```

CinemaApp.Infrastructure
|

```

```

├── Data/
│   ├── CinemaDbContext.cs
│   └── Configurations/
├── Repositories/
│   ├── MovieRepository.cs
│   └── ActorRepository.cs
└── Migrations/

```

И:

CinemaApp.Web

```

├── Controllers/
├── Views/
├── ViewModels/
├── Middleware/
├── Filters/
├── Extensions/
└── wwwroot/

```

Твоят сегашен проект не е „непрофесионален“

Напротив — структурата ти вече показва, че си мислил за separation of concerns. Например имаш отделени integration/service/web tests, което е доста по-добре от всичко да

е в един проект.  GitHub

Просто ако искаш да изглежда **като проект, който бих видял в GitHub на junior/mid .NET developer**, бих направил основната архитектура по-ясна:

Domain → Application → Infrastructure → Web

и бих намалил броя на дребните *.Common, *.Core, *.Models проекти.

Ако искаш, мога да ти направя **конкретен refactoring план за точно този GitHub проект** — кое от сегашните CinemaApp.* да остане, кое да се премести и каква да стане финалната структура на src и tests, без да чупим функционалността.

1. можем ли да го приложим към нашия проект
2. **ChatGPT каза:**

Да, абсолютно. И според мен **има смисъл да го приложим към вашия проект**, но постепенно, а не да пренаписваме всичко наведнъж.

Бих направил следното:

```

src/
├── CinemaApp.Web/
│   ├── Controllers/
│   ├── Views/
│   ├── ViewModels/
│   ├── Middleware/
│   ├── Filters/
│   └── wwwroot/
├── CinemaApp.Application/
│   ├── Interfaces/
│   ├── Services/
│   └── DTOs/

```

```

├── Mapping/
├── CinemaApp.Domain/
│   ├── Entities/
│   ├── Enums/
│   └── Constants/
├── CinemaApp.Infrastructure/
│   ├── Data/
│   ├── Configurations/
│   ├── Repositories/
│   └── Migrations/
├── CinemaApp.Common/
│   ├── Constants/
│   ├── Extensions/
│   └── Helpers/
├── tests/
│   ├── CinemaApp.UnitTests/
│   ├── CinemaApp.IntegrationTests/
│   └── CinemaApp.Web.Tests/

```

Как ще го направим

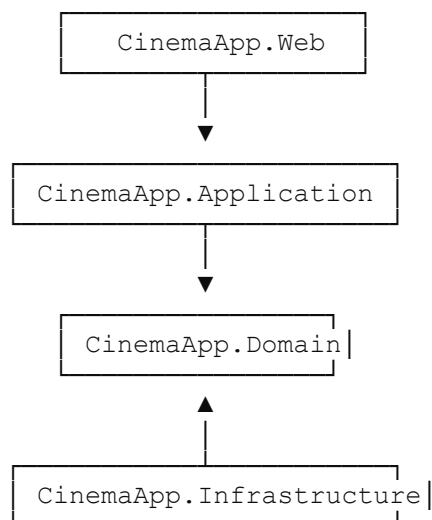
Не бих започнал с местене на файлове на случаен принцип.

Първо ще направим **mapping** на сегашния проект:

Сегашно	Ново
Data.Models	Domain/Entities
Data.Common	Common / Domain/Constants
Services.Core	Application/Services
Services.Models	Application/DTOs
Services.AutoMapping	Application/Mapping
Web.ViewModels	Web/ViewModels
Web.Infrastructure	Web/Infrastructure
EF Core / DbContext	Infrastructure/Data
Repository класове	Infrastructure/Repositories

След това ще оправим **Project References**, namespaces и using-ите.

Накрая целта е dependency flow-ът да е ясен:



Това вече ще изглежда като **реално архитектурно решение**, а не просто като проект, направен така, че да мине курса.

И най-важното: можем да го направим върху сегашния код, без да губим функционалността.

Ако искаш да го направим заедно, бих започнал от **точно сегашната структура на src** и ще ти дам конкретно: „**този проект → тази нова папка; този файл → тук; този reference → махаме/добавяме**“, стъпка по стъпка.

1. към нашия проект StudentManagement
2. ChatGPT каза:

Да — ако говорим за **нашия** StudentManagement **проект**, можем да приложим същия подход. И даже бих го направил така, че структурата да изглежда професионално, но **да не е over-engineered** за приложение за управление на студенти.

Предлагам да целим към:

```
StudentManagement/  
├── src/  
│   ├── StudentManagement.Web/  
│   │   ├── Controllers/  
│   │   ├── Views/  
│   │   ├── ViewModels/  
│   │   ├── Middleware/  
│   │   ├── Filters/  
│   │   ├── Extensions/  
│   │   └── wwwroot/  
│   ├── StudentManagement.Application/  
│   │   ├── Interfaces/  
│   │   ├── Services/  
│   │   ├── DTOs/  
│   │   └── Mapping/  
│   ├── StudentManagement.Domain/  
│   │   ├── Entities/  
│   │   ├── Enums/  
│   │   └── Constants/  
│   ├── StudentManagement.Infrastructure/  
│   │   ├── Data/  
│   │   ├── Configurations/  
│   │   ├── Repositories/  
│   │   └── Migrations/  
│   └── StudentManagement.Common/  
│       ├── Constants/  
│       ├── Extensions/  
│       └── Helpers/  
├── tests/  
│   ├── StudentManagement.UnitTests/  
│   └── StudentManagement.IntegrationTests/  
├── .gitignore  
├── README.md  
└── StudentManagement.sln
```

Как бих разделил отговорностите

StudentManagement.Domain

Тук са **самите бизнес обекти**:

```
Entities/  
├── Student.cs  
├── Course.cs  
├── Enrollment.cs  
└── Teacher.cs
```

Например Student не трябва да знае нищо за EF Core, MVC или HTTP.

```
public class Student  
{  
    public int Id { get; set; }  
  
    public string FirstName { get; set; } = null!;  
  
    public string LastName { get; set; } = null!;  
  
    public string Email { get; set; } = null!;  
}
```

StudentManagement.Application

Тук е **бизнес логиката**.

```
Interfaces/  
├── IStudentService.cs  
├── ICourseService.cs  
└── IEnrollmentService.cs
```

```
Services/  
├── StudentService.cs  
├── CourseService.cs  
└── EnrollmentService.cs
```

```
DTOs/  
├── StudentDto.cs  
├── CreateStudentDto.cs  
└── UpdateStudentDto.cs
```

```
Mapping/  
└── MappingProfile.cs
```

Например:

```
public interface IStudentService  
{  
    Task<IEnumerable<StudentDto>> GetAllAsync();  
  
    Task<StudentDto?> GetByIdAsync(int id);  
  
    Task CreateAsync(CreateStudentDto model);  
  
    Task UpdateAsync(UpdateStudentDto model);  
  
    Task DeleteAsync(int id);  
}
```

StudentManagement.Infrastructure

Тук е **всичко, което говори с database / EF Core**.

```
Data/  
├── StudentManagementDbContext.cs  
└── SeedData.cs
```

```
Configurations/  
├── StudentConfiguration.cs  
├── CourseConfiguration.cs  
└── EnrollmentConfiguration.cs
```

```
Repositories/
```

```
|— IStudentRepository.cs
|— StudentRepository.cs
|— ICourseRepository.cs
|— CourseRepository.cs
```

Migrations/

Това позволява на останалата част от приложението да не се занимава директно с EF Core.

StudentManagement.Web

Това е MVC приложението:

Controllers/

```
|— HomeController.cs
|— StudentsController.cs
|— CoursesController.cs
|— EnrollmentsController.cs
```

Views/

```
|— Home/
|— Students/
|— Courses/
|— Enrollments/
```

ViewModels/

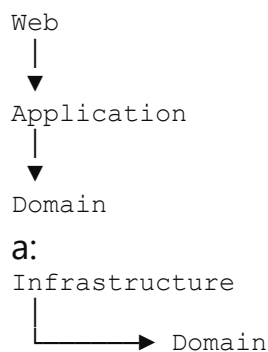
```
|— Student/
|— Course/
|— Enrollment/
```

Тук вече са HTTP/MVC нещата — controllers, views, view models и т.н.

Dependency правилото

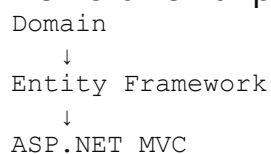
Това е най-важната част.

Искаме:



Тоест **Domain** да бъде най-независимият проект.

Не искаме например:



Това би обърнало архитектурата.

И бих го направил постепенно

Не бих ти казал:

„Изтрий всичко и създай тези 5 проекта.“

По-добрият вариант е:

Стъпка 1: разглеждаме сегашния StudentManagement.

Стъпка 2: определяме кой файл къде принадлежи.

Стъпка 3: създаваме новите projects.

Стъпка 4: местим Models → Domain/Entities.

Стъпка 5: местим services → Application.

Стъпка 6: отделяме EF Core → Infrastructure.

Стъпка 7: оправяме references и namespaces.

Стъпка 8: пускаме application-а и оправяме compile/runtime проблемите.

Стъпка 9: преглеждаме тестовете.

Стъпка 10: накрая оправяме README, naming, .gitignore и GitHub структурата.

Така накрая ще имаме **реално refactored** StudentManagement, а не само папки, които изглеждат професионално.

1. ок да рефакторираме това което имаме го сега
2. **ChatGPT каза:**

Да. Нека го направим **върху сегашния** StudentManagement, без да хвърляме работещия код и без да усложняваме проекта излишно. Ще го рефакторираме **стъпка по стъпка**, като след всяка стъпка проектът трябва да продължава да се компилира.

План

1. Първо правим snapshot на текущата структура

- projects
- папки
- класове
- dependencies/references

2. Подреждаме Domain

- entities
- enums
- constants

3. Изчистваме Application

- interfaces
- services
- DTOs
- mapping

4. Отделяме Infrastructure

- DbContext
- EF configurations
- repositories
- migrations

5. Подреждаме Web

- controllers
- view models
- views
- middleware/filters/extensions

6. Оправяме project references

- премахваме ненужните зависимости
- оставяме dependency flow-а чист

7. Оправяме namespaces и naming

8. Пускаме build + tests и оправяме всичко, което сме счупили.

Важно

Не бих започнал да местя файлове на сляпо, защото не искам да предполагам как изглежда **текущият** StudentManagement.

Ако проектът е в GitHub, **прати ми линка**

към StudentManagement **repository-то** (или към папката му), и ще започнем от реалната структура. Ще ти кажа конкретно:

ТОБА:

StudentManagement.X/Models/Student.cs

↓

СТАБА:

StudentManagement.Domain/Entities/Student.cs

и ще вървим **една промяна наведнъж**, за да не счупим проекта.

1. може ли да допълня да използваме Reflection And Attributes само там където се налага
2. ChatGPT каза:

Да, и това всъщност е добра идея. Можем да добавим **Reflection + Attributes**, но бих ги използвал **само там, където дават реална стойност**, а не за да изглежда проектът по-сложен.

За StudentManagement бих предложил следния принцип:

Къде да използваме Attributes

Например за **валидация/metadata**:

```
public class Student
{
    [Required]
    [MaxLength(50)]
    public string FirstName { get; set; } = null!;

    [Required]
    [MaxLength(50)]
    public string LastName { get; set; } = null!;

    [EmailAddress]
    public string Email { get; set; } = null!;
}
```

Тук Attributes имат смисъл, защото описват metadata за property-тата.

Къде Reflection би бил полезен

Можем например да направим малък общ механизъм за откриване на класове, маркирани със собствен attribute:

```
[AutoRegister]
public class StudentService : IStudentService
{
    // ...
}
```

и при стартиране да използваме Reflection, за да намерим всички класове с [AutoRegister] и да ги регистрираме.

Това е реален пример, в който Reflection има практическа стойност.

Но **няма** да правим нещо от типа:

```
typeof(Student)
    .GetProperties()
    .Where(...)
    .GetValue(...)
```

навсякъде само защото можем. Ако нормален C# код е по-ясен и по-бърз, ще използваме нормалния код.

Какво бих включил в проекта

При рефактора можем да имаме:

```
StudentManagement.Domain/
├── Entities/
├── Enums/
├── Attributes/
│   └── AutoRegisterAttribute.cs
└── Constants/
```

```
StudentManagement.Application/
├── Interfaces/
├── Services/
├── DTOs/
└── Mapping/
```

```
StudentManagement.Infrastructure/
├── Data/
├── Repositories/
├── DependencyInjection/
│   └── ServiceRegistration.cs
```

Например:

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class AutoRegisterAttribute : Attribute
{
}
```

След това Reflection логиката може да бъде концентрирана **на едно място**, вместо да замърсява бизнес логиката.

Така ще можем да покажем в проекта:

- OOP
- SOLID
- Dependency Injection
- Attributes
- Reflection
- Entity Framework Core
- async/await

- DTOs
- layered architecture
- unit/integration tests

без да превръщаме StudentManagement в излишно сложен framework.

Да — добавяме го като част от рефактора, но правилото ще бъде: *Reflection/Attributes* само когато решават конкретен проблем.

1. ок
2. ChatGPT каза:

Супер. 👍

Тогава сме се разбрали за посоката:

- **Рефакторираме съществуващия** StudentManagement, без да пренаписваме всичко от нулата.
- Подреждаме го в по-професионална архитектура.
- Запазваме функционалността, която вече имаме.
- Добавяме **Reflection + Attributes** само на подходящи места.
- След всяка по-голяма промяна проверяваме build и тестовете.
- Накрая ще направим и финален преглед на архитектурата и GitHub структурата.

Следващата стъпка е да видя

текущия StudentManagement **код/структура**, за да започнем с първата реална промяна, вместо да гадаем какво имаме. Прати ми GitHub линка към проекта и започваме.

1. <https://github.com/esp-nv/StudentManagement/tree/main>
2. ChatGPT каза:

Да, вече виждам **реалния ти** StudentManagement. Репозиторието е ASP.NET Core MVC и в момента е сравнително КОМПАКТНО: Controllers, Data, Migrations, Models, Views, wwwroot, Program.cs И

един .csproj.  GitHub+1

И това е важно: **не бих го разделял веднага на 5 проекта**. За сегашния размер ще направим професионален refactoring вътре в решението, а после, ако кодът се разрасне, можем да извадим Application/Infrastructure в отделни projects.

Предлагам първия refactoring

От това:

```

StudentManagement/
├── Controllers/
├── Data/
├── Migrations/
├── Models/
├── Views/
├── wwwroot/
├── Program.cs
└── StudentManagement.csproj

```

КЪМ:

```

StudentManagement/
├── Controllers/
│   ├── HomeController.cs
│   └── StudentsController.cs
├── Data/
│   ├── ApplicationDbContext.cs
│   ├── Configurations/
│   └── Repositories/
├── Models/
│   ├── Entities/
│   ├── DTOs/
│   └── ViewModels/
├── Services/
│   ├── Interfaces/
│   └── Implementations/
├── Attributes/
├── Extensions/
├── Migrations/
├── Views/
├── wwwroot/
├── Program.cs
└── StudentManagement.csproj

```

Но **преди да местим каквото и да е**, искам да видим съдържанието на Controllers, Data и Models, защото от структурата на GitHub не можем надеждно да определим кои класове каква отговорност имат.

И Reflection + Attributes

Ще ги включим, но контролирано:

```

Attributes/
└── AutoRegisterAttribute.cs

```

и например:

```

[AutoRegister]
public class StudentService : IStudentService
{
}

```

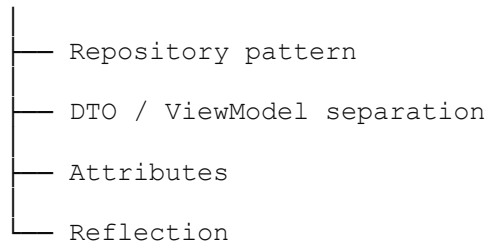
След това една Reflection-базирана регистрация в Extensions/, вместо да разпръскваме Reflection из целия проект.

Така ще можем реално да демонстрираме:

```

ASP.NET Core MVC
├── Services
│   └── Dependency Injection
└── Entity Framework Core

```



Следващата ни конкретна стъпка: да отворим Models и Data и да определим какво точно местим и как. Няма да измисляме класове, които проектът ти няма — ще рефакторираме **реалния код, който вече си написал**.